BCSE302P DATABASE SYSTEMS LAB · CASE STUDY 6

DBMS Project Query Explanation Guide

Railway Reservation and Passenger Service Management System — project structure and every query explained file by file, in simple language

---

Database: Oracle AI Database 26ai Free (release 23.26) · SQL and PL/SQL
Project files covered: parts/run_all.sql, 01_ddl.sql, 02_data.sql, 02b_seat.sql, 03_dml.sql, 04_queries.sql,
railway_case_study6.sql, tests/ (6 SQL suites, 2 Python scripts)
Outputs captured from the project database on 28 October 2026

Contents

How to Use This Guide 5
What is in each chapter 5
How each query is explained 5
A note about recent simplifications 5

1. Project Overview and File Inventory 6
1.1 What the project is about 6
1.2 Database technology and the evidence for it 6
1.3 File inventory 7
1.4 Execution order and why it matters 9
1.5 How the pieces work together 10

2. The Database Schema 11
2.1 Basic vocabulary you need first 11
2.2 Table-by-table explanation 11
2.3 Relationships and the ER diagram 17
2.4 Normalization and functional dependencies 18
2.5 The sample data 19

3. File 01_ddl.sql — Schema, Constraints, Indexes, Trigger 28
Q01 · Clean-up block (Part 0) 29
Q02 · CREATE TABLE station 31
Q03 · CREATE TABLE route 32
Q04 · CREATE TABLE route_stop 33
Q05 · CREATE TABLE train 34
Q06 · CREATE TABLE coach_class 35
Q07 · CREATE TABLE coach 36
Q08 · CREATE TABLE seat 37
Q09 · CREATE TABLE train_schedule 38
Q10 · CREATE TABLE passenger 39
Q11 · CREATE TABLE reservation 40
Q12 · Unique index ux_res_seat_journey 42
Q13 · Unique index ux_res_pass_journey 43
Q14 · Supporting indexes 44
Q15 · CREATE TABLE payment 46
Q16 · CREATE TABLE cancellation 47
Q17 · Sequences 48
Q18 · Trigger trg_reservation_validate 49

4. Files 02_data.sql and 02b_seat.sql — Loading the Sample Data 51
Q19 · INSERT stations (2.1) 52
Q20 · INSERT routes and route stops (2.2) 53
Q21 · INSERT coach classes (2.3) 54
Q22 · INSERT trains (2.4) 55
Q23 · INSERT coaches (2.5) 56
Q24 · Seat generator: INSERT … SELECT with CONNECT BY (2.6 and 02b_seat.sql) 57
Q25 · INSERT train schedules (2.7) 59
Q26 · INSERT passengers (2.8) 61
Q27 · INSERT reservations (2.9) 62

5. File 03_dml.sql — Modification, Integrity Tests and Stored Procedures 63
Q28 · Generate payments with INSERT … SELECT (2.10) 64
Q29 · Generate cancellations with INSERT … SELECT (2.11) 65
Q30 · Transaction control in the scripts (COMMIT and ROLLBACK) 66
Q31 · Row counts with UNION ALL 67

Q32 · ALTER TABLE passenger ADD concession_type (3.1) 68
Q33 · ALTER TABLE station MODIFY (3.2) 69
Q34 · ALTER TABLE passenger RENAME COLUMN (3.3) 70
Q35 · ALTER TABLE train ADD CONSTRAINT (3.4) 71
Q36 · UPDATE concession from age (4.1) 72
Q37 · UPDATE train timetable (4.2) 73
Q38 · Find a PNR into a bind variable (4.3) 74
Q39 · Before-and-after check of the ticket and payment (4.3) 75
Q40 · UPDATE boarding station and recalculate fare (4.3) 76
Q41 · UPDATE payment using a scalar subquery (4.3) 77
Q42 · INSERT then DELETE a test passenger (4.4) 78
Q43–Q49 · Integrity tests 5.1–5.7 (expected failures) 78
Q43 · Test 5.1 — invalid phone 79
Q44 · Test 5.2 — double seat booking 80
Q45 · Test 5.3 — seat of another train 81
Q46 · Test 5.4 — wrong direction 82
Q47 · Test 5.5 — confirmed ticket without a seat 83
Q48 · Test 5.6 — delete a parent row 84
Q49 · Test 5.7 — unknown station 85
Q50 · Procedure book_ticket (6.1) 86
Q51 · Procedure cancel_ticket (6.2) 89
Q52 · Demo reset and new journey 130 (6.3) 92
Q53 · Demo: nine bookings in a loop (6.3) 93
Q54 · Demo: cancel and auto-promote (6.4) 94
Q55 · Demo result: six-table join (6.4) 95

6. File 04_queries.sql — Views, Analysis Queries and the Capacity-Risk Module 96

Q56 · VIEW v_reservation_performance 97
Q57 · VIEW v_train_utilization 99
Q58 · VIEW v_ticket_details 100
Q59 · Querying the two summary views 101
Q60 · Q1 Train-wise reservation statistics 102
Q61 · Q2 Station-wise passenger volume 103
Q62 · Q3 Route utilization 104
Q63 · Q4 Coach occupancy 105
Q64 · Q5a Cancellation statistics by train 106
Q65 · Q5b Cancellation reasons 107
Q66 · Q6a Peak travel routes (DENSE_RANK) 108
Q67 · Q6b Top 5 origin–destination pairs 109
Q68 · Q6c Peak travel days of the week 110
Q69 · Q7 Highest and lowest utilized trains 111
Q70 · Q8 Passenger travel frequency 112
Q71 · Q9 Revenue by train with ROLLUP 113
Q72 · Q10 Routes requiring additional capacity (HAVING) 114
Q73 · A1 Ten-table join: a passenger's travel history 115
Q74 · A2 Correlated subquery: trains above their route's average 116
Q75 · A3 Passengers with repeated cancellations 117
Q76 · A4 Nested subquery: routes above average demand 118
Q77 · A5 WITH clause: stations above average traffic 119
Q78 · A6 NOT EXISTS + EXISTS: trains needing capacity 120
Q79 · A7 Correlated HAVING: above-average spenders per city 121
Q80 · A8 IN and NOT IN: loyal passengers 122
Q81 · A9 Correlated subquery on a CTE: under-used coaches 123
The Capacity-Risk Module (Part 10, BTL-6 challenge) 123
Q82 · CREATE TABLE risk_threshold and its rows 125
Q83 · CREATE TABLE capacity_risk_log 126
Q84 · FUNCTION fn_capacity_risk 127
Q85 · 10.1 Decision-support report 128

Q86 · PROCEDURE prc_capacity_risk_report (10.2) 129
Q87 · 10.3 What-if analysis with ROLLBACK 130

7. Runner Script and SQL*Plus Commands 131

R1 · parts/run_all.sql — the master runner 132

How to Use This Guide

This guide explains the **Railway Reservation and Passenger Service Management System** (BCSE302P Case Study 6) in simple language: first the structure of the project and its database, then **every query, file by file**.

What is in each chapter

Chapter	What you will learn
1. Project Overview and File Inventory	What the project does, every file, the database technology, and the order in which files run
2. Database Schema	Every table, column, key and constraint; the ER diagram; the sample data
3. File 01_ddl.sql	Creating the tables, indexes, sequences and the validation trigger (Q01–Q18)
4. Files 02_data.sql and 02b_seat.sql	Loading the sample data (Q19–Q27)
5. File 03_dml.sql	Payments, cancellations, ALTER, UPDATE, DELETE, integrity tests, booking and cancellation procedures (Q28–Q55)
6. File 04_queries.sql	Views, reports Q1–Q10, advanced queries A1–A9, capacity-risk module (Q56–Q87)
7. Runner Script and SQL*Plus Commands	run_all.sql, the script that runs the whole project

Viva questions, and a short answer to say to the examiner for every query, are in the separate **Viva Questions** PDF.

How each query is explained

1. **The query:** the exact code from your project file, with file name and line numbers.
2. **What it is for:** one line, then a short description.
3. **Simple explanation:** what the query does, in plain everyday words.
4. **Concepts used:** the DBMS ideas behind it.
5. **Things to watch out for:** common mistakes and special cases.

A note about recent simplifications

On 28 October 2026 a few queries were simplified without changing their results: Q9 (revenue report), Q6b (top-5 segments), the seat and waitlist lookups inside `book_ticket` and `cancel_ticket`, and the status counters in Q1 and `v_reservation_performance`. Every changed query was run against the database and returned exactly the same rows as before. This guide shows the **current** code in your files.

1. Project Overview and File Inventory

1.1 What the project is about

Indian Railways sells millions of tickets every day. Behind every ticket the system has to answer questions such as: Which train runs on which route? Which seats are free on the 21 October journey of the Brindavan Express? How much does a ticket from Katpadi to Bengaluru cost in AC Chair Car? Who is next on the waitlist when someone cancels?

This project builds the **database** that answers those questions. It stores:

- **Network data**: stations, routes, and the ordered stops on each route.
- **Rolling stock**: trains, their coaches, the travel class of each coach, and every individual seat or berth.
- **Operations**: dated runs of each train (a *journey* or *schedule*), registered passengers, tickets (reservations with a PNR), payments and cancellations.
- **Decision support**: thresholds and a daily log that classify each train as *CAPACITY REQUIRED*, *HIGH DEMAND* or *NORMAL*.

The project solves five practical problems:

1. **No seat can be sold twice** on the same journey. A unique index enforces this inside the database, so it cannot be bypassed.
2. **Fares are always calculated correctly**: `GREATEST(distance, 50) × fare_per_km + reservation_fee`, computed by a trigger, so nobody can type in a wrong fare by mistake.
3. **Booking and cancellation are automatic**: the `book_ticket` procedure gives the first free seat or puts the passenger on the waitlist; `cancel_ticket` calculates the refund and promotes the first waitlisted passenger.
4. **Management reports**: occupancy, cancellations, revenue, peak routes and stations.
5. **Capacity planning**: a module that flags trains needing extra coaches.

There is **no separate application layer**: no web app or Java/Python program builds SQL from user input. All business logic lives inside the database (constraints, a trigger, procedures and a function). A "user" interacts by calling the procedures, for example `book_ticket(...)`, or by running the reports. The two Python files in `tests/` only send SQL scripts to SQL*Plus for automated testing.

1.2 Database technology and the evidence for it

The project uses **Oracle Database** with **SQL and PL/SQL**. Your database reports itself as *Oracle AI Database 26ai Free Release 23.26.2.0.0*. The README says it was tested on Oracle 23ai/26ai Free.

The code itself confirms this. These features exist only in Oracle (or are written in Oracle's own syntax):

Evidence in the code	Why it identifies Oracle
`VARCHAR2`, `NUMBER(5,2)` data types	Oracle's own types; MySQL uses VARCHAR/DECIMAL

Evidence in the code	Why it identifies Oracle
`CREATE OR REPLACE TRIGGER … FOR EACH ROW` with `:NEW`	Oracle PL/SQL trigger syntax
`EXECUTE IMMEDIATE`, `RAISE_APPLICATION_ERROR`, `DBMS_OUTPUT.PUT_LINE`	PL/SQL built-ins
`DECODE`, `NVL`, `ROWNUM`, `CONNECT BY LEVEL`, `RATIO_TO_REPORT`, `LISTAGG … WITHIN GROUP`	Oracle SQL functions and pseudo-columns
`SEQUENCE … NEXTVAL`, `user_tables`, `dual`	Oracle sequences, data dictionary and the DUAL table
`SET SERVEROUTPUT ON`, `COLUMN … FORMAT`, `PROMPT`, `@@file`, `/`	SQL*Plus client commands
`DROP TABLE … CASCADE CONSTRAINTS PURGE`	Oracle-specific DROP options

Oracle behaviour that affects how this project runs:

- **DDL commits automatically.** Every CREATE, ALTER or DROP first commits any pending work, then runs, then commits again. You cannot roll back a CREATE TABLE.
- **The empty string `''` is NULL in Oracle.** This matters for CHECK constraints and comparisons.
- **Unquoted names are stored in UPPERCASE.** That is why the clean-up block looks for `'STATION'`, not `'station'`.
- **Dates always contain a time.** `SYSDATE` includes hours, minutes and seconds, which is why the code uses `TRUNC(SYSDATE)` to remove the time.
- **Read consistency.** A query sees data as it was when the query started, so readers never block writers. This matters for the concurrency design of `book_ticket`.
- **NLS settings.** `TO_CHAR(date,'DY')` in Q6c returns day names in the session's language, and the scripts set `NLS_DATE_FORMAT = 'DD-MON-YYYY'`.

1.3 File inventory

File	Type	Purpose	Queries / programs	Concepts covered	Depends on
`parts/run_all.sql`	SQL*Plus script	Master runner: runs the five part files in order	SET, PROMPT, `@@` commands	Script orchestration, session settings	All part files
`parts/01_ddl.sql`	SQL + PL/SQL	Clean-up, 12 core tables, indexes, sequences, validation trigger	Q01–Q18	DDL, constraints, keys, function-based indexes, sequences, triggers, dynamic SQL	None (runs first)
`parts/02_data.sql`	SQL (DML)	Loads sample data: 14 stations, 4 routes, 21 stops, 6 classes, 9 trains, 30 coaches, 320 seats, 27 journeys, 150 passengers, 768 tickets	Q19–Q27	INSERT, INSERT… SELECT, row generators, trigger firing	01_ddl.sql

File	Type	Purpose	Queries / programs	Concepts covered	Depends on
`parts/02b_seat.sql`	SQL (DML)	Re-runnable seat generator (inserts only missing seats)	Q24 (variant)	NOT EXISTS, idempotency	01_ddl.sql, coach data
`parts/03_dml.sql`	SQL + PL/SQL	Payments, cancellations, ALTER, UPDATE/DELETE, constraint tests, `book_ticket`, `cancel_ticket`, demo	Q28–Q55	DML, TCL, ALTER, exception handling, procedures, locking, bind variables	01, 02
`parts/04_queries.sql`	SQL + PL/SQL	Views, analysis queries Q1–Q10, advanced A1–A9, capacity-risk module	Q56–Q87	Views, joins, subqueries, GROUP BY/HAVING, window functions, ROLLUP, functions, procedures, what-if analysis	01, 02, 03 (needs `home_city`, payments, cancellations)
`railway_case_study6.sql`	SQL + PL/SQL	The same code as the four part files merged into one file (only header comments differ)	Same as Q01–Q87	Same	None
`tests/01_schema_validation.sql`	PL/SQL test	Checks that all objects exist and are VALID	T1	Data dictionary, assertions	Full project run
`tests/02_data_invariants.sql`	PL/SQL test	12 business-rule checks on the data	T2	Invariants, NOT EXISTS, HAVING	Full project run
`tests/03_booking_workflow_test.sql`	PL/SQL test	Booking, waitlist and error-code tests	T3	Procedure testing, exceptions, ROLLBACK	book_ticket
`tests/04_cancellation_refund_test.sql`	PL/SQL test	Refund tiers and waitlist promotion	T4	Procedure testing	cancel_ticket
`tests/05_concurrency_test.sql`	PL/SQL test	Locks and unique-index safeguards	T5	FOR UPDATE NOWAIT, DUP_VAL_ON_INDEX	book_ticket, indexes
`tests/06_queries_verification.sql`	PL/SQL test	Checks view results and the risk function	T6	View verification	04_queries.sql
`tests/run_all_tests.sql`	SQL*Plus script	Runs the six suites	`@@` commands	Orchestration	All suites
`tests/run_tests.py`	Python	Runs `run_all_tests.sql` through sqlplus and counts PASS/FAIL lines	none (no SQL of its own)	Test automation	sqlplus
`tests/test_concurrency_race.py`	Python	Two threads try to book the last seat at the same time	Setup and verification SQL	Concurrency, race conditions	book_ticket
`README.md`, `DATABASE_DOCUMENTATION.md`, `TEST_PLAN.md`, `AUDIT_REPORT.md`, `CHANGELOG_AUDIT.md`	Documentation	Project description, data dictionary, test plan, audit history	none	—	—

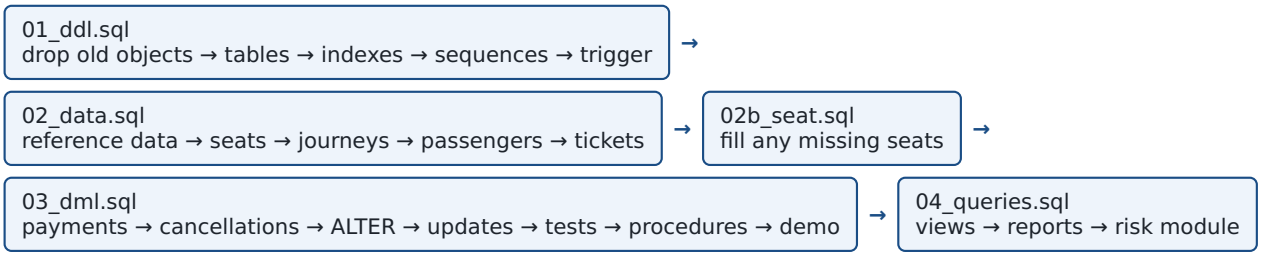
File	Type	Purpose	Queries / programs	Concepts covered	Depends on
`er_diagram.png` / `.svg`	Image	Entity-Relationship diagram (used in Chapter 2)	none	ER modelling	—
`report.docx`, `report.html`	Report	Academic lab report	none	—	—
`run_output.txt`	Text	Saved output of an earlier complete run (used in this guide for messages such as error texts)	none	—	—

WHAT THE PROJECT DOES NOT CONTAIN

So you can answer honestly if asked: there is **no application code that builds SQL from user input** (so no SQL-injection surface in an app layer), **no SAVEPOINT**, **no GRANT/REVOKE (DCL)**, **no synonyms**, **no explicitly declared named cursors** (only implicit cursor FOR loops), **no MERGE**, and **no SET TRANSACTION ISOLATION LEVEL**. The README says the 12 core tables are in BCNF, but the documentation does not list the functional dependencies; Chapter 2 derives them from the schema.

1.4 Execution order and why it matters

The project must run in this order. `run_all.sql` does exactly this:



Why the order matters (real dependencies in the code):

1. **Tables before data.** An INSERT into a table that does not exist fails with ORA-00942.
2. **Parents before children.** Foreign keys mean `station` must be filled before `route`, `route` before `route_stop` and `train`, `train` before `coach` and `train_schedule`, and so on. Inserting a child row whose parent does not exist fails with ORA-02291 (*parent key not found*).
3. **The trigger before the tickets.** `trg_reservation_validate` is created in 01_ddl.sql. The 768 ticket INSERTs in 02_data.sql do **not** supply a fare; the trigger calculates it. Without the trigger every ticket would have a NULL fare.
4. **Tickets before payments and cancellations.** 03_dml.sql builds payments and cancellations *from* the reservation rows with `INSERT … SELECT`.
5. **ALTER before code that uses the new names.** Part 3.3 renames `passenger.city` to `home_city`. The INSERT in Part 4.4 and queries Q8, A7 and A8 refer to `home_city`, so they only work after the rename.
6. **Procedures before the demo.** The demo in Parts 6.3–6.4 calls `book_ticket` and `cancel_ticket`, so they must already exist.

7. **Views before the reports.** Q3, Q7, Q8, Q10, A2, A6, A8 and the risk module all read the views.
8. **The function before the procedure.** `prc_capacity_risk_report` calls `fn_capacity_risk`.

There are **no circular dependencies**. Because the clean-up block runs first, the whole sequence can be repeated any number of times.

ONE SUBTLE PREREQUISITE

`02b_seat.sql` and the seat INSERT in `02_data.sql` produce the same 320 seats. The 02b version only inserts seats that do not already exist (`WHERE NOT EXISTS`), so running it after 02_data.sql adds 0 rows. It is a safety net, not a second copy.

1.5 How the pieces work together

Think of the project in four layers:

1. **Structure (01_ddl.sql)**: the tables and the rules that can never be broken (constraints, unique indexes, the trigger).
2. **Data (02_data.sql, 02b_seat.sql)**: a realistic snapshot, with two completed journeys and one upcoming journey per train.
3. **Behaviour (03_dml.sql)**: modifications, proof that bad data is rejected, and the two stored procedures that act like the booking counter.
4. **Information (04_queries.sql)**: views that turn raw tickets into business measures, reports that answer management questions, and the risk module that recommends action.

A typical "user story" across the layers: a passenger calls `book_ticket` (layer 3). The procedure inserts a reservation, and the trigger (layer 1) validates the route and calculates the fare. Later the view `v_reservation_performance` (layer 4) counts that ticket towards the journey's occupancy, and the risk module may flag the train as needing more capacity.

2. The Database Schema

A **schema** is the set of tables and other objects that belong to one database user. Before studying individual queries, you need to know what each table stores and how the tables connect. Every query in this project is just a way of combining these tables.

2.1 Basic vocabulary you need first

- **Table (relation)**: a grid of rows and columns. Each row is one thing (one station, one ticket).
- **Column (attribute)**: one property of the thing, such as `station_name`.
- **Data type**: what kind of value a column holds. `VARCHAR2(60)` is text up to 60 bytes; `NUMBER(5)` is a whole number of up to 5 digits; `NUMBER(5,2)` is up to 5 digits with 2 after the decimal point (max 999.99); `CHAR(1)` is exactly one character, padded with spaces; `DATE` stores a date **and** a time down to the second.
- **NULL**: "no value / unknown". It is not zero and not an empty string. A comparison with NULL (`x = NULL`) is never true; you must write `x IS NULL`.
- **Primary key (PK)**: the column(s) that uniquely identify each row. Automatically NOT NULL and UNIQUE. Oracle creates a unique index for it.
- **Foreign key (FK)**: a column whose value must exist as a primary or unique key in another (parent) table. This is **referential integrity**: no ticket can point to a passenger who does not exist.
- **UNIQUE**: no two rows may share the value. Unlike a PK, it allows NULLs, and multiple NULLs are allowed.
- **NOT NULL**: the column must always have a value.
- **CHECK**: a condition every row must satisfy, e.g. `platforms BETWEEN 1 AND 30`. A CHECK passes when the condition is TRUE **or UNKNOWN** (because of NULL). It fails only when it is FALSE.
- **DEFAULT**: the value used when an INSERT does not mention the column.
- **ON DELETE CASCADE**: when a parent row is deleted, the matching child rows are deleted automatically.

2.2 Table-by-table explanation

STATION: every station on the network

Column	Type	NULL?	Constraint / default	Meaning
station_code	VARCHAR2(5)	No	**PK** `pk_station`; CHECK `ck_station_code`: must equal `UPPER(station_code)`	Short code such as `MAS`, `SBC`
station_name	VARCHAR2(60), widened to 80 by ALTER in 03_dml	No	NOT NULL	Full name, e.g. *Chennai Central*
city	VARCHAR2(40)	No	NOT NULL	City the station serves
state	VARCHAR2(30)	No	NOT NULL	State
rail_zone	VARCHAR2(5)	No	NOT NULL	Railway zone, e.g. `SR`

Column	Type	NULL?	Constraint / default	Meaning
platforms	NUMBER(2)	No	DEFAULT 1; CHECK `ck_station_pf` 1–30	Number of platforms

The upper-case check guarantees that `'mas'` and `'MAS'` can never both exist as different stations.

ROUTE: a named corridor between two stations

Column	Type	NULL?	Constraint	Meaning
route_id	NUMBER(4)	No	**PK** `pk_route`	Route number
route_name	VARCHAR2(60)	No	NOT NULL, **UNIQUE** `uq_route_name`	e.g. *Chennai - Bengaluru*
origin_code	VARCHAR2(5)	No	**FK** `fk_route_origin` → station	Starting station
dest_code	VARCHAR2(5)	No	**FK** `fk_route_dest` → station	Final station
distance_km	NUMBER(5)	No	CHECK `ck_route_km` > 0	Total length
—	—	—	CHECK `ck_route_ends` : origin_code <> dest_code	A route cannot start and end at the same station

ROUTE references STATION **twice** (two different foreign keys to the same parent). Queries that need both city names must join STATION twice with different aliases.

ROUTE_STOP: the ordered stations on each route

Column	Type	NULL?	Constraint	Meaning
route_id	NUMBER(4)	No (part of PK)	**FK** `fk_rs_route` → route **ON DELETE CASCADE**	Which route
stop_seq	NUMBER(2)	No (part of PK)	CHECK `ck_rs_seq` >= 1	Position of the stop: 1 = origin
station_code	VARCHAR2(5)	No	**FK** `fk_rs_station` → station	Which station
km_from_origin	NUMBER(5)	No	CHECK `ck_rs_km` >= 0	Distance from the start of the route
—	—	—	**PK** `pk_route_stop` (route_id, stop_seq); **UNIQUE** `uq_rs_station` (route_id, station_code)	One station per position; a station appears only once per route

This is an **associative (junction) table**. A route has many stations and a station lies on many routes, a many-to-many (M:N) relationship. Relational tables cannot store M:N directly, so ROUTE_STOP holds one row per (route, station) pair, plus the extra facts *position* and *distance*. The trigger uses `stop_seq` to check travel direction and `km_from_origin` to compute distance.

TRAIN: a train service that always runs on one route

Column	Type	NULL?	Constraint / default	Meaning
train_no	NUMBER(5)	No	**PK** `pk_train`; CHECK `ck_train_no` 10000–99999	Five-digit train number
train_name	VARCHAR2(50)	No	NOT NULL; **UNIQUE** `uq_train_name` (added later by ALTER)	e.g. *Brindavan Express*
train_type	VARCHAR2(15)	No	CHECK `ck_train_type` IN (SUPERFAST, EXPRESS, SHATABDI, TEJAS, INTERCITY, MAIL)	Category
route_id	NUMBER(4)	No	**FK** `fk_train_route` → route	The route it runs on

Column	Type	NULL?	Constraint / default	Meaning
dep_time	VARCHAR2(5)	No	CHECK `ck_train_dep`: `REGEXP_LIKE(dep_time,'^([01][0-9]|2[0-3]):[0-5][0-9]$')`	Departure time as text `HH:MM`
run_days	VARCHAR2(7)	No	DEFAULT 'YYYYYYY'; CHECK `^[YN]{7}$`	One Y/N flag per weekday
is_active	CHAR(1)	No	DEFAULT 'Y'; CHECK IN ('Y','N')	Whether the service is active

The regular expression for `dep_time` accepts `00:00`–`23:59` only. `[01][0-9]` matches hours 00–19, `2[0-3]` matches 20–23, `[0-5][0-9]` matches minutes; `^` and `$` anchor the whole string.

COACH_CLASS: travel classes and fare rules

Column	Type	NULL?	Constraint	Meaning
class_code	VARCHAR2(3)	No	**PK** `pk_coach_class`	`1A`, `2A`, `3A`, `SL`, `CC`, `EC`
class_name	VARCHAR2(30)	No	NOT NULL	e.g. *AC Chair Car*
seats_per_coach	NUMBER(3)	No	CHECK 1–100	Seats in one coach of this class (scaled down for the demo)
fare_per_km	NUMBER(5,2)	No	CHECK > 0	Rupees per km
reservation_fee	NUMBER(5)	No	CHECK >= 0	Flat fee per ticket

COACH: a physical coach attached to a train

Column	Type	NULL?	Constraint	Meaning
coach_id	NUMBER(5)	No	**PK** `pk_coach`	Surrogate key
train_no	NUMBER(5)	No	**FK** `fk_coach_train` → train **ON DELETE CASCADE**	Owning train
coach_label	VARCHAR2(4)	No	NOT NULL	Label painted on the coach, e.g. `C1`, `E1`, `S1`
class_code	VARCHAR2(3)	No	**FK** `fk_coach_class` → coach_class	Class of all seats in the coach
—	—	—	**UNIQUE** `uq_coach_label` (train_no, coach_label)	Two coaches of the same train cannot both be `C1`

The DDL comment calls COACH a *weak entity identified by train + label*: in real life a coach is identified by its train and label. The project gives it a surrogate key `coach_id` for convenience and keeps the natural identity as a UNIQUE constraint.

SEAT: every seat or berth

Column	Type	NULL?	Constraint	Meaning
seat_id	NUMBER(7)	No	**PK** `pk_seat`	Generated as `coach_id × 100 + seat_no`
coach_id	NUMBER(5)	No	**FK** `fk_seat_coach` → coach **ON DELETE CASCADE**	Owning coach
seat_no	NUMBER(3)	No	CHECK >= 1	Seat number inside the coach
berth_type	VARCHAR2(12)	No	CHECK IN (LOWER, MIDDLE, UPPER, SIDE LOWER, SIDE UPPER, WINDOW, AISLE, MIDDLE SEAT)	Kind of berth/seat
—	—	—	**UNIQUE** `uq_seat` (coach_id, seat_no)	No duplicate seat numbers in a coach

TRAIN_SCHEDULE: one actual run of a train on a date

Column	Type	NULL?	Constraint / default	Meaning
schedule_id	NUMBER(6)	No	**PK** `pk_schedule`	Journey number (101, 102, ...)
train_no	NUMBER(5)	No	**FK** `fk_sched_train` → train	Which train
journey_date	DATE	No	CHECK `ck_sched_date`: `journey_date = TRUNC(journey_date)`	Date of travel, with no time part
sched_status	VARCHAR2(10)	No	DEFAULT 'SCHEDULED'; CHECK IN (SCHEDULED, DEPARTED, COMPLETED, CANCELLED)	Lifecycle state
—	—	—	**UNIQUE** `uq_sched` (train_no, journey_date)	A train runs at most once per date

`TRUNC(date)` sets the time to midnight. The CHECK forbids storing `21-OCT-2026 07:50`, so that `uq_sched` really means "once per day". Otherwise two rows on the same day with different times would both be allowed.

PASSENGER: registered travellers

Column	Type	NULL?	Constraint / default	Meaning
passenger_id	NUMBER(6)	No	**PK** `pk_passenger`	Passenger number
full_name	VARCHAR2(60)	No	NOT NULL	Name
gender	CHAR(1)	No	CHECK IN ('M','F','T')	Gender
dob	DATE	No	NOT NULL	Date of birth
phone	VARCHAR2(10)	No	**UNIQUE** `uq_pass_phone`; CHECK `^[6-9][0-9]{9}$`	Indian mobile number: 10 digits starting 6–9
email	VARCHAR2(60)	**Yes**	**UNIQUE** `uq_pass_email`; CHECK e-mail pattern	Optional e-mail
city → **home_city**	VARCHAR2(40)	Yes	renamed by ALTER in 03_dml	Home city
registered_on	DATE	No	DEFAULT SYSDATE	Registration date
concession_type	VARCHAR2(10)	No	**added by ALTER**; DEFAULT 'NONE'; CHECK IN (NONE, SENIOR, STUDENT, DIVYANG)	Concession category

`email` is nullable **and** unique: many passengers may have no e-mail (NULLs do not clash with each other), but two passengers cannot share the same e-mail. The phone is stored as text, not a number, because it is an identifier, not something you do arithmetic on.

RESERVATION: one ticket (PNR)

Column	Type	NULL?	Constraint / default	Meaning
pnr	NUMBER(10)	No	**PK** `pk_reservation`	Passenger Name Record number
passenger_id	NUMBER(6)	No	**FK** `fk_res_pass` → passenger	Who travels
schedule_id	NUMBER(6)	No	**FK** `fk_res_sched` → train_schedule	Which journey
class_code	VARCHAR2(3)	No	**FK** `fk_res_class` → coach_class	Booked class
seat_id	NUMBER(7)	**Yes**	**FK** `fk_res_seat` → seat	Allotted seat; NULL while waitlisted or after cancelling

Column	Type	NULL?	Constraint / default	Meaning
from_station	VARCHAR2(5)	No	**FK** `fk_res_from` → station	Boarding station
to_station	VARCHAR2(5)	No	**FK** `fk_res_to` → station	Destination
booking_date	DATE	No	DEFAULT TRUNC(SYSDATE)	Day of booking
fare	NUMBER(8,2)	Yes (filled by trigger)	CHECK `ck_res_fare` > 0	Ticket price
status	VARCHAR2(10)	No	DEFAULT 'CONFIRMED'; CHECK IN (CONFIRMED, WAITLIST, CANCELLED, COMPLETED)	Ticket state
—	—	—	CHECK `ck_res_stations`: from_station <> to_station	Cannot travel from a station to itself
—	—	—	CHECK `ck_res_seat_status` (see below)	Status and seat must agree

`ck_res_seat_status` says: *(status is CONFIRMED or COMPLETED **and** a seat is set) **or** (status is WAITLIST or CANCELLED **and** no seat is set)*. A confirmed ticket without a seat, or a waitlisted ticket holding a seat, is rejected. Test 5.5 in 03_dml.sql proves this.

RESERVATION also has two **function-based unique indexes** (explained in detail as Q12 and Q13):

- `ux_res_seat_journey`: the same seat cannot be sold twice on the same journey.
- `ux_res_pass_journey`: a passenger cannot hold two active (non-cancelled) tickets on the same journey.

PAYMENT: exactly one payment per ticket

Column	Type	NULL?	Constraint / default	Meaning
payment_id	NUMBER(8)	No	**PK** `pk_payment`	Payment number
pnr	NUMBER(10)	No	**UNIQUE** `uq_pay_pnr`; **FK** `fk_pay_res` → reservation **ON DELETE CASCADE**	Ticket paid for
amount	NUMBER(8,2)	No	CHECK > 0	Amount paid
pay_mode	VARCHAR2(10)	No	CHECK IN (UPI, CARD, NETBANKING, WALLET, CASH)	Payment method
paid_on	DATE	No	NOT NULL	Payment date
pay_status	VARCHAR2(14)	No	DEFAULT 'SUCCESS'; CHECK IN (SUCCESS, REFUNDED, PARTIAL_REFUND)	Refund state

A foreign key alone gives a one-to-many relationship. Adding **UNIQUE on pnr** limits it to at most one payment per ticket, which turns it into **one-to-one**.

CANCELLATION: at most one cancellation per ticket

Column	Type	NULL?	Constraint / default	Meaning
cancel_id	NUMBER(8)	No	**PK** `pk_cancellation`	Cancellation number
pnr	NUMBER(10)	No	**UNIQUE** `uq_cancel_pnr`; **FK** `fk_cancel_res` → reservation **ON DELETE CASCADE**	Cancelled ticket
cancelled_on	DATE	No	NOT NULL	Date of cancellation

Column	Type	NULL?	Constraint / default	Meaning
refund_amount	NUMBER(8,2)	No	DEFAULT 0; CHECK >= 0	Money returned
reason	VARCHAR2(60)	Yes	—	Reason given

The relationship is **1 : 0..1**: every cancellation has exactly one ticket, but most tickets have no cancellation.

RISK_THRESHOLD and CAPACITY_RISK_LOG: decision-support tables (created in 04_queries.sql)

Table	Column	Type	Constraint	Meaning
RISK_THRESHOLD	param_code	VARCHAR2(20)	**PK**	Name of the rule parameter, e.g. `CR_MIN_OCCUPANCY`
	param_value	NUMBER(6,2)	NOT NULL, CHECK >= 0	Threshold value
	description	VARCHAR2(100)	NOT NULL	Explanation
CAPACITY_RISK_LOG	run_date	DATE	NOT NULL, part of **PK**	Day of the classification run
	train_no	NUMBER(5)	**FK** → train, part of **PK**	Train classified
	occupancy_pct, cancellation_rate, demand_pct	NUMBER(6,2)	nullable	Measures at the time of the run
	risk_status	VARCHAR2(20)	NOT NULL	CAPACITY REQUIRED / HIGH DEMAND / NORMAL

Storing the thresholds in a table, instead of hard-coding them, lets management change a rule with one UPDATE and no code change. Section 10.3 of the project demonstrates exactly this with its "what-if" analysis.

2.3 Relationships and the ER diagram

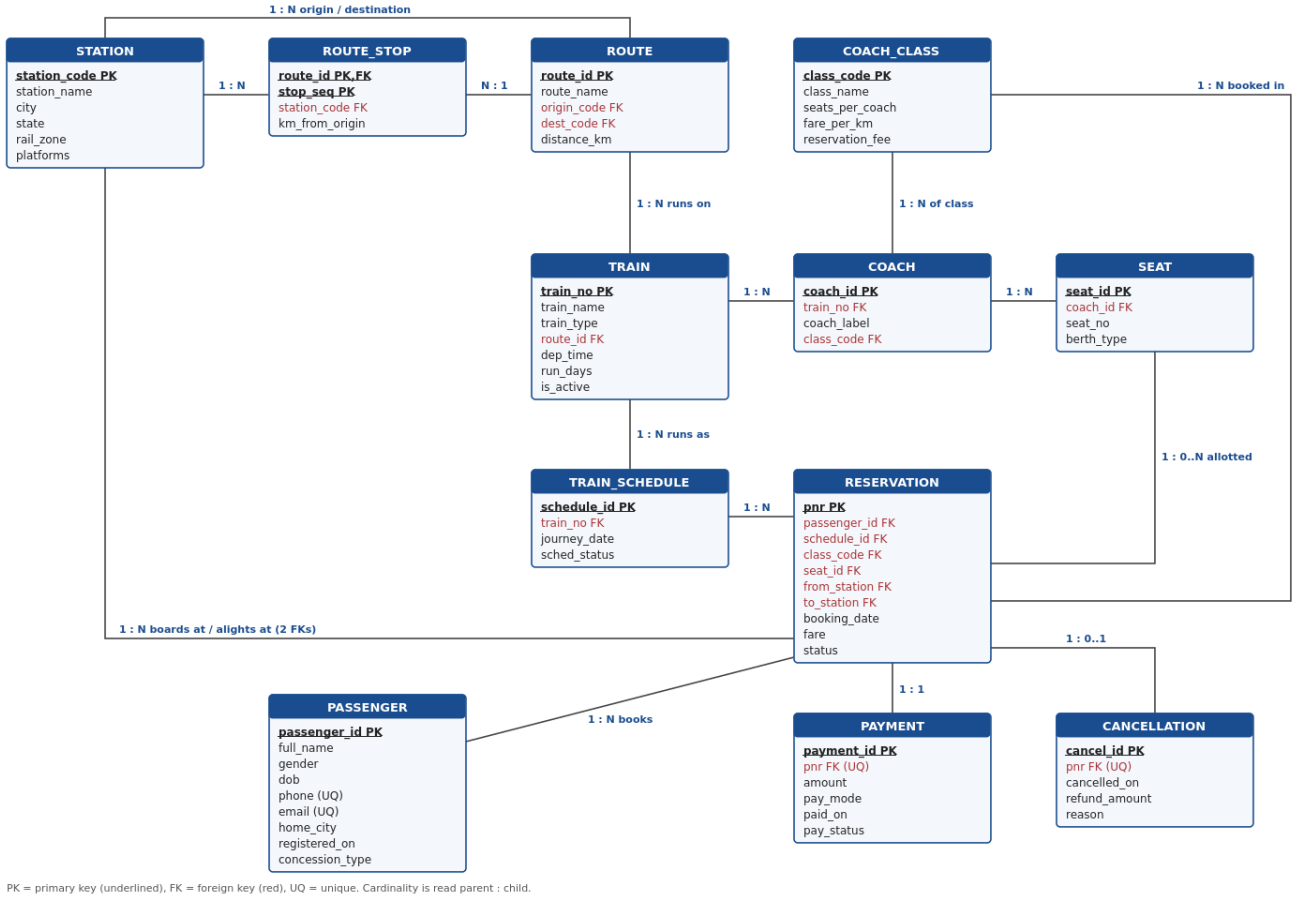


Figure 2.1: Entity-Relationship diagram from the project (er_diagram.png). PK = underlined, FK = red, UQ = unique. Cardinality is read parent : child.

Parent	Child	Cardinality	Implemented by	Meaning
STATION	ROUTE	1 : N (twice)	`origin_code`, `dest_code` FKs	A station can start or end many routes
ROUTE ↔ STATION	ROUTE_STOP	M : N resolved	composite PK + two FKs	Routes pass through many stations and vice versa
ROUTE	TRAIN	1 : N	`train.route_id`	Many trains share a route
TRAIN	COACH	1 : N	`coach.train_no` (cascade)	A train has many coaches
COACH_CLASS	COACH	1 : N	`coach.class_code`	Many coaches of each class
COACH	SEAT	1 : N	`seat.coach_id` (cascade)	A coach has many seats
TRAIN	TRAIN_SCHEDULE	1 : N	`train_schedule.train_no`	A train runs on many dates
PASSENGER	RESERVATION	1 : N	`reservation.passenger_id`	A passenger books many tickets
TRAIN_SCHEDULE	RESERVATION	1 : N	`reservation.schedule_id`	A journey carries many tickets
COACH_CLASS	RESERVATION	1 : N	`reservation.class_code`	Tickets are booked in a class
SEAT	RESERVATION	1 : 0..N	nullable `reservation.seat_id`	A seat is sold on many journeys, at most once per journey (unique index)
STATION	RESERVATION	1 : N (twice)	`from_station`, `to_station`	Boarding and alighting stations

Parent	Child	Cardinality	Implemented by	Meaning
RESERVATION	PAYMENT	1 : 1	FK + UNIQUE `pnr`	Exactly one payment per ticket
RESERVATION	CANCELLATION	1 : 0..1	FK + UNIQUE `pnr`	At most one cancellation per ticket
TRAIN	CAPACITY_RISK_LOG	1 : N	`capacity_risk_log.train_no`	One log row per train per run date

How relationships affect queries. Every JOIN in this project follows one of these foreign keys, for example `JOIN train t ON t.train_no = s.train_no`. Following a 1 : N relationship from the "1" side to the "N" side multiplies rows: one train joined to its 3 journeys produces 3 rows. That is why aggregate queries group the results back. Following a nullable FK (`seat_id`) needs a **LEFT JOIN** if rows without a seat (waitlisted tickets) must be kept.

Why ON DELETE CASCADE only in some places. Cascades are used where the child has no meaning without its parent: stops of a deleted route, coaches of a deleted train, seats of a deleted coach, the payment and cancellation of a deleted ticket. They are deliberately **not** used from PASSENGER or TRAIN_SCHEDULE to RESERVATION, because deleting a passenger should never silently delete ticket history. Test 5.6 proves that deleting passenger 1 is refused.

2.4 Normalization and functional dependencies

A **functional dependency (FD)** `A → B` means "if you know A, there is only one possible B". A table is in **BCNF** (Boyce-Codd Normal Form) when the left side of every non-trivial FD is a candidate key, i.e. every fact depends on "the key, the whole key, and nothing but the key".

FDs derived from the schema (the documentation does not list them; these follow from the keys and the meaning of the columns):

Table	Functional dependencies	Candidate keys	BCNF?
STATION	station_code → name, city, state, zone, platforms	station_code	Yes
ROUTE	route_id → name, origin, dest, distance; route_name → route_id	route_id, route_name	Yes (both left sides are keys)
ROUTE_STOP	(route_id, stop_seq) → station_code, km; (route_id, station_code) → stop_seq, km	both	Yes
TRAIN	train_no → name, type, route, dep_time, run_days, active; train_name → train_no	train_no, train_name	Yes
COACH_CLASS	class_code → name, seats, fare, fee	class_code	Yes
COACH	coach_id → train_no, label, class; (train_no, label) → coach_id	both	Yes
SEAT	seat_id → coach, seat_no, berth; (coach_id, seat_no) → seat_id	both	Yes
TRAIN_SCHEDULE	schedule_id → train, date, status; (train_no, date) → schedule_id	both	Yes
PASSENGER	passenger_id → all; phone → passenger_id; email → passenger_id	id, phone, (email)	Yes
RESERVATION	pnr → all other columns	pnr	Yes, with one noted design choice (below)

Table	Functional dependencies	Candidate keys	BCNF?
PAYMENT, CANCELLATION	id → all; pnr → id	id, pnr	Yes

Design choices worth discussing in a viva:

- **`reservation.fare` is a stored derived value.** It can be calculated from the route distance and the class rate. It is still stored on purpose: a ticket's price must not change if the class rate changes next year (*historical correctness*). This is a controlled exception, kept correct by the trigger.
- **`reservation.class_code` alongside `seat_id`.** For a confirmed ticket the class could be derived from seat → coach → class. It is stored because waitlisted tickets have no seat but still need a class. The trigger (rule R2) guarantees the two never disagree.
- **No transitive dependency is stored.** Distances are not stored in RESERVATION; they are computed from ROUTE_STOP in the view `v_ticket_details`. City names are not stored in ROUTE; they are joined from STATION.

2.5 The sample data

All of this data is real and was read from your database after a full run.

Stations, routes and stops

ACTUAL OUTPUT FROM YOUR DATABASE (CAPTURED 28-OCT-2026)

STATION_CODE	STATION_NAME	CITY	STATE	RAIL_ZONE	PLATFORMS
CBE	Coimbatore Junction	Coimbatore	Tamil Nadu	SR	6
DG	Dindigul Junction	Dindigul	Tamil Nadu	SR	5
ED	Erode Junction	Erode	Tamil Nadu	SR	5
ERS	Ernakulam Junction	Kochi	Kerala	SR	6
JTJ	Jolarpettai Junction	Jolarpettai	Tamil Nadu	SR	5
KPD	Katpadi Junction	Vellore	Tamil Nadu	SR	5
MAS	Chennai Central	Chennai	Tamil Nadu	SR	17
MDU	Madurai Junction	Madurai	Tamil Nadu	SR	7
PGT	Palakkad Junction	Palakkad	Kerala	SR	5
SA	Salem Junction	Salem	Tamil Nadu	SR	6
SBC	KSR Bengaluru City	Bengaluru	Karnataka	SWR	10
TPJ	Tiruchchirappalli Junction	Tiruchirappalli	Tamil Nadu	SR	8
TVC	Thiruvananthapuram Central	Thiruvananthapuram	Kerala	SR	5
VM	Villupuram Junction	Villupuram	Tamil Nadu	SR	6

14 row(s)

ACTUAL OUTPUT FROM YOUR DATABASE (CAPTURED 28-OCT-2026)

ROUTE_ID	ROUTE_NAME	ORIGIN_CODE	DEST_CODE	DISTANCE_KM
1	Chennai - Bengaluru	MAS	SBC	359
2	Chennai - Madurai	MAS	MDU	493
3	Chennai - Coimbatore	MAS	CBE	494
4	Bengaluru - Ernakulam	SBC	ERS	588

4 row(s)

ACTUAL OUTPUT FROM YOUR DATABASE (CAPTURED 28-OCT-2026)

ROUTE_ID	STOP_SEQ	STATION_CODE	KM_FROM_ORIGIN
1	1	MAS	0
1	2	KPD	129
1	3	JTJ	213
1	4	SBC	359
2	1	MAS	0
2	2	VM	159
2	3	TPJ	337
2	4	DG	431
2	5	MDU	493
3	1	MAS	0
3	2	KPD	129
3	3	JTJ	213
3	4	SA	334
3	5	ED	394
3	6	CBE	494
4	1	SBC	0
4	2	SA	198
4	3	ED	258
4	4	CBE	358
4	5	PGT	412
4	6	ERS	588

21 row(s)

Read the stops of route 1 as the line *Chennai Central (0 km) → Katpadi (129) → Jolarpettai (213) → Bengaluru (359)*. The fare from Katpadi to Bengaluru uses 359 − 129 = 230 km.

Classes, trains and coaches

ACTUAL OUTPUT FROM YOUR DATABASE (CAPTURED 28-OCT-2026)

CLASS_CODE	CLASS_NAME	SEATS_PER_COACH	FARE_PER_KM	RESERVATION_FEE
1A	First AC	4	4.2	60
2A	AC 2-Tier	8	2.5	50
3A	AC 3-Tier	10	1.8	40
CC	AC Chair Car	12	1.45	40
EC	Executive Chair Car	8	2.9	60
SL	Sleeper	12	0.65	20

6 row(s)

`seats_per_coach` is deliberately small (4–12) so that trains fill up and the waitlist logic can be demonstrated with a small amount of data.

ACTUAL OUTPUT FROM YOUR DATABASE (CAPTURED 28-OCT-2026)

TRAIN_NO	TRAIN_NAME	TRAIN_TYPE	ROUTE_ID	DEP_TIME	RUN_DAYS	IS_ACTIVE
12027	Chennai Shatabdi	SHATABDI	1	17:30	YYYYYYY	Y
12607	Lalbagh Express	SUPERFAST	1	15:35	YYYYYYY	Y
12635	Vaigai Express	SUPERFAST	2	13:50	YYYYYYY	Y
12637	Pandian Express	SUPERFAST	2	21:40	YYYYYYY	Y

TRAIN_NO	TRAIN_NAME	TRAIN_TYPE	ROUTE_ID	DEP_TIME	RUN_DAYS	IS_ACTIVE
12639	Brindavan Express	SUPERFAST	1	07:40	YYYYYYY	Y
12671	Nilgiri Express	SUPERFAST	3	21:05	YYYYYYY	Y
12675	Kovai Express	SUPERFAST	3	06:10	YYYYYYY	Y
12677	Ernakulam Intercity	INTERCITY	4	06:10	YYYYYYY	Y
22671	Madurai Tejas	TEJAS	2	06:00	YYYYYYY	Y

9 row(s)

The `dep_time` of 12639 is `07:40`, not the `07:50` inserted in 02_data.sql, because the UPDATE in Part 4.2 of 03_dml.sql changed it.

ACTUAL OUTPUT FROM YOUR DATABASE (CAPTURED 28-OCT-2026)

COACH_ID	TRAIN_NO	COACH_LABEL	CLASS_CODE
1	12639	C1	CC
2	12639	C2	CC
3	12639	E1	EC
4	12607	C1	CC
5	12607	C2	CC
6	12607	C3	CC
7	12027	C1	CC
8	12027	C2	CC
9	12027	E1	EC
10	12635	C1	CC
11	12635	C2	CC
12	12635	C3	CC
13	12637	S1	SL
14	12637	S2	SL
15	12637	B1	3A
16	12637	A1	2A
17	22671	C1	CC
18	22671	C2	CC
19	22671	E1	EC
20	12675	C1	CC
21	12675	C2	CC
22	12675	E1	EC
23	12671	S1	SL
24	12671	S2	SL
25	12671	B1	3A
26	12671	A1	2A
27	12671	H1	1A
28	12677	C1	CC
29	12677	C2	CC
30	12677	E1	EC

30 row(s)

Seats

The 320 seats are generated by one query (Q24). Seat numbers by class:

ACTUAL OUTPUT FROM YOUR DATABASE (CAPTURED 28-OCT-2026)

CLASS_CODE	SEATS
1A	4
2A	16
3A	20
CC	192
EC	40
SL	48

6 row(s)

Seats of coach 1 (C1, AC Chair Car, 12 seats) and coach 3 (E1, Executive Chair Car, 8 seats). Note `seat_id = coach_id × 100 + seat_no`:

ACTUAL OUTPUT FROM YOUR DATABASE (CAPTURED 28-OCT-2026)

SEAT_ID	COACH_ID	SEAT_NO	BERTH_TYPE
101	1	1	WINDOW
102	1	2	MIDDLE SEAT
103	1	3	AISLE
104	1	4	WINDOW
105	1	5	MIDDLE SEAT
106	1	6	AISLE
107	1	7	WINDOW
108	1	8	MIDDLE SEAT
109	1	9	AISLE
110	1	10	WINDOW
111	1	11	MIDDLE SEAT
112	1	12	AISLE
301	3	1	WINDOW
302	3	2	AISLE
303	3	3	WINDOW
304	3	4	AISLE
305	3	5	WINDOW
306	3	6	AISLE
307	3	7	WINDOW
308	3	8	AISLE

20 row(s)

Journeys

Each train has two COMPLETED journeys and one SCHEDULED journey; journey 130 is the demo journey created by 03_dml.sql (always 7 days after the run date).

ACTUAL OUTPUT FROM YOUR DATABASE (CAPTURED 28-OCT-2026)

SCHEDULE_ID	TRAIN_NO	JOURNEY_DATE	SCHED_STATUS
101	12639	20-SEP-2026	COMPLETED
102	12639	04-OCT-2026	COMPLETED
103	12639	21-OCT-2026	SCHEDULED
104	12607	21-SEP-2026	COMPLETED
105	12607	05-OCT-2026	COMPLETED

SCHEDULE_ID	TRAIN_NO	JOURNEY_DATE	SCHED_STATUS
106	12607	22-OCT-2026	SCHEDULED
107	12027	22-SEP-2026	COMPLETED
108	12027	06-OCT-2026	COMPLETED
109	12027	23-OCT-2026	SCHEDULED
110	12635	23-SEP-2026	COMPLETED
111	12635	07-OCT-2026	COMPLETED
112	12635	24-OCT-2026	SCHEDULED
113	12637	24-SEP-2026	COMPLETED
114	12637	08-OCT-2026	COMPLETED
115	12637	25-OCT-2026	SCHEDULED
116	22671	25-SEP-2026	COMPLETED
117	22671	09-OCT-2026	COMPLETED
118	22671	21-OCT-2026	SCHEDULED
119	12675	26-SEP-2026	COMPLETED
120	12675	10-OCT-2026	COMPLETED
121	12675	22-OCT-2026	SCHEDULED
122	12671	27-SEP-2026	COMPLETED
123	12671	11-OCT-2026	COMPLETED
124	12671	23-OCT-2026	SCHEDULED
125	12677	28-SEP-2026	COMPLETED
126	12677	12-OCT-2026	COMPLETED
127	12677	24-OCT-2026	SCHEDULED
130	12677	04-NOV-2026	SCHEDULED

28 row(s)

Passengers (first 8 of 150)

ACTUAL OUTPUT FROM YOUR DATABASE (CAPTURED 28-OCT-2026)

PASSENGER_ID	FULL_NAME	GENDER	DOB	PHONE	EMAIL	HOME_CITY	REGISTERED_ON	CONCESSION_TYPE
1	Balaji Mani	M	10-JUL-1993	7624642010	balaji.mani1@mail.com	Kochi	28-JAN-2025	NONE
2	Ajay Thomas	M	24-FEB-1997	8518184808	ajay.thomas2@mail.com	Vellore	12-NOV-2025	NONE
3	Nithya Balaji	F	11-AUG-2006	8386632201	nithya.balaji3@mail.com	Chennai	03-APR-2025	STUDENT
4	Manoj Chandran	M	25-MAR-1958	8935800437	manoj.chandran4@mail.com	Chennai	28-SEP-2025	SENIOR
5	Priya Pandian	F	04-NOV-1982	7031005178	priya.pandian5@mail.com	Tiruchirappalli	26-AUG-2025	NONE
6	Janani Shankar	F	28-SEP-1983	7551561306	janani.shankar6@mail.com	Bengaluru	19-APR-2025	NONE
7	Gayathri Varma	F	16-DEC-1978	7600100795	gayathri.varma7@mail.com	Palakkad	14-DEC-2025	NONE
8	Kishore Kumar	M	21-JUN-1980	9571985370	kishore.kumar8@mail.com	Kochi	12-NOV-2025	NONE

8 row(s)

Reservations, payments and cancellations

First six tickets. The `FARE` column was **not** in the INSERT statements; the trigger calculated it:

ACTUAL OUTPUT FROM YOUR DATABASE (CAPTURED 28-OCT-2026)

PNR	PASSENGER_ID	SCHEDULE_ID	CLASS_CODE	SEAT_ID	FROM_STATION	TO_STATION	BOOKING_DATE	FARE	STATUS
4200000001	11	101	CC	205	MAS	SBC	11-AUG-2026	561	COMPLETED
4200000002	16	101	CC	103	KPD	SBC	12-AUG-2026	374	COMPLETED
4200000003	108	101	CC	112	MAS	SBC	12-AUG-2026	561	COMPLETED
4200000004	65	101	CC	208	MAS	SBC	14-AUG-2026	561	COMPLETED
4200000005	42	101	CC	206	JTJ	SBC	15-AUG-2026	252	COMPLETED
4200000006	129	101	CC	*NULL*	MAS	SBC	15-AUG-2026	561	CANCELLED

6 row(s)

Tickets by status. Confirmed and completed tickets always have a seat; waitlisted and cancelled never do, exactly as `ck_res_seat_status` demands:

ACTUAL OUTPUT FROM YOUR DATABASE (CAPTURED 28-OCT-2026)

STATUS	N	WITH_SEAT
CANCELLED	36	0
COMPLETED	487	487
CONFIRMED	234	234
WAITLIST	20	0

4 row(s)

ACTUAL OUTPUT FROM YOUR DATABASE (CAPTURED 28-OCT-2026)

PAYMENT_ID	PNR	AMOUNT	PAY_MODE	PAID_ON	PAY_STATUS
900001	4200000001	561	CARD	11-AUG-2026	SUCCESS
900002	4200000002	374	UPI	12-AUG-2026	SUCCESS
900003	4200000003	561	NETBANKING	12-AUG-2026	SUCCESS
900004	4200000004	561	WALLET	14-AUG-2026	SUCCESS
900005	4200000005	252	UPI	15-AUG-2026	SUCCESS
900006	4200000006	561	CARD	15-AUG-2026	PARTIAL_REFUND

6 row(s)

ACTUAL OUTPUT FROM YOUR DATABASE (CAPTURED 28-OCT-2026)

PAY_STATUS	PAY_MODE	N
PARTIAL_REFUND	CARD	6
PARTIAL_REFUND	NETBANKING	8
PARTIAL_REFUND	UPI	14
PARTIAL_REFUND	WALLET	8
REFUNDED	CARD	4
REFUNDED	NETBANKING	2
REFUNDED	UPI	6
REFUNDED	WALLET	3
SUCCESS	CARD	144
SUCCESS	NETBANKING	144
SUCCESS	UPI	296
SUCCESS	WALLET	142

12 row(s)

ACTUAL OUTPUT FROM YOUR DATABASE (CAPTURED 28-OCT-2026)

CANCEL_ID	PNR	CANCELLED_ON	REFUND_AMOUNT	REASON
700001	4200000006	02-SEP-2026	504.9	Booked alternate travel
700002	4200000061	28-SEP-2026	226.8	Medical emergency
700003	4200000109	02-SEP-2026	504.9	Medical emergency
700004	4200000136	22-SEP-2026	504.9	Change of plans
700005	4200000175	01-SEP-2026	504.9	Official work
700006	4200000183	05-SEP-2026	336.6	Official work

6 row(s)

Important values used later: journey 101 (train 12639, completed) is used by integrity test 5.2; journey 103 (12639, scheduled) by test 5.3–5.5 and the boarding-point change; passenger 1 by query A1 and test 5.6; seat 2801 (coach 28 of train 12677) by test 5.3; journey 130 by the booking demo.

Database objects created

ACTUAL OUTPUT FROM YOUR DATABASE (CAPTURED 28-OCT-2026)

TABLE_NAME	CONSTRAINT_TYPE	N
CANCELLATION	C	1
CANCELLATION	P	1
CANCELLATION	R	1
CANCELLATION	U	1
CAPACITY_RISK_LOG	P	1
CAPACITY_RISK_LOG	R	1
COACH	P	1
COACH	R	2
COACH	U	1
COACH_CLASS	C	3
COACH_CLASS	P	1
PASSENGER	C	4
PASSENGER	P	1
PASSENGER	U	2
PAYMENT	C	3
PAYMENT	P	1
PAYMENT	R	1
PAYMENT	U	1
RESERVATION	C	4
RESERVATION	P	1
RESERVATION	R	6
RISK_THRESHOLD	C	1
RISK_THRESHOLD	P	1
ROUTE	C	2
ROUTE	P	1
ROUTE	R	2
ROUTE	U	1
ROUTE_STOP	C	2
ROUTE_STOP	P	1

TABLE_NAME	CONSTRAINT_TYPE	N
ROUTE_STOP	R	2
ROUTE_STOP	U	1
SEAT	C	2
SEAT	P	1
SEAT	R	1
SEAT	U	1
STATION	C	2
STATION	P	1
TRAIN	C	5
TRAIN	P	1
TRAIN	R	1
TRAIN	U	1
TRAIN_SCHEDULE	C	2
TRAIN_SCHEDULE	P	1
TRAIN_SCHEDULE	R	1
TRAIN_SCHEDULE	U	1

45 row(s)

(C = CHECK/NOT NULL constraint with a name, P = primary key, R = referential/foreign key, U = unique.)

ACTUAL OUTPUT FROM YOUR DATABASE (CAPTURED 28-OCT-2026)

INDEX_NAME	TABLE_NAME	UNIQUENESS	INDEX_TYPE
PK_CANCELLATION	CANCELLATION	UNIQUE	NORMAL
UQ_CANCEL_PNR	CANCELLATION	UNIQUE	NORMAL
PK_CAPACITY_RISK_LOG	CAPACITY_RISK_LOG	UNIQUE	NORMAL
IX_COACH_CLASS	COACH	NONUNIQUE	NORMAL
PK_COACH	COACH	UNIQUE	NORMAL
UQ_COACH_LABEL	COACH	UNIQUE	NORMAL
PK_COACH_CLASS	COACH_CLASS	UNIQUE	NORMAL
PK_PASSENGER	PASSENGER	UNIQUE	NORMAL
UQ_PASS_EMAIL	PASSENGER	UNIQUE	NORMAL
UQ_PASS_PHONE	PASSENGER	UNIQUE	NORMAL
PK_PAYMENT	PAYMENT	UNIQUE	NORMAL
UQ_PAY_PNR	PAYMENT	UNIQUE	NORMAL
IX_RES_FROM_STN	RESERVATION	NONUNIQUE	NORMAL
IX_RES_PASSENGER	RESERVATION	NONUNIQUE	NORMAL
IX_RES_SCHEDULE_STATUS	RESERVATION	NONUNIQUE	NORMAL
IX_RES_SEAT	RESERVATION	NONUNIQUE	NORMAL
IX_RES_TO_STN	RESERVATION	NONUNIQUE	NORMAL
PK_RESERVATION	RESERVATION	UNIQUE	NORMAL
UX_RES_PASS_JOURNEY	RESERVATION	UNIQUE	FUNCTION-BASED NORMAL
UX_RES_SEAT_JOURNEY	RESERVATION	UNIQUE	FUNCTION-BASED NORMAL
PK_RISK_THRESHOLD	RISK_THRESHOLD	UNIQUE	NORMAL
PK_ROUTE	ROUTE	UNIQUE	NORMAL
UQ_ROUTE_NAME	ROUTE	UNIQUE	NORMAL

INDEX_NAME	TABLE_NAME	UNIQUENESS	INDEX_TYPE
IX_RS_STATION	ROUTE_STOP	NONUNIQUE	NORMAL
PK_ROUTE_STOP	ROUTE_STOP	UNIQUE	NORMAL
UQ_RS_STATION	ROUTE_STOP	UNIQUE	NORMAL
PK_SEAT	SEAT	UNIQUE	NORMAL
UQ_SEAT	SEAT	UNIQUE	NORMAL
PK_STATION	STATION	UNIQUE	NORMAL
IX_TRAIN_ROUTE	TRAIN	NONUNIQUE	NORMAL
PK_TRAIN	TRAIN	UNIQUE	NORMAL
UQ_TRAIN_NAME	TRAIN	UNIQUE	NORMAL
PK_SCHEDULE	TRAIN_SCHEDULE	UNIQUE	NORMAL
UQ_SCHED	TRAIN_SCHEDULE	UNIQUE	NORMAL

34 row(s)

The `FUNCTION-BASED NORMAL` index type marks the two expression indexes `UX_RES_SEAT_JOURNEY` and `UX_RES_PASS_JOURNEY`. Every PK and UNIQUE constraint created its own unique index automatically (`PK_…` , `UQ_…`).

ACTUAL OUTPUT FROM YOUR DATABASE (CAPTURED 28-OCT-2026)

SEQUENCE_NAME	LAST_NUMBER
SEQ_CANCEL	701002
SEQ_PAYMENT	901010
SEQ_PNR	4200001010

3 row(s)

`LAST_NUMBER` is the next value the sequence will hand out. `SEQ_PNR` has advanced past 4200001001 because the demo booked 9 tickets.

3. File 01_ddl.sql — Schema, Constraints, Indexes, Trigger

What this file does: it removes any previous copy of the project, then creates the 12 core tables with all their constraints, the indexes on RESERVATION, three sequences and the validation trigger. Everything in it is **DDL (Data Definition Language)**: statements that define structure rather than data. The file starts with SQL*Plus session settings (SET, ALTER SESSION, COLUMN); those are explained in Chapter 7.

DDL IN ORACLE — READ THIS ONCE

In Oracle every DDL statement (CREATE, ALTER, DROP, TRUNCATE, RENAME) performs an **implicit COMMIT** before and after it runs. Consequences: (1) a DDL statement can never be rolled back with ROLLBACK; (2) any uncommitted INSERT/UPDATE you made earlier in the session becomes permanent the moment a DDL statement runs. Some other databases (e.g. PostgreSQL) allow DDL inside a transaction; Oracle does not.

Q01 · Clean-up block (Part 0)

The query

```
BEGIN
  FOR v IN (SELECT view_name FROM user_views
            WHERE view_name IN ('V_RESERVATION_PERFORMANCE','V_TRAIN_UTILIZATION','V_TICKET_DETAILS')) L
OOP
    EXECUTE IMMEDIATE 'DROP VIEW ' || v.view_name;
  END LOOP;
  FOR t IN (SELECT table_name FROM user_tables
            WHERE table_name IN ('CAPACITY_RISK_LOG','RISK_THRESHOLD','CANCELLATION','PAYMENT','RESERVAT
ION',
                                 'PASSENGER','TRAIN_SCHEDULE','SEAT','COACH','COACH_CLASS','TRAIN',
                                 'ROUTE_STOP','ROUTE','STATION')) LOOP
    EXECUTE IMMEDIATE 'DROP TABLE ' || t.table_name || ' CASCADE CONSTRAINTS PURGE';
  END LOOP;
  FOR s IN (SELECT sequence_name FROM user_sequences
            WHERE sequence_name IN ('SEQ_PNR','SEQ_PAYMENT','SEQ_CANCEL')) LOOP
    EXECUTE IMMEDIATE 'DROP SEQUENCE ' || s.sequence_name;
  END LOOP;
  FOR p IN (SELECT object_name, object_type FROM user_objects
            WHERE object_type IN ('PROCEDURE','FUNCTION')
            AND object_name IN ('BOOK_TICKET','CANCEL_TICKET','PRC_CAPACITY_RISK_REPORT','FN_CAPACITY_RI
SK')) LOOP
    EXECUTE IMMEDIATE 'DROP ' || p.object_type || ' ' || p.object_name;
  END LOOP;
END;
/
```

Source: parts/01_ddl.sql, lines 29–50

What it is for

One line: it finds which of the project's objects already exist and drops only those, so the whole script can be re-run safely.

Detailed: `CREATE TABLE station` fails with *ORA-00955: name is already used by an existing object* if STATION already exists. A plain `DROP TABLE station` fails with *ORA-00942: table or view does not exist* on the very first run. This block avoids both problems by asking Oracle's **data dictionary** which objects exist and dropping exactly those. Without it, a second run of `run_all.sql` would fail on every CREATE and the old data would remain.

Simple explanation

Imagine you want to rebuild a Lego house. First you must take the old house apart, or the new bricks will not fit.

This block does exactly that. It asks Oracle: "Which of my project's tables, views, sequences and procedures already exist?" Oracle keeps a list of everything it has (called the *data dictionary*). The block reads that list and deletes only the project's objects that are actually there.

- If you run the project for the first time, nothing exists, so nothing is deleted and there is no error.
- If you run it again, the old objects are deleted, so the CREATE statements that follow can build them fresh.

`EXECUTE IMMEDIATE` is used because PL/SQL is not allowed to contain a DROP command directly; the command has to be built as text and then run. `CASCADE CONSTRAINTS` means "also remove any links from other tables pointing to this one", so the order of deleting does not matter. `PURGE` means "delete permanently, do not put it in the recycle bin".

Concepts used

DDL, the **data dictionary** (metadata about the database stored as tables), **anonymous PL/SQL blocks**, **implicit cursors**, **dynamic SQL**, **referential integrity** (why CASCADE CONSTRAINTS is needed), **dependency invalidation** (a view on a dropped table becomes INVALID, not dropped), and **idempotency** (running a script twice gives the same result as running it once).

Things to watch out for

- A new object added to the project but not to these lists makes the next run fail with ORA-00955. Today the lists match all created objects.
- If another session has a lock on one of these tables, DROP fails with *ORA-00054: resource busy*. Because `run_all.sql` uses `WHENEVER SQLERROR CONTINUE`, the script keeps going and later statements fail. Close other sessions before re-running.
- PURGE is permanent: fine for a lab, risky on a production database.
- Common misconception: "dropping a table drops its views" — false; they become INVALID.

Q02 · CREATE TABLE station

The query

```
-- 1.1 STATION : every railway station served by the network
CREATE TABLE station (
  station_code   VARCHAR2(5)   CONSTRAINT pk_station PRIMARY KEY,
  station_name   VARCHAR2(60)  NOT NULL,
  city           VARCHAR2(40)  NOT NULL,
  state          VARCHAR2(30)  NOT NULL,
  rail_zone      VARCHAR2(5)   NOT NULL,
  platforms      NUMBER(2)     DEFAULT 1 NOT NULL
                 CONSTRAINT ck_station_pf CHECK (platforms BETWEEN 1 AND 30),
  CONSTRAINT ck_station_code CHECK (station_code = UPPER(station_code))
);
```

Source: parts/01_ddl.sql, lines 56–66

What it is for

One line: creates the table that stores every railway station.

Detailed: STATION is the most referenced table in the project. Routes, route stops and both ends of every ticket point to it. It must be created first because foreign keys can only reference a table that already exists.

Simple explanation

This creates the **STATION** table: a list of all railway stations, like a phone book of stations.

Each station has a short code (like `MAS` for Chennai Central), a full name, city, state, railway zone and number of platforms.

- The **code is the primary key**: no two stations can have the same code, and every station must have one.
- The code must be in **capital letters**, so nobody can add `mas` and `MAS` as two different stations.
- Platforms must be between 1 and 30, and if you don't give a number, it becomes 1 automatically.

Every other table that mentions a station (routes, stops, tickets) points to this table.

Concepts used

Entity integrity (every row has a non-null unique identifier), **domain constraints** (CHECK limits a column to valid values), **default values**, **inline vs out-of-line constraints**, and **named constraints**. Naming constraints makes error messages readable: `ORA-02290: check constraint (SHREY.CK_STATION_PF) violated` tells you exactly which rule failed.

Things to watch out for

- `BETWEEN 1 AND 30` is inclusive; writing `> 1 AND < 30` would wrongly reject 1 and 30.
- VARCHAR2 lengths are in **bytes** by default; multi-byte characters (e.g. Tamil script) take more than one byte each.

Q03 · CREATE TABLE route

The query

```
-- 1.2 ROUTE : a named corridor between an origin and a destination
CREATE TABLE route (
  route_id       NUMBER(4)     CONSTRAINT pk_route PRIMARY KEY,
  route_name     VARCHAR2(60)  NOT NULL CONSTRAINT uq_route_name UNIQUE,
  origin_code    VARCHAR2(5)   NOT NULL CONSTRAINT fk_route_origin REFERENCES station(station_code),
  dest_code      VARCHAR2(5)   NOT NULL CONSTRAINT fk_route_dest   REFERENCES station(station_code),
  distance_km    NUMBER(5)     NOT NULL CONSTRAINT ck_route_km CHECK (distance_km > 0),
  CONSTRAINT ck_route_ends CHECK (origin_code <> dest_code)
);
```

Source: parts/01_ddl.sql, lines 68–76

What it is for

One line: creates the table of corridors (e.g. Chennai–Bengaluru), each linking an origin and a destination station.

Detailed: A route is the path a train follows. Storing it separately means several trains (12639, 12607, 12027 all run Chennai–Bengaluru) share one route definition instead of repeating it.

Simple explanation

This creates the **ROUTE** table: the corridors trains run on, for example *Chennai – Bengaluru*.

Each route has a number, a unique name, a starting station, an ending station and a total distance.

- The start and end stations must be real stations from the STATION table (these are **foreign keys**).
- A route cannot start and end at the same station.
- The distance must be more than 0.
- Two routes cannot have the same name.

Several trains can use the same route, so the route is stored once and shared.

Concepts used

Foreign keys and referential integrity, **multiple relationships between the same two tables** (role names: origin and destination), **candidate/alternate keys** (UNIQUE route_name), and **row-level CHECK across columns**.

Things to watch out for

The FK column's data type must be compatible with the parent's. A common student error is referencing a column that is neither PK nor UNIQUE, which gives ORA-02270: no matching unique or primary key.

Q04 · CREATE TABLE route_stop

The query

```
-- 1.3 ROUTE_STOP : ordered list of stations on a route (resolves the M:N ROUTE-STATION)
CREATE TABLE route_stop (
  route_id       NUMBER(4)     CONSTRAINT fk_rs_route REFERENCES route(route_id) ON DELETE CASCADE,
  stop_seq       NUMBER(2)     CONSTRAINT ck_rs_seq CHECK (stop_seq >= 1),
  station_code   VARCHAR2(5)   NOT NULL CONSTRAINT fk_rs_station REFERENCES station(station_code),
  km_from_origin NUMBER(5)     NOT NULL CONSTRAINT ck_rs_km CHECK (km_from_origin >= 0),
  CONSTRAINT pk_route_stop PRIMARY KEY (route_id, stop_seq),
  CONSTRAINT uq_rs_station UNIQUE (route_id, station_code)
);
```

Source: parts/01_ddl.sql, lines 78–86

What it is for

One line: stores the ordered list of stations on each route, with each station's distance from the origin.

Detailed: This table is the heart of fare calculation and direction checking. The trigger reads `stop_seq` to make sure the boarding station comes before the destination, and `km_from_origin` to compute the distance travelled. It resolves the **many-to-many** relationship between ROUTE and STATION.

Simple explanation

This creates the **ROUTE_STOP** table: the list of stations on each route, in order, with how far each one is from the start.

Think of the stop list printed on a bus: *Stop 1 Chennai (0 km), Stop 2 Katpadi (129 km), Stop 3 Jolarpettai (213 km), Stop 4 Bengaluru (359 km)*. Each line of that list is one row here.

- A route and a stop number together identify one row (a **composite primary key**).
- The same station cannot appear twice on the same route.
- If a route is deleted, its stops are deleted automatically.

This table is very important: it is used to check that a passenger travels in the right direction and to calculate the distance for the fare.

Concepts used

Associative entity, **composite keys**, **two candidate keys** ((route_id, stop_seq) and (route_id, station_code)), **ON DELETE CASCADE**, and the notion that a relationship can carry its own attributes (position, distance).

Things to watch out for

Nothing in the schema forces `km_from_origin` to increase with `stop_seq`. A data-entry error (stop 3 at 100 km, stop 2 at 200 km) would give a negative segment distance; the fare formula's `GREATEST(…, 50)` would hide it by charging 50 km. A CHECK cannot compare rows, so enforcing this would need a trigger.

Q05 · CREATE TABLE train

The query

```
-- 1.4 TRAIN : a train service that always runs on one route
CREATE TABLE train (
  train_no       NUMBER(5)     CONSTRAINT pk_train PRIMARY KEY
                 CONSTRAINT ck_train_no CHECK (train_no BETWEEN 10000 AND 99999),
  train_name     VARCHAR2(50)  NOT NULL,
  train_type     VARCHAR2(15)  NOT NULL
                 CONSTRAINT ck_train_type CHECK (train_type IN
('SUPERFAST','EXPRESS','SHATABDI','TEJAS','INTERCITY','MAIL')),
  route_id       NUMBER(4)     NOT NULL CONSTRAINT fk_train_route REFERENCES route(route_id),
  dep_time       VARCHAR2(5)   NOT NULL
                 CONSTRAINT ck_train_dep CHECK (REGEXP_LIKE(dep_time, '^([01][0-9]|2[0-3]):[0-5][0-9]$')
),
  run_days       VARCHAR2(7)   DEFAULT 'YYYYYYY' NOT NULL
                 CONSTRAINT ck_train_days CHECK (REGEXP_LIKE(run_days, '^[YN]{7}$')),
  is_active      CHAR(1)       DEFAULT 'Y' NOT NULL CONSTRAINT ck_train_active CHECK (is_active IN
('Y','N'))
);
```

Source: parts/01_ddl.sql, lines 88–101

What it is for

One line: creates the table of train services, each permanently assigned to one route.

Detailed: A *train* here is the service (number, name, type, departure time), not a dated run; dated runs are in TRAIN_SCHEDULE. This separation avoids repeating the name, type and route for every date.

Simple explanation

This creates the **TRAIN** table: each train service, like *12639 Brindavan Express*.

Each train has a 5-digit number, a name, a type (SUPERFAST, EXPRESS, …), the route it runs on, a departure time, the days it runs and whether it is active.

- The train number must be exactly 5 digits.
- The type must be one of the allowed types.
- The departure time must look like a real clock time, `HH:MM` between 00:00 and 23:59 (checked with a pattern called a *regular expression*).
- The route must exist in the ROUTE table.

This table describes the train in general. Its trips on specific dates are stored separately in TRAIN_SCHEDULE.

Concepts used

Domain integrity through CHECK and regular expressions; multiple constraints on one column; separating a *type* entity (TRAIN) from its *instances* (TRAIN_SCHEDULE).

Things to watch out for

`run_days` and `is_active` are stored but no query in the project uses them. They document real-world attributes for future use; say so honestly if asked.

Q06 · CREATE TABLE coach_class

The query

```
-- 1.5 COACH_CLASS : travel class with fare rule and seats per coach
CREATE TABLE coach_class (
  class_code      VARCHAR2(3)  CONSTRAINT pk_coach_class PRIMARY KEY,
  class_name      VARCHAR2(30) NOT NULL,
  seats_per_coach NUMBER(3)    NOT NULL CONSTRAINT ck_cc_seats CHECK (seats_per_coach BETWEEN 1 AND 100)
,
  fare_per_km     NUMBER(5,2)  NOT NULL CONSTRAINT ck_cc_fare  CHECK (fare_per_km > 0),
  reservation_fee NUMBER(5)    NOT NULL CONSTRAINT ck_cc_fee   CHECK (reservation_fee >= 0)
);
```

Source: parts/01_ddl.sql, lines 103–110

What it is for

One line: creates the lookup table of travel classes with their fare rules and seats per coach.

Detailed: Every fare in the project is computed from this table: `fare_per_km` and `reservation_fee`. `seats_per_coach` is used to generate seats (Q24) and to compute capacity in the views.

Simple explanation

This creates the **COACH_CLASS** table: the travel classes, like *Sleeper*, *AC Chair Car* or *First AC*.

For each class it stores how many seats one coach has, the price per kilometre and a fixed reservation fee.

Every ticket price in the project is calculated from this table: **fare = distance × price per km + reservation fee**.

Keeping these numbers in a table (not hidden in code) means that if the railway changes prices, you only change one row.

Concepts used

A **lookup/reference table**, `NUMBER(p,s)` precision and scale, and keeping business parameters as data instead of code.

Things to watch out for

`NUMBER(5,2)` means 5 digits **in total**, 2 after the point, so 1000.00 does not fit (ORA-01438: value larger than specified precision).

Q07 · CREATE TABLE coach

The query

```
-- 1.6 COACH : physical coach attached to a train (weak entity identified by train + label)
CREATE TABLE coach (
  coach_id       NUMBER(5)     CONSTRAINT pk_coach PRIMARY KEY,
  train_no       NUMBER(5)     NOT NULL CONSTRAINT fk_coach_train REFERENCES train(train_no) ON DELETE C
ASCADE,
  coach_label    VARCHAR2(4)   NOT NULL,
  class_code     VARCHAR2(3)   NOT NULL CONSTRAINT fk_coach_class REFERENCES coach_class(class_code),
  CONSTRAINT uq_coach_label UNIQUE (train_no, coach_label)
);
```

Source: parts/01_ddl.sql, lines 112–119

What it is for

One line: creates the table of physical coaches, each belonging to one train and one class.

Detailed: COACH connects trains to seats and determines the class of every seat. It is how the trigger checks that a seat belongs to the right train and class (rule R2).

Simple explanation

This creates the **COACH** table: the actual coaches attached to each train, like coach *C1* of train 12639.

Each coach has an id, belongs to one train, has a label (C1, E1, S1…) and a class.

- A train cannot have two coaches with the same label.
- If a train is deleted, its coaches are deleted too.
- The class decides what kind of seats the coach has and how many.

Concepts used

Weak entity (identified through its owner train), **surrogate vs natural keys**, **cascading deletes along a chain** (TRAIN → COACH → SEAT).

Things to watch out for

Cascade chains can delete far more than expected. Here the chain stops at RESERVATION, because `fk_res_seat` has no cascade: a seat that appears on any ticket cannot be deleted.

Q08 · CREATE TABLE seat

The query

```
-- 1.7 SEAT : individual seat / berth inside a coach
CREATE TABLE seat (
  seat_id        NUMBER(7)     CONSTRAINT pk_seat PRIMARY KEY,
  coach_id       NUMBER(5)     NOT NULL CONSTRAINT fk_seat_coach REFERENCES coach(coach_id) ON DELETE CA
SCADE,
  seat_no        NUMBER(3)     NOT NULL CONSTRAINT ck_seat_no CHECK (seat_no >= 1),
  berth_type     VARCHAR2(12)  NOT NULL
                 CONSTRAINT ck_berth CHECK (berth_type IN ('LOWER','MIDDLE','UPPER','SIDE LOWER','SIDE
UPPER',
                                                           'WINDOW','AISLE','MIDDLE SEAT')),
  CONSTRAINT uq_seat UNIQUE (coach_id, seat_no)
);
```

Source: parts/01_ddl.sql, lines 121–130

What it is for

One line: creates the table of individual seats and berths inside each coach.

Detailed: A seat is what a confirmed ticket holds. The seat table is fixed (it describes the physical train); whether a seat is free on a given date is worked out from RESERVATION.

Simple explanation

This creates the **SEAT** table: every single seat or berth inside every coach.

Each seat has an id, the coach it belongs to, a seat number and a type (LOWER, UPPER, WINDOW, AISLE…).

- A coach cannot have two seats with the same number.
- If a coach is deleted, its seats are deleted too.

Important idea: this table only says *which seats exist*. Whether a seat is free on a particular day is found by checking the RESERVATION table. That way one seat can be sold again and again on different dates.

Concepts used

Composite UNIQUE as a candidate key, enumerated domains, and the important modelling idea of separating **static inventory** (seats) from **dynamic allocation** (reservations per journey).

Things to watch out for

The `coach_id × 100 + seat_no` pattern is safe only while seat numbers stay at or below 100. Seat 101 of coach 1 would get id 201, the same as seat 1 of coach 2, and the INSERT would fail on the primary key. The CHECK `seats_per_coach BETWEEN 1 AND 100` on COACH_CLASS keeps it safe.

Q09 · CREATE TABLE train_schedule

The query

```
-- 1.8 TRAIN_SCHEDULE : one actual run (journey) of a train on a date
CREATE TABLE train_schedule (
  schedule_id    NUMBER(6)     CONSTRAINT pk_schedule PRIMARY KEY,
  train_no       NUMBER(5)     NOT NULL CONSTRAINT fk_sched_train REFERENCES train(train_no),
  journey_date   DATE          NOT NULL,
  sched_status   VARCHAR2(10)  DEFAULT 'SCHEDULED' NOT NULL
                 CONSTRAINT ck_sched_status CHECK (sched_status IN ('SCHEDULED','DEPARTED','COMPLETED','
CANCELLED')),
  CONSTRAINT uq_sched UNIQUE (train_no, journey_date),
  CONSTRAINT ck_sched_date CHECK (journey_date = TRUNC(journey_date))
);
```

Source: parts/01_ddl.sql, lines 132–141

What it is for

One line: creates the table of dated runs (journeys) of each train.

Detailed: A ticket is for a specific journey, e.g. "12639 on 21-Oct-2026", not for a train in general. Every reservation points to one schedule row.

Simple explanation

This creates the **TRAIN_SCHEDULE** table: each actual trip of a train on a specific date, like *12639 on 21 October*.

You never book "a train"; you book a train on a date. That date-specific trip is a row in this table.

- A train can run only once per date.
- The date must be just a date, with no time part. (Oracle dates normally include a time; without this rule, two rows on the same day with different times would sneak past the "once per date" rule.)
- The status must be SCHEDULED, DEPARTED, COMPLETED or CANCELLED. New bookings are only allowed while it is SCHEDULED.

Concepts used

Composite UNIQUE as a natural key, Oracle DATE always containing a time, state-machine columns, and using a CHECK to make another constraint meaningful.

Things to watch out for

Students often write `WHERE journey_date = SYSDATE` and get no rows, because SYSDATE includes the current time. Use `TRUNC(SYSDATE)`.

Q10 · CREATE TABLE passenger

The query

```
-- 1.9 PASSENGER : registered traveller
CREATE TABLE passenger (
  passenger_id   NUMBER(6)     CONSTRAINT pk_passenger PRIMARY KEY,
  full_name      VARCHAR2(60)  NOT NULL,
  gender         CHAR(1)       NOT NULL CONSTRAINT ck_pass_gender CHECK (gender IN ('M','F','T')),
  dob            DATE          NOT NULL,
  phone          VARCHAR2(10)  NOT NULL CONSTRAINT uq_pass_phone UNIQUE
                 CONSTRAINT ck_pass_phone CHECK (REGEXP_LIKE(phone, '^[6-9][0-9]{9}$')),
  email          VARCHAR2(60)  CONSTRAINT uq_pass_email UNIQUE
                 CONSTRAINT ck_pass_email CHECK (REGEXP_LIKE(email, '^[A-Za-z0-9._]+@[A-Za-z0-9.]+\.[A-
Za-z]{2,}$')),
  city           VARCHAR2(40),
  registered_on  DATE          DEFAULT SYSDATE NOT NULL
);
```

Source: parts/01_ddl.sql, lines 143–155

What it is for

One line: creates the table of registered travellers.

Detailed: Passengers book tickets. The table shows several validation techniques: a regex for phone numbers and e-mails, UNIQUE on a nullable column, and DEFAULT SYSDATE.

Simple explanation

This creates the **PASSENGER** table: the people who register and travel.

It stores id, name, gender, date of birth, phone, e-mail, city and registration date.

- The phone number must be 10 digits starting with 6, 7, 8 or 9 (a real Indian mobile), and no two passengers can share a phone.
- The e-mail is optional, but if given it must look like an e-mail address and must be unique.
- The registration date is filled in automatically with today's date if not given.

Concepts used

Candidate keys (phone, email), **UNIQUE with NULLs**, **regular-expression CHECKs**, **three-valued logic**: when email is NULL, `REGEXP_LIKE(NULL, …)` is UNKNOWN, so the CHECK passes, which is exactly what we want for an optional column.

Things to watch out for

The e-mail regex is deliberately simple: it rejects valid addresses containing `+` or `-` (e.g. `a-b@x.com`). Mention this as a limitation if asked.

Q11 · CREATE TABLE reservation

The query

```
-- 1.10 RESERVATION : one ticket (PNR) = one passenger on one journey; holds the ticket status
CREATE TABLE reservation (
  pnr            NUMBER(10)    CONSTRAINT pk_reservation PRIMARY KEY,
  passenger_id   NUMBER(6)     NOT NULL CONSTRAINT fk_res_pass  REFERENCES passenger(passenger_id),
  schedule_id    NUMBER(6)     NOT NULL CONSTRAINT fk_res_sched REFERENCES train_schedule(schedule_id),
  class_code     VARCHAR2(3)   NOT NULL CONSTRAINT fk_res_class REFERENCES coach_class(class_code),
  seat_id        NUMBER(7)     CONSTRAINT fk_res_seat REFERENCES seat(seat_id),
  from_station   VARCHAR2(5)   NOT NULL CONSTRAINT fk_res_from REFERENCES station(station_code),
  to_station     VARCHAR2(5)   NOT NULL CONSTRAINT fk_res_to   REFERENCES station(station_code),
  booking_date   DATE          DEFAULT TRUNC(SYSDATE) NOT NULL,
  fare           NUMBER(8,2)   CONSTRAINT ck_res_fare CHECK (fare > 0),
  status         VARCHAR2(10)  DEFAULT 'CONFIRMED' NOT NULL
                 CONSTRAINT ck_res_status CHECK (status IN ('CONFIRMED','WAITLIST','CANCELLED','COMPLETE
D')),
  CONSTRAINT ck_res_stations CHECK (from_station <> to_station),
  -- a confirmed / completed ticket must hold a seat; a waitlisted / cancelled one must not
  CONSTRAINT ck_res_seat_status CHECK (
        (status IN ('CONFIRMED','COMPLETED') AND seat_id IS NOT NULL)
     OR (status IN ('WAITLIST','CANCELLED')  AND seat_id IS NULL))
);
```

Source: parts/01_ddl.sql, lines 157–175

What it is for

One line: creates the ticket table; each row is one passenger on one journey.

Detailed: RESERVATION is the central fact table of the project. It links six parents (passenger, schedule, class, seat, two stations) and carries the ticket's state. Most reports read it, the trigger guards it and both procedures write it.

Simple explanation

This creates the **RESERVATION** table: the tickets. One row = one passenger on one train trip, between two stations, in one class. Each ticket has a **PNR** number.

It also stores the seat (if any), booking date, fare and status (CONFIRMED, WAITLIST, CANCELLED, COMPLETED).

- Every ticket must point to a real passenger, trip, class and stations.
- You cannot travel from a station to the same station.
- The key rule: a **confirmed or completed ticket must have a seat**, and a **waitlisted or cancelled ticket must not have a seat**.
- The fare is left empty when inserting; a trigger (Q18) calculates it automatically.

This is the most important table: almost every report reads it.

Concepts used

A **fact/transaction table**, **multiple FKs**, **nullable foreign keys**, **cross-column CHECK constraints encoding business rules**, and **derived data maintained by a trigger** (fare).

Things to watch out for

- With a nullable FK, `JOIN seat st ON st.seat_id = r.seat_id` (inner join) silently drops waitlisted and cancelled tickets. Queries that need them use LEFT JOIN (e.g. `v_ticket_details`, A1).
- `COUNT(seat_id)` counts only confirmed/completed tickets, while `COUNT(*)` counts all; the status summary in Chapter 2 shows both.

Q12 · Unique index ux_res_seat_journey

The query

```
-- No seat can be sold twice on the same journey (function-based unique index ignores NULL seats)
CREATE UNIQUE INDEX ux_res_seat_journey
  ON reservation (CASE WHEN seat_id IS NOT NULL THEN schedule_id END, seat_id);
```

Source: parts/01_ddl.sql, lines 177–179

What it is for

One line: guarantees inside the database that the same seat can never be sold twice on the same journey.

Detailed: This is the most important integrity rule of a reservation system. A simple `UNIQUE (schedule_id, seat_id)` constraint would also work for seats, but this project uses a **function-based unique index**, so the rule ignores tickets that have no seat. Even if two sessions run `book_ticket` at exactly the same moment, only one of them can store the seat; the other gets ORA-00001.

Simple explanation

This is the rule that **stops the same seat from being sold twice on the same trip**.

It is a special kind of index (like the index at the back of a book, which can also forbid duplicates). It only cares about tickets that actually have a seat. For those, the pair "trip + seat" must be unique.

Tickets without a seat (waitlisted or cancelled) are simply left out of this index, so you can have as many waitlisted tickets as you like.

Because this rule lives inside the database, it works even if two people try to book the last seat at the very same moment: only one of them can succeed.

Concepts used

Indexes (a sorted structure, a B-tree, that lets Oracle find rows by key quickly), **unique indexes as constraints**, **function-based indexes** (keys computed from expressions), and a key Oracle rule: **a B-tree index does not store entries whose key columns are all NULL**. Rows with key (NULL, NULL) are simply absent from the index, so they can never violate uniqueness.

Things to watch out for

A cancelled ticket has seat_id NULL (set by cancel_ticket), so its seat is released from the index automatically. If someone set status = 'CANCELLED' but forgot to clear the seat, `ck_res_seat_status` would reject the update, so the seat can never be blocked by a cancelled ticket.

Q13 · Unique index ux_res_pass_journey

The query

```
-- A passenger cannot hold two active tickets on the same journey
CREATE UNIQUE INDEX ux_res_pass_journey
  ON reservation (CASE WHEN status <> 'CANCELLED' THEN passenger_id END,
                  CASE WHEN status <> 'CANCELLED' THEN schedule_id  END);
```

Source: parts/01_ddl.sql, lines 180–183

What it is for

One line: stops one passenger from holding two active (non-cancelled) tickets on the same journey.

Detailed: Business rule BR8: one person, one active ticket per journey. A passenger who **cancels** must still be allowed to book again, so cancelled tickets are excluded from the rule.

Simple explanation

This is the rule that **a passenger cannot hold two active tickets on the same trip**.

It works like Q12: for every ticket that is *not cancelled*, the pair "passenger + trip" must be unique. Cancelled tickets are left out, so a passenger who cancels can book the same trip again.

Concepts used

A **conditional (partial) uniqueness** rule. Some databases have `CREATE UNIQUE INDEX … WHERE status <> 'CANCELLED'` (partial indexes, e.g. PostgreSQL). Oracle has no WHERE clause on indexes, so the same effect is achieved by turning excluded rows into all-NULL keys.

Things to watch out for

`status <> 'CANCELLED'` is never UNKNOWN here because status is NOT NULL. If status were nullable, NULL-status rows would map to NULL keys and escape the rule.

Q14 · Supporting indexes

The query

```
CREATE INDEX ix_res_schedule_status ON reservation (schedule_id, status);

-- Supporting indexes on foreign keys to eliminate unindexed FK locking and speed up joins
CREATE INDEX ix_res_passenger   ON reservation (passenger_id);
CREATE INDEX ix_res_from_stn    ON reservation (from_station);
CREATE INDEX ix_res_to_stn      ON reservation (to_station);
CREATE INDEX ix_res_seat        ON reservation (seat_id);
CREATE INDEX ix_rs_station      ON route_stop (station_code);
CREATE INDEX ix_train_route     ON train (route_id);
CREATE INDEX ix_coach_class     ON coach (class_code);
```

Source: parts/01_ddl.sql, lines 184–193

What it is for

One line: creates eight ordinary (non-unique) indexes that speed up common lookups and joins and avoid locking problems on foreign keys.

Detailed: `ix_res_schedule_status` supports the most frequent question in the project: *which tickets of journey X have status Y?* The other seven index foreign-key columns.

Simple explanation

These are ordinary **indexes**: they make searches faster, like an index in a book helps you find a page without reading the whole book.

- The most important one is on (trip, status) of tickets, because the project constantly asks "show me the tickets of this trip with this status".
- The others are on the columns that link tables (foreign keys). Besides speed, this prevents Oracle from locking a whole table when a linked parent row is deleted.

The downside of indexes: every insert or update must also update the indexes, so writes become a little slower.

Concepts used

B-tree indexes, **composite indexes and column order** (an index on (schedule_id, status) can serve searches on schedule_id alone or on both, but not on status alone, except through a less efficient skip scan), and the **unindexed foreign key** locking issue: in Oracle, when a parent key is deleted or updated and the child FK column has no index, Oracle must lock the whole child table briefly to check for children. With an index, it locks only the relevant index entries.

PERFORMANCE CLAIMS ARE THEORETICAL HERE

No execution plans or timings were measured for this guide. With only 777 tickets Oracle may well choose full table scans, because they are cheap for small tables. Whether an index is actually used can be checked with `EXPLAIN PLAN FOR <query>; SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);` (the *Explain Plan* tab in SQL Developer).

Things to watch out for

`reservation.class_code` and `route.origin_code/dest_code` have no index. This matters little because COACH_CLASS and STATION rows are almost never deleted or have their keys updated (the unindexed-FK lock only happens on parent DELETE or key UPDATE).

Q15 · CREATE TABLE payment

The query

```
-- 1.11 PAYMENT : exactly one payment per reservation (1:1)
CREATE TABLE payment (
  payment_id     NUMBER(8)     CONSTRAINT pk_payment PRIMARY KEY,
  pnr            NUMBER(10)    NOT NULL CONSTRAINT uq_pay_pnr UNIQUE
                 CONSTRAINT fk_pay_res REFERENCES reservation(pnr) ON DELETE CASCADE,
  amount         NUMBER(8,2)   NOT NULL CONSTRAINT ck_pay_amt CHECK (amount > 0),
  pay_mode       VARCHAR2(10)  NOT NULL CONSTRAINT ck_pay_mode CHECK (pay_mode IN ('UPI','CARD','NETBANK
ING','WALLET','CASH')),
  paid_on        DATE          NOT NULL,
  pay_status     VARCHAR2(14)  DEFAULT 'SUCCESS' NOT NULL
                 CONSTRAINT ck_pay_status CHECK (pay_status IN ('SUCCESS','REFUNDED','PARTIAL_REFUND'))
);
```

Source: parts/01_ddl.sql, lines 195–205

What it is for

One line: stores exactly one payment for each ticket.

Detailed: Payment data is kept apart from the ticket because it is a different concept (money movement), has its own lifecycle (SUCCESS → PARTIAL_REFUND/REFUNDED), and its own identifier.

Simple explanation

This creates the **PAYMENT** table: the payment made for each ticket.

It stores payment id, the ticket (PNR), amount, mode (UPI, CARD…), date and status (SUCCESS, PARTIAL_REFUND, REFUNDED).

- Each ticket can have **only one** payment (one-to-one), because the PNR here is unique.
- If a ticket is deleted, its payment is deleted too.

Concepts used

A **one-to-one relationship implemented with FK + UNIQUE**, and **ON DELETE CASCADE** for dependent data.

Things to watch out for

Payment `amount` is copied from the fare at booking time. If the fare changes later (as in the boarding-point change, Q40), the payment must be updated too, which Q41 does. Forgetting it would make `amount <> fare`; test suite 02 invariant 10 checks exactly this.

Q16 · CREATE TABLE cancellation

The query

```
-- 1.12 CANCELLATION : at most one cancellation per reservation (1:0..1)
CREATE TABLE cancellation (
  cancel_id      NUMBER(8)     CONSTRAINT pk_cancellation PRIMARY KEY,
  pnr            NUMBER(10)    NOT NULL CONSTRAINT uq_cancel_pnr UNIQUE
                 CONSTRAINT fk_cancel_res REFERENCES reservation(pnr) ON DELETE CASCADE,
  cancelled_on   DATE          NOT NULL,
  refund_amount  NUMBER(8,2)   DEFAULT 0 NOT NULL CONSTRAINT ck_cancel_refund CHECK (refund_amount >= 0)
,
  reason         VARCHAR2(60)
);
```

Source: parts/01_ddl.sql, lines 207–215

What it is for

One line: records the cancellation, if any, of a ticket, with the refund given and the reason.

Detailed: Most tickets are never cancelled. Putting cancellation data in its own table avoids four mostly-NULL columns in RESERVATION.

Simple explanation

This creates the **CANCELLATION** table: a record for every ticket that was cancelled, with the date, the refund amount and the reason.

- A ticket can be cancelled **at most once**.
- Most tickets are never cancelled, so keeping this in a separate table avoids lots of empty columns in the ticket table.

Concepts used

Optional participation (1 : 0..1), avoiding NULL-heavy tables (a sign of good normalization), DEFAULT values.

Things to watch out for

Nothing stops refund_amount being larger than the fare. `cancel_ticket` never does this (maximum 100% for waitlisted tickets), but a CHECK cannot compare it with RESERVATION.fare (another table).

Q17 · Sequences

The query

```
CREATE SEQUENCE seq_pnr     START WITH 4200001001 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_payment START WITH 901001     INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_cancel  START WITH 701001     INCREMENT BY 1 NOCACHE;
```

Source: parts/01_ddl.sql, lines 217–219

What it is for

One line: creates three number generators for new PNRs, payment ids and cancellation ids.

Detailed: When `book_ticket` creates a ticket it needs a new, never-used PNR. A **sequence** hands out the next number with `seq_pnr.NEXTVAL`, safely even when many sessions book at once.

Simple explanation

These create three **sequences**: automatic number machines.

Every time you ask `seq_pnr.NEXTVAL`, Oracle gives you the next number: 4200001001, then 4200001002, and so on. They are used for new PNRs, payment ids and cancellation ids.

They start above the numbers already used in the sample data, so there are no clashes. Numbers may sometimes be skipped (for example if a booking is undone), but they are never repeated.

Concepts used

Sequences (independent objects that generate unique numbers), **NEXTVAL/CURRVAL**, **surrogate keys**, and the fact that sequences are **non-transactional**: a number taken by a transaction that rolls back is not returned, so gaps appear.

Things to watch out for

- Sequences guarantee **uniqueness, not gap-free numbering**. Test suites 03–05 call book_ticket and then ROLLBACK, so their PNRs are consumed and lost. That is normal.
- `CURRVAL` can only be used after `NEXTVAL` in the same session (ORA-08002 otherwise).

Q18 · Trigger trg_reservation_validate

The query

```
-- 1.13 TRIGGER : business rules that span several tables (cannot be CHECK constraints)
--   R1 boarding station must come before the destination on the train's route
--   R2 the seat must belong to a coach of THIS train and of the booked class
--   R3 a ticket cannot be booked after the journey date
--   R4 fare = max(distance, 50 km) * fare_per_km + reservation fee  (auto-calculated)
-- ------------------------------------------------------------------------------------
CREATE OR REPLACE TRIGGER trg_reservation_validate
BEFORE INSERT OR UPDATE OF schedule_id, class_code, seat_id, from_station, to_station, fare
ON reservation
FOR EACH ROW
DECLARE
  v_train     train.train_no%TYPE;
  v_route     train.route_id%TYPE;
  v_jdate     train_schedule.journey_date%TYPE;
  v_from_seq  route_stop.stop_seq%TYPE;
  v_to_seq    route_stop.stop_seq%TYPE;
  v_from_km   route_stop.km_from_origin%TYPE;
  v_to_km     route_stop.km_from_origin%TYPE;
  v_seat_trn  train.train_no%TYPE;
  v_seat_cls  coach_class.class_code%TYPE;
  v_rate      coach_class.fare_per_km%TYPE;
  v_fee       coach_class.reservation_fee%TYPE;
BEGIN
  SELECT t.train_no, t.route_id, s.journey_date
    INTO v_train, v_route, v_jdate
    FROM train_schedule s JOIN train t ON t.train_no = s.train_no
   WHERE s.schedule_id = :NEW.schedule_id;

  -- R1
  BEGIN
    SELECT stop_seq, km_from_origin INTO v_from_seq, v_from_km
      FROM route_stop WHERE route_id = v_route AND station_code = :NEW.from_station;
    SELECT stop_seq, km_from_origin INTO v_to_seq, v_to_km
      FROM route_stop WHERE route_id = v_route AND station_code = :NEW.to_station;
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      RAISE_APPLICATION_ERROR(-20001, 'Station is not on the route of train ' || v_train);
  END;
  IF v_from_seq >= v_to_seq THEN
    RAISE_APPLICATION_ERROR(-20002, 'Boarding station must come before destination on the route');
  END IF;

  -- R2
  IF :NEW.seat_id IS NOT NULL THEN
    SELECT c.train_no, c.class_code INTO v_seat_trn, v_seat_cls
      FROM seat st JOIN coach c ON c.coach_id = st.coach_id
     WHERE st.seat_id = :NEW.seat_id;
    IF v_seat_trn <> v_train OR v_seat_cls <> :NEW.class_code THEN
      RAISE_APPLICATION_ERROR(-20003, 'Seat ' || :NEW.seat_id || ' does not belong to train '
                              || v_train || ' / class ' || :NEW.class_code);
    END IF;
  END IF;

  -- R3
  IF INSERTING AND :NEW.booking_date > v_jdate THEN
    RAISE_APPLICATION_ERROR(-20004, 'Booking date cannot be after the journey date');
  END IF;

  -- R4
  IF :NEW.fare IS NULL THEN
    SELECT fare_per_km, reservation_fee INTO v_rate, v_fee
```

```
      FROM coach_class WHERE class_code = :NEW.class_code;
    :NEW.fare := ROUND(GREATEST(v_to_km - v_from_km, 50) * v_rate + v_fee, 0);
  END IF;
END;
/
```

Source: parts/01_ddl.sql, lines 222–287

What it is for

One line: before any ticket is inserted or changed, it checks the route direction and the seat, blocks booking after the journey date, and calculates the fare.

Detailed: These four business rules (R1–R4) involve several tables, so they cannot be CHECK constraints. A **trigger** is PL/SQL code that Oracle runs automatically on a table event. Because it is BEFORE and FOR EACH ROW, it runs once for every row, before the row is written, and can both reject the row and change its values.

Simple explanation

A **trigger** is code that runs automatically whenever something happens to a table, like an alarm that goes off by itself.

This trigger runs **before every ticket is saved or changed**, and does four things:

1. **Direction check:** both stations must be on the train's route, and the boarding station must come before the destination. (You cannot book Bengaluru → Chennai on a Chennai → Bengaluru train.)
2. **Seat check:** if a seat is given, it must belong to a coach of this train and of the booked class.
3. **Date check:** you cannot book after the trip date.
4. **Fare calculation:** if no fare is given, it calculates it: distance (at least 50 km) × class price per km + reservation fee, rounded to whole rupees.

If any check fails, the ticket is rejected with a clear error message. Because it runs *before* saving, it can also fill in the fare.

Concepts used

Triggers (event, timing, level), **BEFORE vs AFTER**, **row-level vs statement-level**, **:NEW/:OLD pseudo-records**, **conditional predicates** (INSERTING/UPDATING/DELETING), **SELECT INTO and its exceptions**, **nested blocks with exception handlers**, **user-defined errors**, **derived values**, and **multi-table business rules** that declarative constraints cannot express.

Things to watch out for

- **Invalid schedule_id:** the first SELECT INTO finds nothing and raises **ORA-01403 no data found** (not handled), *before* the foreign-key check could report ORA-02291. The insert is still rejected, but the message is less clear.
- **Invalid seat_id:** the same happens in R2.
- **UPDATE of from_station without setting fare to NULL:** R1 is re-checked, but R4 is skipped because a fare already exists, so the old fare stays. That is why Q40 sets `fare = NULL` together with the station change.
- **Multi-row INSERT...SELECT:** the trigger fires once per row: the 768 sample tickets fire it 768 times.

4. Files 02_data.sql and 02b_seat.sql — Loading the Sample Data

What these files do: they fill the empty tables with a realistic snapshot of a small railway: 14 stations, 4 routes, 21 stops, 6 classes, 9 trains, 30 coaches, 320 seats, 27 journeys, 150 passengers and 768 tickets. Everything here is **DML (Data Manipulation Language)**. The file ends with one `COMMIT`.

INSERT BASICS — READ THIS ONCE

`INSERT INTO table VALUES (v1, v2, …)` must supply a value for **every** column, in the table's column order. `INSERT INTO table (col1, col2) VALUES (v1, v2)` supplies only the listed columns; the others get their DEFAULT, or NULL if there is none. `DATE '2026-09-20'` is an **ANSI date literal**: always year-month-day, independent of session settings, so it is safer than `'20-SEP-2026'`, which depends on NLS_DATE_FORMAT. Every INSERT is checked against all constraints and fires the row triggers on its table. A failing INSERT rolls back only that statement, not the earlier ones.

Load order follows the foreign keys: parents first (STATION → ROUTE → ROUTE_STOP → COACH_CLASS → TRAIN → COACH → SEAT → TRAIN_SCHEDULE → PASSENGER → RESERVATION). Inserting a child before its parent fails with ORA-02291.

Q19 · INSERT stations (2.1)

The query

```
-- 2.1 STATION
INSERT INTO station VALUES ('MAS', 'Chennai Central', 'Chennai', 'Tamil Nadu', 'SR', 17);
INSERT INTO station VALUES ('KPD', 'Katpadi Junction', 'Vellore', 'Tamil Nadu', 'SR', 5);
INSERT INTO station VALUES ('JTJ', 'Jolarpettai Junction', 'Jolarpettai', 'Tamil Nadu', 'SR', 5);
INSERT INTO station VALUES ('SBC', 'KSR Bengaluru City', 'Bengaluru', 'Karnataka', 'SWR', 10);
INSERT INTO station VALUES ('VM', 'Villupuram Junction', 'Villupuram', 'Tamil Nadu', 'SR', 6);
INSERT INTO station VALUES ('TPJ', 'Tiruchchirappalli Junction', 'Tiruchirappalli', 'Tamil Nadu', 'SR',
8);
INSERT INTO station VALUES ('DG', 'Dindigul Junction', 'Dindigul', 'Tamil Nadu', 'SR', 5);
INSERT INTO station VALUES ('MDU', 'Madurai Junction', 'Madurai', 'Tamil Nadu', 'SR', 7);
INSERT INTO station VALUES ('SA', 'Salem Junction', 'Salem', 'Tamil Nadu', 'SR', 6);
INSERT INTO station VALUES ('ED', 'Erode Junction', 'Erode', 'Tamil Nadu', 'SR', 5);
INSERT INTO station VALUES ('CBE', 'Coimbatore Junction', 'Coimbatore', 'Tamil Nadu', 'SR', 6);
INSERT INTO station VALUES ('PGT', 'Palakkad Junction', 'Palakkad', 'Kerala', 'SR', 5);
INSERT INTO station VALUES ('ERS', 'Ernakulam Junction', 'Kochi', 'Kerala', 'SR', 6);
INSERT INTO station VALUES ('TVC', 'Thiruvananthapuram Central', 'Thiruvananthapuram', 'Kerala', 'SR',
5);
```

Source: parts/02_data.sql, lines 1–15

What it is for

One line: inserts the 14 stations of the network.

Detailed: Every other table refers to stations, so they must exist first. The stations cover four corridors from Chennai and Bengaluru.

Simple explanation

These lines add the **14 stations** to the STATION table, one station per line.

They are added first because everything else (routes, stops, tickets) refers to stations.

Concepts used

DML INSERT with positional VALUES, constraint checking on insert (uppercase code, platforms 1–30, NOT NULLs, PK uniqueness), statement-level atomicity.

Things to watch out for

Apostrophes inside text must be doubled: `'Devi''s Halt'`. After the ALTER in 03_dml.sql widens `station_name`, longer names are allowed.

Q20 · INSERT routes and route stops (2.2)

The query

```
-- 2.2 ROUTE and ROUTE_STOP
INSERT INTO route VALUES (1, 'Chennai - Bengaluru', 'MAS', 'SBC', 359);
INSERT INTO route VALUES (2, 'Chennai - Madurai', 'MAS', 'MDU', 493);
INSERT INTO route VALUES (3, 'Chennai - Coimbatore', 'MAS', 'CBE', 494);
INSERT INTO route VALUES (4, 'Bengaluru - Ernakulam', 'SBC', 'ERS', 588);
INSERT INTO route_stop VALUES (1, 1, 'MAS', 0);
INSERT INTO route_stop VALUES (1, 2, 'KPD', 129);
INSERT INTO route_stop VALUES (1, 3, 'JTJ', 213);
INSERT INTO route_stop VALUES (1, 4, 'SBC', 359);
INSERT INTO route_stop VALUES (2, 1, 'MAS', 0);
INSERT INTO route_stop VALUES (2, 2, 'VM', 159);
INSERT INTO route_stop VALUES (2, 3, 'TPJ', 337);
INSERT INTO route_stop VALUES (2, 4, 'DG', 431);
INSERT INTO route_stop VALUES (2, 5, 'MDU', 493);
INSERT INTO route_stop VALUES (3, 1, 'MAS', 0);
INSERT INTO route_stop VALUES (3, 2, 'KPD', 129);
INSERT INTO route_stop VALUES (3, 3, 'JTJ', 213);
INSERT INTO route_stop VALUES (3, 4, 'SA', 334);
INSERT INTO route_stop VALUES (3, 5, 'ED', 394);
INSERT INTO route_stop VALUES (3, 6, 'CBE', 494);
INSERT INTO route_stop VALUES (4, 1, 'SBC', 0);
INSERT INTO route_stop VALUES (4, 2, 'SA', 198);
INSERT INTO route_stop VALUES (4, 3, 'ED', 258);
INSERT INTO route_stop VALUES (4, 4, 'CBE', 358);
INSERT INTO route_stop VALUES (4, 5, 'PGT', 412);
INSERT INTO route_stop VALUES (4, 6, 'ERS', 588);
```

Source: parts/02_data.sql, lines 17–42

What it is for

One line: creates the four corridors and their 21 ordered stops with distances.

Detailed: The ROUTE rows must be inserted before ROUTE_STOP rows (FK). Each stop's `km_from_origin` is what the trigger later subtracts to compute ticket distances.

Simple explanation

These lines add the **4 routes** and then the **21 stops** of those routes, with the distance of each stop from the start.

Routes are added before stops because each stop must refer to an existing route.

Concepts used

Parent-child insertion order, composite primary keys, and FK checks on two parents (ROUTE_STOP references both ROUTE and STATION).

Things to watch out for

Stop distances must increase with stop_seq for fares to make sense; this is not enforced.

Q21 · INSERT coach classes (2.3)

The query

```
-- 2.3 COACH_CLASS  (seats_per_coach is scaled down for demonstration)
INSERT INTO coach_class VALUES ('1A', 'First AC', 4, 4.20, 60);
INSERT INTO coach_class VALUES ('2A', 'AC 2-Tier', 8, 2.50, 50);
INSERT INTO coach_class VALUES ('3A', 'AC 3-Tier', 10, 1.80, 40);
INSERT INTO coach_class VALUES ('SL', 'Sleeper', 12, 0.65, 20);
INSERT INTO coach_class VALUES ('CC', 'AC Chair Car', 12, 1.45, 40);
INSERT INTO coach_class VALUES ('EC', 'Executive Chair Car', 8, 2.90, 60);
```

Source: parts/02_data.sql, lines 44–50

What it is for

One line: loads the six travel classes with their seats per coach, fare per km and reservation fee.

Detailed: These values decide every fare and every train's capacity. The comment notes that seats per coach are scaled down for demonstration (real coaches have 50–80 seats).

Simple explanation

These lines add the **6 travel classes** with their seats per coach and prices.

The number of seats per coach is kept small on purpose, so trains fill up quickly and waitlists can be demonstrated with a small amount of data.

Concepts used

Reference (lookup) data, decimal numbers in `NUMBER(5,2)`.

Things to watch out for

Must be loaded before COACH (FK) and before any ticket (the trigger reads it).

Q22 · INSERT trains (2.4)

The query

```
-- 2.4 TRAIN
INSERT INTO train (train_no, train_name, train_type, route_id, dep_time) VALUES (12639, 'Brindavan
Express', 'SUPERFAST', 1, '07:50');
INSERT INTO train (train_no, train_name, train_type, route_id, dep_time) VALUES (12607, 'Lalbagh
Express', 'SUPERFAST', 1, '15:35');
INSERT INTO train (train_no, train_name, train_type, route_id, dep_time) VALUES (12027, 'Chennai
Shatabdi', 'SHATABDI', 1, '17:30');
INSERT INTO train (train_no, train_name, train_type, route_id, dep_time) VALUES (12635, 'Vaigai
Express', 'SUPERFAST', 2, '13:50');
INSERT INTO train (train_no, train_name, train_type, route_id, dep_time) VALUES (12637, 'Pandian
Express', 'SUPERFAST', 2, '21:40');
INSERT INTO train (train_no, train_name, train_type, route_id, dep_time) VALUES (22671, 'Madurai
Tejas', 'TEJAS', 2, '06:00');
INSERT INTO train (train_no, train_name, train_type, route_id, dep_time) VALUES (12675, 'Kovai
Express', 'SUPERFAST', 3, '06:10');
INSERT INTO train (train_no, train_name, train_type, route_id, dep_time) VALUES (12671, 'Nilgiri
Express', 'SUPERFAST', 3, '21:05');
INSERT INTO train (train_no, train_name, train_type, route_id, dep_time) VALUES (12677, 'Ernakulam
Intercity', 'INTERCITY', 4, '06:10');
```

Source: parts/02_data.sql, lines 52–61

What it is for

One line: inserts the nine train services and assigns each to a route.

Detailed: These INSERTs use an **explicit column list**, so `run_days` and `is_active` are omitted and receive their DEFAULTs ('YYYYYYY' and 'Y').

Simple explanation

These lines add the **9 trains**.

The INSERTs list only some columns. The missing ones (run days and active flag) get their **default** values automatically ('YYYYYYY' and 'Y').

Concepts used

Column lists and DEFAULT values, regex CHECK on dep_time, FK to ROUTE.

Things to watch out for

If a column in the list has no DEFAULT and no value is given, it becomes NULL, and fails if NOT NULL.

Q23 · INSERT coaches (2.5)

The query

```
-- 2.5 COACH
INSERT INTO coach VALUES (1, 12639, 'C1', 'CC');
INSERT INTO coach VALUES (2, 12639, 'C2', 'CC');
INSERT INTO coach VALUES (3, 12639, 'E1', 'EC');
INSERT INTO coach VALUES (4, 12607, 'C1', 'CC');
INSERT INTO coach VALUES (5, 12607, 'C2', 'CC');
INSERT INTO coach VALUES (6, 12607, 'C3', 'CC');
INSERT INTO coach VALUES (7, 12027, 'C1', 'CC');
INSERT INTO coach VALUES (8, 12027, 'C2', 'CC');
INSERT INTO coach VALUES (9, 12027, 'E1', 'EC');
INSERT INTO coach VALUES (10, 12635, 'C1', 'CC');
INSERT INTO coach VALUES (11, 12635, 'C2', 'CC');
INSERT INTO coach VALUES (12, 12635, 'C3', 'CC');
INSERT INTO coach VALUES (13, 12637, 'S1', 'SL');
INSERT INTO coach VALUES (14, 12637, 'S2', 'SL');
INSERT INTO coach VALUES (15, 12637, 'B1', '3A');
INSERT INTO coach VALUES (16, 12637, 'A1', '2A');
INSERT INTO coach VALUES (17, 22671, 'C1', 'CC');
INSERT INTO coach VALUES (18, 22671, 'C2', 'CC');
INSERT INTO coach VALUES (19, 22671, 'E1', 'EC');
INSERT INTO coach VALUES (20, 12675, 'C1', 'CC');
INSERT INTO coach VALUES (21, 12675, 'C2', 'CC');
INSERT INTO coach VALUES (22, 12675, 'E1', 'EC');
INSERT INTO coach VALUES (23, 12671, 'S1', 'SL');
INSERT INTO coach VALUES (24, 12671, 'S2', 'SL');
INSERT INTO coach VALUES (25, 12671, 'B1', '3A');
INSERT INTO coach VALUES (26, 12671, 'A1', '2A');
INSERT INTO coach VALUES (27, 12671, 'H1', '1A');
INSERT INTO coach VALUES (28, 12677, 'C1', 'CC');
INSERT INTO coach VALUES (29, 12677, 'C2', 'CC');
INSERT INTO coach VALUES (30, 12677, 'E1', 'EC');
```

Source: parts/02_data.sql, lines 63–93

What it is for

One line: attaches 30 coaches to the nine trains, each with a label and class.

Detailed: The coach composition defines each train's capacity and which classes it offers. `book_ticket` refuses a class the train does not have (ORA-20011).

Simple explanation

These lines attach **30 coaches** to the 9 trains, saying which train and which class each coach belongs to.

The coach ids are chosen carefully so seat ids are easy to read (see Q24).

Concepts used

FKs to two parents (TRAIN, COACH_CLASS); composite UNIQUE (train_no, coach_label).

Things to watch out for

A train without coaches would disappear from `v_reservation_performance`, which inner-joins to the capacity subquery.

Q24 · Seat generator: INSERT … SELECT with CONNECT BY (2.6 and 02b_seat.sql)

The query

From **02_data.sql**:

```
-- 2.6 SEAT : generated from COACH x COACH_CLASS (seat_id = coach_id*100 + seat_no)
--     berth pattern: SL/3A bay of 8, 2A bay of 6, 1A lower/upper, chair cars window/middle/aisle
INSERT INTO seat (seat_id, coach_id, seat_no, berth_type)
SELECT c.coach_id * 100 + n.lvl, c.coach_id, n.lvl,
       CASE
         WHEN c.class_code IN ('SL','3A') THEN
              DECODE(MOD(n.lvl, 8), 1,'LOWER', 2,'MIDDLE', 3,'UPPER', 4,'LOWER', 5,'MIDDLE', 6,'UPPER',
7,'SIDE LOWER', 'SIDE UPPER')
         WHEN c.class_code = '2A' THEN
              DECODE(MOD(n.lvl, 6), 1,'LOWER', 2,'UPPER', 3,'LOWER', 4,'UPPER', 5,'SIDE LOWER', 'SIDE
UPPER')
         WHEN c.class_code = '1A' THEN DECODE(MOD(n.lvl, 2), 1, 'LOWER', 'UPPER')
         WHEN c.class_code = 'EC' THEN DECODE(MOD(n.lvl, 2), 1, 'WINDOW', 'AISLE')
         ELSE DECODE(MOD(n.lvl, 3), 1, 'WINDOW', 2, 'MIDDLE SEAT', 'AISLE')
       END
FROM   coach c
       JOIN coach_class cc ON cc.class_code = c.class_code
       JOIN (SELECT LEVEL AS lvl FROM dual CONNECT BY LEVEL <= 100) n ON n.lvl <= cc.seats_per_coach;
```

Source: parts/02_data.sql, lines 96–111

The re-runnable version in **02b_seat.sql**:

```
INSERT INTO seat (seat_id, coach_id, seat_no, berth_type)
SELECT c.coach_id * 100 + n.lvl, c.coach_id, n.lvl,
       CASE
         WHEN c.class_code IN ('SL','3A') THEN
              DECODE(MOD(n.lvl, 8), 1,'LOWER', 2,'MIDDLE', 3,'UPPER', 4,'LOWER', 5,'MIDDLE', 6,'UPPER',
7,'SIDE LOWER', 'SIDE UPPER')
         WHEN c.class_code = '2A' THEN
              DECODE(MOD(n.lvl, 6), 1,'LOWER', 2,'UPPER', 3,'LOWER', 4,'UPPER', 5,'SIDE LOWER', 'SIDE
UPPER')
         WHEN c.class_code = '1A' THEN DECODE(MOD(n.lvl, 2), 1, 'LOWER', 'UPPER')
         WHEN c.class_code = 'EC' THEN DECODE(MOD(n.lvl, 2), 1, 'WINDOW', 'AISLE')
         ELSE DECODE(MOD(n.lvl, 3), 1, 'WINDOW', 2, 'MIDDLE SEAT', 'AISLE')
       END
FROM   coach c
       JOIN coach_class cc ON cc.class_code = c.class_code
       JOIN (SELECT LEVEL AS lvl FROM dual CONNECT BY LEVEL <= 100) n ON n.lvl <= cc.seats_per_coach
WHERE  NOT EXISTS (
       SELECT 1 FROM seat s WHERE s.seat_id = c.coach_id * 100 + n.lvl
);

COMMIT;
```

Source: parts/02b_seat.sql, lines 7–25

What it is for

One line: generates all 320 seats in one statement: for each coach, seat numbers 1 to seats_per_coach, each with the correct berth type.

Detailed: Typing 320 INSERTs would be slow and error-prone. This query multiplies each coach by a generated list of numbers 1–100 and keeps only the numbers up to that coach's class capacity. The 02b version adds `WHERE NOT EXISTS (…)`, so running it again inserts only missing seats.

Simple explanation

Instead of typing 320 seat rows by hand, **one single statement creates all the seats**.

How it works, in simple steps:

1. It makes a list of numbers 1, 2, 3 … 100 (using a trick called `CONNECT BY LEVEL`).
2. For each coach, it takes the numbers up to that coach's number of seats (12 numbers for a 12-seat coach).
3. Each number becomes a seat. The seat id is `coach id × 100 + seat number`, so seat 205 means coach 2, seat 5.
4. The seat type (LOWER, MIDDLE, UPPER, WINDOW…) is chosen by a repeating pattern depending on the class.

The version in `02b_seat.sql` adds one extra rule, "only add a seat if it doesn't already exist", so it can be run again safely without errors.

Concepts used

INSERT … SELECT (set-based DML), **row generators**, **hierarchical queries** (CONNECT BY, LEVEL), **non-equi joins**, **CASE vs DECODE**, **MOD**, **correlated NOT EXISTS**, **idempotent scripts**.

Things to watch out for

DECODE treats two NULLs as equal while CASE does not (`CASE x WHEN NULL` never matches). Not an issue here since MOD never returns NULL for non-NULL input.

Q25 · INSERT train schedules (2.7)

The query

```
-- 2.7 TRAIN_SCHEDULE  (two completed journeys + one upcoming journey per train)
INSERT INTO train_schedule VALUES (101, 12639, DATE '2026-09-20', 'COMPLETED');
INSERT INTO train_schedule VALUES (102, 12639, DATE '2026-10-04', 'COMPLETED');
INSERT INTO train_schedule VALUES (103, 12639, DATE '2026-10-21', 'SCHEDULED');
INSERT INTO train_schedule VALUES (104, 12607, DATE '2026-09-21', 'COMPLETED');
INSERT INTO train_schedule VALUES (105, 12607, DATE '2026-10-05', 'COMPLETED');
INSERT INTO train_schedule VALUES (106, 12607, DATE '2026-10-22', 'SCHEDULED');
INSERT INTO train_schedule VALUES (107, 12027, DATE '2026-09-22', 'COMPLETED');
INSERT INTO train_schedule VALUES (108, 12027, DATE '2026-10-06', 'COMPLETED');
INSERT INTO train_schedule VALUES (109, 12027, DATE '2026-10-23', 'SCHEDULED');
INSERT INTO train_schedule VALUES (110, 12635, DATE '2026-09-23', 'COMPLETED');
INSERT INTO train_schedule VALUES (111, 12635, DATE '2026-10-07', 'COMPLETED');
INSERT INTO train_schedule VALUES (112, 12635, DATE '2026-10-24', 'SCHEDULED');
INSERT INTO train_schedule VALUES (113, 12637, DATE '2026-09-24', 'COMPLETED');
INSERT INTO train_schedule VALUES (114, 12637, DATE '2026-10-08', 'COMPLETED');
INSERT INTO train_schedule VALUES (115, 12637, DATE '2026-10-25', 'SCHEDULED');
INSERT INTO train_schedule VALUES (116, 22671, DATE '2026-09-25', 'COMPLETED');
INSERT INTO train_schedule VALUES (117, 22671, DATE '2026-10-09', 'COMPLETED');
INSERT INTO train_schedule VALUES (118, 22671, DATE '2026-10-21', 'SCHEDULED');
INSERT INTO train_schedule VALUES (119, 12675, DATE '2026-09-26', 'COMPLETED');
INSERT INTO train_schedule VALUES (120, 12675, DATE '2026-10-10', 'COMPLETED');
INSERT INTO train_schedule VALUES (121, 12675, DATE '2026-10-22', 'SCHEDULED');
INSERT INTO train_schedule VALUES (122, 12671, DATE '2026-09-27', 'COMPLETED');
INSERT INTO train_schedule VALUES (123, 12671, DATE '2026-10-11', 'COMPLETED');
INSERT INTO train_schedule VALUES (124, 12671, DATE '2026-10-23', 'SCHEDULED');
INSERT INTO train_schedule VALUES (125, 12677, DATE '2026-09-28', 'COMPLETED');
INSERT INTO train_schedule VALUES (126, 12677, DATE '2026-10-12', 'COMPLETED');
INSERT INTO train_schedule VALUES (127, 12677, DATE '2026-10-24', 'SCHEDULED');
```

Source: parts/02_data.sql, lines 113–140

What it is for

One line: creates 27 dated journeys: two completed and one upcoming for each of the nine trains.

Detailed: Completed journeys feed the historical reports; SCHEDULED journeys can still be booked and cancelled.

Simple explanation

These lines add the **27 train trips** (schedules): for each of the 9 trains, two trips that have already happened (COMPLETED) and one upcoming trip (SCHEDULED).

Dates are written as `DATE '2026-09-20'`, which always means year-month-day, so they work on any computer.

Concepts used

ANSI DATE literals, CHECK on TRUNC, composite UNIQUE (train_no, journey_date).

DATES RELATIVE TO THE RUN DATE

These dates are fixed. On 28-Oct-2026, the "SCHEDULED" journeys dated 21–25 Oct are already in the past. The reports treat them as normal data, but `cancel_ticket` would refuse to cancel their tickets ("Train has already departed"), and the trigger would refuse **new** bookings on them because the booking date would be after the journey date. That is why the demo creates a fresh journey 130 at `TRUNC(SYSDATE) + 7`.

Things to watch out for

Oracle DATE arithmetic: `journey_date - booking_date` is a number of days, which the cancellation logic uses.

Q26 · INSERT passengers (2.8)

The query

```
-- 2.8 PASSENGER
INSERT INTO passenger VALUES (1, 'Balaji Mani', 'M', DATE '1993-07-10', '7624642010', 'balaji.mani1@mail
.com', 'Kochi', DATE '2025-01-28');
INSERT INTO passenger VALUES (2, 'Ajay Thomas', 'M', DATE '1997-02-24', '8518184808', 'ajay.thomas2@mail
.com', 'Vellore', DATE '2025-11-12');
INSERT INTO passenger VALUES (3, 'Nithya Balaji', 'F', DATE '2006-08-11', '8386632201', 'nithya.balaji3@
mail.com', 'Chennai', DATE '2025-04-03');
INSERT INTO passenger VALUES (4, 'Manoj Chandran', 'M', DATE '1958-03-25', '8935800437', 'manoj.chandran
4@mail.com', 'Chennai', DATE '2025-09-28');
INSERT INTO passenger VALUES (5, 'Priya Pandian', 'F', DATE '1982-11-04', '7031005178', 'priya.pandian5@
mail.com', 'Tiruchirappalli', DATE '2025-08-26');
INSERT INTO passenger VALUES (6, 'Janani Shankar', 'F', DATE '1983-09-28', '7551561306', 'janani.shankar
6@mail.com', 'Bengaluru', DATE '2025-04-19');
-- … (remaining lines follow the same pattern)
```

Source: parts/02_data.sql, from line 142 (first 7 lines shown)

What it is for

One line: registers 150 passengers.

Detailed: Passengers have varied ages (for the concession update), home cities (for A7's city comparison) and unique phones and e-mails.

Simple explanation

These lines add the **150 passengers**, each with a valid phone number and e-mail.

They give a value for every column in order, so no column name list is written.

Concepts used

Regex CHECKs on phone and email, UNIQUE on both.

Things to watch out for

Positional INSERTs break silently if two columns of the same type are swapped (e.g. dob and registered_on): no error, wrong data. Column lists prevent that.

Q27 · INSERT reservations (2.9)

The query

```
-- 2.9 RESERVATION  (fare is calculated by trigger TRG_RESERVATION_VALIDATE)
INSERT INTO reservation (pnr, passenger_id, schedule_id, class_code, seat_id, from_station, to_station,
booking_date, status) VALUES (4200000001, 11, 101, 'CC', 205, 'MAS', 'SBC', DATE '2026-08-11', 'COMPLETE
D');
INSERT INTO reservation (pnr, passenger_id, schedule_id, class_code, seat_id, from_station, to_station,
booking_date, status) VALUES (4200000002, 16, 101, 'CC', 103, 'KPD', 'SBC', DATE '2026-08-12', 'COMPLETE
D');
INSERT INTO reservation (pnr, passenger_id, schedule_id, class_code, seat_id, from_station, to_station,
booking_date, status) VALUES (4200000003, 108, 101, 'CC', 112, 'MAS', 'SBC', DATE '2026-08-12', 'COMPLET
ED');
-- … (remaining lines follow the same pattern)
```

Source: parts/02_data.sql, from line 294 (first 4 lines shown)

(768 INSERT statements in total, PNRs 4200000001–4200000768.)

What it is for

One line: loads 768 historical and upcoming tickets; the trigger validates each one and calculates its fare.

Detailed: These rows are the raw material for every report. They include COMPLETED tickets on past journeys, CONFIRMED tickets on upcoming journeys, 20 WAITLIST tickets and 35 CANCELLED tickets (seat NULL).

Simple explanation

These lines add the **768 tickets**.

Notice that the fare is **not** given. For every ticket, the trigger (Q18) checks the route, direction and seat, and calculates the fare by itself. Then the database checks all the rules: confirmed tickets have seats, waitlisted and cancelled ones don't, and no seat is sold twice.

Concepts used

Triggers firing on INSERT, BEFORE-trigger value assignment, unique function-based indexes, CHECK `ck_res_seat_status`, six foreign keys per row.

Things to watch out for

Every INSERT runs the trigger's 4–6 queries, so loading 768 rows performs a few thousand small lookups; still fast. For millions of rows, direct-path loads or disabling triggers temporarily would be considered.

5. File 03_dml.sql — Modification, Integrity Tests and Stored Procedures

What this file does: it completes the sample data (payments and cancellations, derived from the tickets), changes the table structure with ALTER, demonstrates UPDATE and DELETE, proves that the constraints and the trigger reject bad data, and creates the two stored procedures that implement booking and cancellation, followed by a live demo.

Q28 · Generate payments with INSERT … SELECT (2.10)

The query

```
-- 2.10 PAYMENT : one payment per reservation (INSERT ... SELECT)
INSERT INTO payment (payment_id, pnr, amount, pay_mode, paid_on, pay_status)
SELECT 900000 + ROW_NUMBER() OVER (ORDER BY r.pnr),
       r.pnr,
       r.fare,
       DECODE(MOD(r.pnr, 5), 0, 'UPI', 1, 'CARD', 2, 'UPI', 3, 'NETBANKING', 'WALLET'),
       r.booking_date,
       CASE WHEN r.status = 'CANCELLED' THEN 'PARTIAL_REFUND'
            WHEN r.status = 'WAITLIST' AND s.sched_status = 'COMPLETED' THEN 'REFUNDED'
            ELSE 'SUCCESS' END
FROM   reservation r JOIN train_schedule s ON s.schedule_id = r.schedule_id;
```

Source: parts/03_dml.sql, lines 2–12

What it is for

One line: creates one payment row for every ticket, with the amount equal to the fare and a status that reflects the ticket's fate.

Detailed: Instead of writing 768 payment INSERTs, the payments are **derived** from the reservations, which guarantees one payment per ticket and `amount = fare`.

Simple explanation

This creates **one payment for every ticket** with a single statement, instead of 768 separate ones.

- The payment amount is copied from the ticket's fare.
- Payment ids are numbered 900001, 900002, … in PNR order (`ROW_NUMBER`).
- The payment mode is spread across UPI, CARD, NETBANKING and WALLET using a simple formula.
- The status is PARTIAL_REFUND for cancelled tickets, REFUNDED for waitlisted tickets on trips that already happened (they never got a seat), and SUCCESS for everything else.

Concepts used

INSERT … SELECT, **window functions** (ROW_NUMBER), **searched CASE** with multiple conditions, **DECODE**, **MOD**, deriving one table from another to keep them consistent.

Things to watch out for

The CASE branch order matters: if `ELSE 'SUCCESS'` logic were written as the first WHEN (e.g. `WHEN r.status <> 'CANCELLED'`), waitlisted tickets would match it before the REFUNDED branch.

Q29 · Generate cancellations with INSERT … SELECT (2.11)

The query

```
-- 2.11 CANCELLATION : refund policy  >= 2 days before journey : 90%,  1 day : 75%,  same day : 50%
INSERT INTO cancellation (cancel_id, pnr, cancelled_on, refund_amount, reason)
SELECT 700000 + ROW_NUMBER() OVER (ORDER BY r.pnr),
       r.pnr,
       r.booking_date + FLOOR((s.journey_date - r.booking_date) / 2),
       ROUND(r.fare * CASE
               WHEN s.journey_date - (r.booking_date + FLOOR((s.journey_date - r.booking_date) / 2)) >=
2 THEN 0.90
               WHEN s.journey_date - (r.booking_date + FLOOR((s.journey_date - r.booking_date) / 2)) =
1  THEN 0.75
               ELSE 0.50 END, 2),
       DECODE(MOD(r.pnr, 4), 0, 'Change of plans', 1, 'Medical emergency',
                             2, 'Booked alternate travel', 'Official work')
FROM   reservation r JOIN train_schedule s ON s.schedule_id = r.schedule_id
WHERE  r.status = 'CANCELLED';
```

Source: parts/03_dml.sql, lines 14–26

What it is for

One line: creates a cancellation record, with a refund calculated by the project's refund policy, for each of the 35 cancelled tickets.

Detailed: The data generator places each cancellation halfway between the booking date and the journey date, then applies the policy: ≥ 2 days before the journey 90%, 1 day 75%, same day 50%.

Simple explanation

This creates a **cancellation record for every cancelled ticket** (35 of them).

For each one it picks a cancellation date halfway between booking and travel, and gives a refund depending on how many days were left: 2 or more days → 90%, 1 day → 75%, same day → 50%. It also picks a reason from four options.

Then `COMMIT` saves all the data permanently.

Concepts used

Date arithmetic in Oracle, FLOOR and ROUND, conditional expressions, deriving data from existing rows, filtering in INSERT … SELECT.

Things to watch out for

If a cancelled ticket were booked on the journey day itself, the midpoint is the same day: 0 days left → 50%.

Q30 · Transaction control in the scripts (COMMIT and ROLLBACK)

The query

```
COMMIT;      -- end of 02_data.sql, after 2.11, end of Part 4, end of Part 6 (03_dml.sql)
COMMIT;      -- after creating the risk tables, and inside prc_capacity_risk_report (04_queries.sql)
ROLLBACK;    -- end of the what-if analysis 10.3 (04_queries.sql)
```

What it is for

One line: COMMIT makes changes permanent; ROLLBACK undoes changes since the last COMMIT.

Detailed: A **transaction** is a group of statements that succeed or fail together. In Oracle a transaction starts automatically with the first DML statement and ends with COMMIT, ROLLBACK, or any DDL (implicit commit).

Simple explanation

A **transaction** is a group of changes that should succeed or fail together.

- `COMMIT` = **save**. Changes become permanent and other users can see them.
- `ROLLBACK` = **undo**. All changes since the last save are thrown away.

In Oracle, a transaction starts by itself with the first change. Commands that change table structure (CREATE, ALTER, DROP) save automatically.

The project saves at the end of each part, and uses ROLLBACK in the what-if analysis (Q87) so the experiment leaves no trace.

Concepts used

ACID: **A**tomicity (all or nothing), **C**onsistency (constraints hold before and after), **I**solation (others don't see uncommitted changes), **D**urability (committed data survives crashes, via the redo log). Also **statement-level rollback** (a failing statement undoes only itself) and **implicit commit on DDL**.

Things to watch out for

`prc_capacity_risk_report` contains a COMMIT. Calling it from another transaction commits **that caller's** pending work too, which can surprise a caller.

Q31 · Row counts with UNION ALL

The query

```sql
-- Row counts of every table
SELECT 'STATION' tbl, COUNT(*) row_count FROM station UNION ALL
SELECT 'ROUTE', COUNT(*) FROM route UNION ALL
SELECT 'ROUTE_STOP', COUNT(*) FROM route_stop UNION ALL
SELECT 'TRAIN', COUNT(*) FROM train UNION ALL
SELECT 'COACH_CLASS', COUNT(*) FROM coach_class UNION ALL
SELECT 'COACH', COUNT(*) FROM coach UNION ALL
SELECT 'SEAT', COUNT(*) FROM seat UNION ALL
SELECT 'TRAIN_SCHEDULE', COUNT(*) FROM train_schedule UNION ALL
SELECT 'PASSENGER', COUNT(*) FROM passenger UNION ALL
SELECT 'RESERVATION', COUNT(*) FROM reservation UNION ALL
SELECT 'PAYMENT', COUNT(*) FROM payment UNION ALL
SELECT 'CANCELLATION', COUNT(*) FROM cancellation;
```

Source: parts/03_dml.sql, lines 30–42

What it is for

One line: shows how many rows each of the 12 core tables contains, in one result.

Detailed: A quick sanity check after loading data, so you can confirm every table was filled.

Simple explanation

This simply **counts the rows in every table** and shows the counts in one list.

Each line counts one table; `UNION ALL` glues the twelve answers together into one result. It is a quick check that all the data was loaded.

Concepts used

Aggregate functions without GROUP BY (one row per SELECT), **set operators** (UNION ALL vs UNION), column aliases.

Things to watch out for

Without ORDER BY, the order of UNION ALL output is the order of the SELECTs in practice, but SQL does not guarantee any order without ORDER BY.

Q32 · ALTER TABLE passenger ADD concession_type (3.1)

The query

```
-- 3.1 add a concession category to passengers (with domain constraint)
ALTER TABLE passenger ADD (
  concession_type VARCHAR2(10) DEFAULT 'NONE' NOT NULL
  CONSTRAINT ck_pass_concession CHECK (concession_type IN ('NONE','SENIOR','STUDENT','DIVYANG'))
);
```

Source: parts/03_dml.sql, lines 47–51

What it is for

One line: adds a new mandatory column with a default and an allowed-values check to an existing, populated table.

Detailed: Demonstrates schema evolution: a new business requirement (concessions for seniors and students) arrives after data exists.

Simple explanation

This **adds a new column** `concession_type` to the PASSENGER table, after the table already has data.

Because the new column is required (NOT NULL), it is given a default value 'NONE'. Oracle fills 'NONE' into every existing passenger. Only four values are allowed: NONE, SENIOR, STUDENT, DIVYANG.

Concepts used

DDL on existing tables, DEFAULT with NOT NULL, implicit commit. (In modern Oracle, adding a column with a DEFAULT is a fast metadata-only operation; existing rows are not physically rewritten.)

Things to watch out for

After this ALTER the table has 9 columns, so the 8-value positional INSERTs of 02_data.sql would no longer work.

Q33 · ALTER TABLE station MODIFY (3.2)

The query

```
-- 3.2 widen a column
ALTER TABLE station MODIFY (station_name VARCHAR2(80));
```

Source: parts/03_dml.sql, lines 52–53

What it is for

One line: increases the maximum length of station_name from 60 to 80.

Detailed: Some real station names are long ("Puratchi Thalaivar Dr. M.G. Ramachandran Central"). Widening a column never invalidates existing data.

Simple explanation

This **makes the station name column longer** (from 60 to 80 characters), so long station names fit. Making a column bigger is always safe.

Concepts used

Changing a column definition; widening is always safe, **narrowing** fails if any existing value is too long (ORA-01441).

Things to watch out for

To add or remove NOT NULL use `MODIFY (col NOT NULL)` / `MODIFY (col NULL)`.

Q34 · ALTER TABLE passenger RENAME COLUMN (3.3)

The query

```sql
-- 3.3 rename a column to a clearer name
ALTER TABLE passenger RENAME COLUMN city TO home_city;
```

Source: parts/03_dml.sql, lines 54–55

What it is for

One line: renames `city` to the clearer `home_city`.

Detailed: STATION also has a `city` column; renaming the passenger column avoids confusion in queries that join both tables.

Simple explanation

This **renames** the passenger column `city` to `home_city`, to avoid confusion with the station table's `city`. The data stays the same; only the name changes. Every later query uses the new name.

Concepts used

Schema evolution, **dependent object invalidation**: views, procedures or queries that still refer to `city` would fail.

Things to watch out for

Renaming a column does not rename constraints or indexes that mention it.

Q35 · ALTER TABLE train ADD CONSTRAINT (3.4)

The query

```
-- 3.4 add a table-level constraint after creation
ALTER TABLE train ADD CONSTRAINT uq_train_name UNIQUE (train_name);
```

Source: parts/03_dml.sql, lines 56–57

What it is for

One line: adds a UNIQUE constraint on train names after the table already contains data.

Detailed: Oracle checks all existing rows when the constraint is added. Because the 9 names are distinct, it succeeds.

Simple explanation

This **adds a rule** to the existing TRAIN table: no two trains may have the same name. Oracle first checks the existing 9 trains; since all names are different, the rule is added.

Concepts used

Adding constraints to populated tables; validation of existing data; constraints create indexes.

Things to watch out for

Q9 groups by `t.train_name`; this constraint guarantees that grouping by name is the same as grouping by train.

Q36 · UPDATE concession from age (4.1)

The query

```
-- 4.1 derive concession category from age
UPDATE passenger SET concession_type = 'SENIOR'  WHERE MONTHS_BETWEEN(SYSDATE, dob) / 12 >= 60;
UPDATE passenger SET concession_type = 'STUDENT' WHERE MONTHS_BETWEEN(SYSDATE, dob) / 12 < 23;
SELECT passenger_id, full_name, TRUNC(MONTHS_BETWEEN(SYSDATE, dob) / 12) age, concession_type
FROM   passenger WHERE concession_type <> 'NONE' ORDER BY passenger_id;
```

Source: parts/03_dml.sql, lines 62–66

What it is for

One line: marks passengers aged 60 or more as SENIOR and under 23 as STUDENT, then lists everyone who has a concession.

Detailed: Demonstrates UPDATE with a computed condition, using today's date to compute age.

Simple explanation

This **gives concessions based on age**:

- Passengers aged 60 or more become SENIOR.
- Passengers younger than 23 become STUDENT.

Age is calculated from today's date and the date of birth (months between them ÷ 12). Then a SELECT lists everyone who got a concession, to check the result. Because it uses today's date, the result can change when the script is run on another day.

Concepts used

UPDATE with WHERE, date functions, SYSDATE-dependent results, verifying an update with a SELECT.

DEPENDS ON THE RUN DATE

Ages are computed from SYSDATE, so a passenger can move between categories on a later run.

Things to watch out for

The two UPDATEs cannot overlap (≥ 60 and < 23). Running them again changes nothing new but still reports the same row counts.

Q37 · UPDATE train timetable (4.2)

The query

```sql
-- 4.2 timetable revision
UPDATE train SET dep_time = '07:40' WHERE train_no = 12639;
```

Source: parts/03_dml.sql, lines 68–69

What it is for

One line: changes the departure time of train 12639 from 07:50 to 07:40.

Detailed: A single-row UPDATE identified by the primary key: the safest kind.

Simple explanation

This **changes the departure time** of train 12639 from 07:50 to 07:40. It changes exactly one row because it uses the train number (the primary key).

Concepts used

Primary-key-based UPDATE, CHECK on update, row locking.

Things to watch out for

Updating a non-existent train number updates 0 rows, which is **not** an error. Check the row count.

Q38 · Find a PNR into a bind variable (4.3)

The query

```
-- 4.3 RESERVATION MODIFICATION : passenger changes boarding point MAS -> KPD on an upcoming journey.
--     Setting fare to NULL makes the trigger recalculate the fare for the shorter distance.
VARIABLE v_pnr NUMBER
BEGIN
  SELECT MIN(pnr) INTO :v_pnr FROM reservation
  WHERE  schedule_id = 103 AND status = 'CONFIRMED' AND from_station = 'MAS' AND to_station = 'SBC';
END;
/
```

Source: parts/03_dml.sql, lines 71–78

What it is for

One line: picks one confirmed MAS→SBC ticket on journey 103 and stores its PNR in a SQL*Plus variable for the next statements.

Detailed: The following SELECTs and UPDATEs refer to `:v_pnr`, so the PNR is chosen once and reused.

Simple explanation

This **finds one ticket to modify**: the first confirmed Chennai → Bengaluru ticket on trip 103, and stores its PNR in a variable `:v_pnr` so the next statements can use it.

`MIN(pnr)` is used so that exactly one PNR comes back, even if several tickets match.

Concepts used

Bind variables, SELECT INTO in an anonymous block, aggregate guaranteeing one row.

Things to watch out for

Re-running this part after the boarding point has been changed finds a **different** PNR, because the first ticket no longer starts at MAS.

Q39 · Before-and-after check of the ticket and payment (4.3)

The query

```
SELECT r.pnr, r.from_station, r.to_station, r.fare, p.amount AS paid
FROM   reservation r JOIN payment p ON p.pnr = r.pnr WHERE r.pnr = :v_pnr;
```

Source: parts/03_dml.sql — the same query appears twice: before and after the UPDATEs

What it is for

One line: shows the ticket's stations, fare and amount paid, before and after the change.

Detailed: Running the same SELECT twice around an UPDATE is a simple way to prove what changed.

Simple explanation

This shows the chosen ticket's **stations, fare and the amount paid**, by joining the ticket with its payment.

It is run once **before** and once **after** the change, so you can see what changed: from MAS→SBC, ₹561 to KPD→SBC, ₹374.

Concepts used

Inner join, column alias, bind variable in WHERE.

Things to watch out for

If :v_pnr is NULL, `WHERE r.pnr = NULL` matches nothing: comparisons with NULL are never true.

Q40 · UPDATE boarding station and recalculate fare (4.3)

The query

```
UPDATE reservation SET from_station = 'KPD', fare = NULL WHERE pnr = :v_pnr;
```

Source: parts/03_dml.sql, Part 4.3

What it is for

One line: changes the ticket's boarding station from Chennai to Katpadi and makes the trigger recompute the fare.

Detailed: Setting `fare = NULL` in the same statement is the trick that tells the trigger "no fare given, please calculate one" (rule R4).

Simple explanation

This **changes the boarding station** of the ticket from Chennai (MAS) to Katpadi (KPD) and at the same time **clears the fare**.

Clearing the fare is the trick: when the trigger sees an empty fare, it recalculates it for the new, shorter distance (230 km instead of 359 km), so the fare becomes ₹374.

Concepts used

UPDATE firing a BEFORE UPDATE trigger (`UPDATE OF from_station, fare`), trigger-maintained derived columns, CHECK on NULL (`fare > 0` is UNKNOWN for NULL, so it would pass; but the trigger replaces NULL first).

Things to watch out for

The payment row is not updated by the trigger; it must be updated separately (Q41).

Q41 · UPDATE payment using a scalar subquery (4.3)

The query

```
UPDATE payment SET amount = (SELECT fare FROM reservation WHERE pnr = :v_pnr) WHERE pnr = :v_pnr;
```

Source: parts/03_dml.sql, Part 4.3

What it is for

One line: copies the new fare into the payment amount.

Detailed: Keeps PAYMENT consistent with RESERVATION (test-suite invariant 10: amount = fare).

Simple explanation

This **updates the payment amount** to the new fare, by reading the fare from the ticket. This keeps the payment and the ticket in agreement.

Concepts used

Subqueries in SET, scalar subquery rules (more than one row → ORA-01427), data consistency across tables.

Things to watch out for

In a real system the 187-rupee difference would be refunded; the project simply updates the amount.

Q42 · INSERT then DELETE a test passenger (4.4)

The query

```
-- 4.4 DELETE : a registered passenger who never booked can be removed
INSERT INTO passenger (passenger_id, full_name, gender, dob, phone, email, home_city)
VALUES (999, 'Test Account', 'M', DATE '1999-05-05', '9000000001', 'test.account@mail.com', 'Chennai');
DELETE FROM passenger WHERE passenger_id = 999;

COMMIT;
```

Source: parts/03_dml.sql, lines 88–93

What it is for

One line: shows that a passenger with no bookings can be deleted, then commits Part 4.

Detailed: Contrast with test 5.6: a passenger **with** bookings cannot be deleted.

Simple explanation

This adds a test passenger (id 999) and then deletes it, to show that a passenger **with no tickets can be deleted**. Then `COMMIT` saves Part 4.

The INSERT names its columns because the table now has 9 columns, and it uses the new name `home_city`.

Concepts used

DELETE with FK protection, DEFAULTs, the difference between DELETE (rows, DML, can roll back), TRUNCATE (all rows, DDL, cannot roll back) and DROP (the table itself).

Things to watch out for

DELETE does not reset sequences or free space immediately; TRUNCATE would, but fails on a table referenced by enabled FKs (ORA-02266).

Q43–Q49 · Integrity tests 5.1–5.7 (expected failures)

Part 5 runs seven statements that **must fail**. Each is wrapped in its own anonymous block with an exception handler, so the error is caught, printed with `SQLERRM`, and the script continues.

The common pattern:

```
BEGIN
  <statement that should be rejected>;
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('label -> ' || SQLERRM);
END;
/
```

- `EXCEPTION WHEN OTHERS` catches any error.
- `SQLERRM` returns the error message text of the current exception.
- `DBMS_OUTPUT.PUT_LINE` writes to the output buffer, which SQL*Plus shows because of `SET SERVEROUTPUT ON` (DBMS Output tab in SQL Developer).
- The failed statement is rolled back automatically; nothing is inserted or deleted.

Q43 · Test 5.1 — invalid phone

The query

```
--  PART 5 : ENFORCEMENT OF INTEGRITY - each statement below is EXPECTED TO BE REJECTED
-- ================================================================================
BEGIN
  INSERT INTO passenger (passenger_id, full_name, gender, dob, phone, email)
  VALUES (998, 'Bad Phone', 'M', DATE '1990-01-01', '12345', 'bad.phone@mail.com');
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('5.1 invalid phone       -> ' || SQLERRM);
END;
/
```

Source: parts/03_dml.sql, lines 96–103

What it is for

One line: proves the phone CHECK rejects '12345'.

Detailed: A domain constraint stops malformed data at the database boundary.

Simple explanation

This test tries to add a passenger with the phone number **'12345'**. That is not a valid 10-digit mobile number, so the database **refuses** it. The error is caught and printed, and the script carries on.

Concepts used

CHECK constraints, regex validation, exception handling.

Things to watch out for

`WHEN OTHERS` without re-raising hides errors in real programs; acceptable here because the purpose is to display them.

Q44 · Test 5.2 — double seat booking

The query

```
BEGIN   -- seat already sold on journey 101
  INSERT INTO reservation (pnr, passenger_id, schedule_id, class_code, seat_id, from_station,
to_station, booking_date, status)
  SELECT 4299999991, 60, 101, class_code, seat_id, 'MAS', 'SBC', DATE '2026-09-10', 'COMPLETED'
  FROM   reservation WHERE schedule_id = 101 AND seat_id = 101;
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('5.2 double seat booking -> ' || SQLERRM);
END;
/
```

Source: parts/03_dml.sql, lines 104–110

What it is for

One line: tries to sell seat 101 of journey 101 a second time; the unique index must refuse.

Detailed: The INSERT … SELECT copies the class and seat of the existing ticket for seat 101 on journey 101 and assigns them to passenger 60.

Simple explanation

This test tries to **sell a seat that is already sold** on trip 101. The unique index from Q12 refuses it with "unique constraint violated". Double booking is impossible.

Concepts used

Function-based unique index, ORA-00001, INSERT … SELECT.

Things to watch out for

Passenger 60 may also already travel on journey 101; then ux_res_pass_journey could fire instead. Here the seat index reported the error.

Q45 · Test 5.3 — seat of another train

The query

```
BEGIN   -- seat 2801 is in coach C1 of train 12677, journey 103 is train 12639
  INSERT INTO reservation (pnr, passenger_id, schedule_id, class_code, seat_id, from_station,
to_station, status)
  VALUES (4299999992, 60, 103, 'CC', 2801, 'MAS', 'SBC', 'CONFIRMED');
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('5.3 seat of other train -> ' || SQLERRM);
END;
/
```

Source: parts/03_dml.sql, lines 111–116

What it is for

One line: tries to give a journey-103 ticket (train 12639) a seat from train 12677; the trigger must refuse.

Detailed: No CHECK or FK can catch this, because seat 2801 exists; only the trigger's rule R2 can.

Simple explanation

This test tries to give a ticket on train 12639 a **seat that belongs to another train** (12677). The seat exists, so the normal link check is happy, but the trigger notices the seat is on the wrong train and refuses it.

Concepts used

Trigger-enforced multi-table rules, RAISE_APPLICATION_ERROR, error stack.

Things to watch out for

ORA-06512 lines are the call stack, useful for debugging.

Q46 · Test 5.4 — wrong direction

The query

```
BEGIN   -- destination before boarding point
  INSERT INTO reservation (pnr, passenger_id, schedule_id, class_code, from_station, to_station, status)
  VALUES (4299999993, 60, 103, 'CC', 'SBC', 'MAS', 'WAITLIST');
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('5.4 wrong direction     -> ' || SQLERRM);
END;
/
```

Source: parts/03_dml.sql, lines 117–122

What it is for

One line: tries to book SBC→MAS on a MAS→SBC train; the trigger must refuse.

Detailed: A WAITLIST status with no seat is used so that only the direction rule is being tested.

Simple explanation

This test tries to book **Bengaluru → Chennai on a Chennai → Bengaluru train**. The trigger sees that the boarding station comes after the destination on the route and refuses it.

Concepts used

Trigger rule R1, using ordering data (stop_seq) for validation.

Things to watch out for

Return journeys need a separate route or train in this model.

Q47 · Test 5.5 — confirmed ticket without a seat

The query

```
BEGIN   -- confirmed ticket without a seat (booking_date <= journey_date ensures CK_RES_SEAT_STATUS is tested directly)
  INSERT INTO reservation (pnr, passenger_id, schedule_id, class_code, from_station, to_station, booking_date, status)
  VALUES (4299999994, 60, 103, 'CC', 'MAS', 'SBC', DATE '2026-10-20', 'CONFIRMED');
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('5.5 confirmed, no seat  -> ' || SQLERRM);
END;
/
```

Source: parts/03_dml.sql, lines 123–128

What it is for

One line: proves that a CONFIRMED ticket must have a seat.

Detailed: The comment explains that booking_date 20-Oct-2026 is set so the trigger's date rule (R3) passes and the CHECK constraint is what fails.

Simple explanation

This test tries to create a **CONFIRMED ticket without a seat**. The rule "confirmed tickets must have a seat" refuses it. A fixed booking date is used so that this is the rule that fails, not the date check.

Concepts used

Cross-column CHECK, order of trigger vs constraint checks, test isolation.

Things to watch out for

Good test design: make sure the error you expect is the one you get.

Q48 · Test 5.6 — delete a parent row

The query

```
BEGIN   -- passenger with bookings cannot be deleted (child rows exist)
  DELETE FROM passenger WHERE passenger_id = 1;
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('5.6 delete parent row   -> ' || SQLERRM);
END;
/
```

Source: parts/03_dml.sql, lines 129–133

What it is for

One line: proves that a passenger with tickets cannot be deleted.

Detailed: Referential integrity protects ticket history.

Simple explanation

This test tries to **delete passenger 1**, who has tickets. The database refuses, because deleting the passenger would leave tickets pointing to nobody (referential integrity).

Concepts used

FK delete rule (restrict), ORA-02292.

Things to watch out for

ON DELETE SET NULL is impossible here because passenger_id is NOT NULL.

Q49 · Test 5.7 — unknown station

The query

```
BEGIN   -- unknown station code
  INSERT INTO route_stop VALUES (1, 9, 'XYZ', 400);
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('5.7 unknown station     -> ' || SQLERRM);
END;
/
```

Source: parts/03_dml.sql, lines 134–138

What it is for

One line: proves that a stop cannot reference a station that does not exist.

Detailed: The positional INSERT tries route 1, stop 9, station 'XYZ'.

Simple explanation

This test tries to add a route stop for a station code **'XYZ' that does not exist**. The database refuses, because a stop must refer to a real station.

Concepts used

Referential integrity on insert, ORA-02291.

Things to watch out for

FK checks happen per statement; a deferred constraint (`DEFERRABLE INITIALLY DEFERRED`) would delay the check to COMMIT, which the project does not use.

Q50 · Procedure book_ticket (6.1)

The query

```
-- 6.1 BOOK_TICKET : allocates the first free seat of the requested class, else WAITLIST
--     Hardened with: row locking (FOR UPDATE) & DUP_VAL_ON_INDEX bounded retry loop for concurrent
safety
CREATE OR REPLACE PROCEDURE book_ticket (
  p_passenger IN  NUMBER,
  p_schedule  IN  NUMBER,
  p_class     IN  VARCHAR2,
  p_from      IN  VARCHAR2,
  p_to        IN  VARCHAR2,
  p_mode      IN  VARCHAR2 DEFAULT 'UPI',
  p_pnr       OUT NUMBER,
  p_status    OUT VARCHAR2
) AS
  v_sched_status train_schedule.sched_status%TYPE;
  v_coaches      NUMBER;
  v_seat         seat.seat_id%TYPE;
  v_fare         reservation.fare%TYPE;
  v_pax_count    NUMBER;
  v_pax_active   NUMBER;
  v_retries      NUMBER := 0;
BEGIN
  -- Validate passenger existence
  SELECT COUNT(*) INTO v_pax_count FROM passenger WHERE passenger_id = p_passenger;
  IF v_pax_count = 0 THEN
    RAISE_APPLICATION_ERROR(-20015, 'Passenger ' || p_passenger || ' does not exist');
  END IF;

  -- Validate and lock schedule to serialize concurrent allocation per journey
  BEGIN
    SELECT sched_status INTO v_sched_status
    FROM   train_schedule
    WHERE  schedule_id = p_schedule
    FOR UPDATE;
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      RAISE_APPLICATION_ERROR(-20014, 'Journey schedule ' || p_schedule || ' not found');
  END;

  IF v_sched_status <> 'SCHEDULED' THEN
    RAISE_APPLICATION_ERROR(-20010, 'Booking closed for journey ' || p_schedule);
  END IF;

  -- Validate passenger does not already hold an active ticket on this journey (BR8)
  SELECT COUNT(*) INTO v_pax_active
  FROM   reservation
  WHERE  schedule_id = p_schedule
  AND    passenger_id = p_passenger
  AND    status <> 'CANCELLED';

  IF v_pax_active > 0 THEN
    RAISE_APPLICATION_ERROR(-20017, 'Passenger ' || p_passenger || ' already holds an active ticket for
journey ' || p_schedule);
  END IF;

  SELECT COUNT(*) INTO v_coaches
  FROM   coach c JOIN train_schedule s ON s.train_no = c.train_no
  WHERE  s.schedule_id = p_schedule AND c.class_code = p_class;
  IF v_coaches = 0 THEN
    RAISE_APPLICATION_ERROR(-20011, 'Class ' || p_class || ' is not available on this train');
  END IF;
```

```
  -- Concurrency-safe seat allocation loop: bounded retry if seat race condition is encountered
  LOOP
    v_retries := v_retries + 1;
    IF v_retries > 50 THEN
      RAISE_APPLICATION_ERROR(-20018, 'Seat allocation exceeded max retries due to concurrent demand');
    END IF;

    v_seat := NULL;
    BEGIN
      SELECT st.seat_id INTO v_seat
      FROM   seat st
             JOIN coach c          ON c.coach_id = st.coach_id
             JOIN train_schedule s ON s.train_no = c.train_no
      WHERE  s.schedule_id = p_schedule
      AND    c.class_code  = p_class
      AND    NOT EXISTS (SELECT 1 FROM reservation r
                         WHERE r.schedule_id = p_schedule AND r.seat_id = st.seat_id)
      ORDER  BY st.seat_id
      FETCH FIRST 1 ROW ONLY;
    EXCEPTION
      WHEN NO_DATA_FOUND THEN
        NULL;  -- no free seat: v_seat stays NULL -> WAITLIST
    END;

    p_status := CASE WHEN v_seat IS NULL THEN 'WAITLIST' ELSE 'CONFIRMED' END;
    p_pnr    := seq_pnr.NEXTVAL;

    BEGIN
      INSERT INTO reservation (pnr, passenger_id, schedule_id, class_code, seat_id,
                               from_station, to_station, booking_date, status)
      VALUES (p_pnr, p_passenger, p_schedule, p_class, v_seat, p_from, p_to, TRUNC(SYSDATE), p_status)
      RETURNING fare INTO v_fare;

      EXIT; -- Insert succeeded
    EXCEPTION
      WHEN DUP_VAL_ON_INDEX THEN
        IF v_seat IS NOT NULL THEN
          -- Candidate seat was taken concurrently: retry for next seat
          NULL;
        ELSE
          RAISE;
        END IF;
    END;
  END LOOP;

  INSERT INTO payment VALUES (seq_payment.NEXTVAL, p_pnr, v_fare, p_mode, TRUNC(SYSDATE), 'SUCCESS');
END;
/
```

Source: parts/03_dml.sql, lines 143–249

What it is for

One line: books a ticket: it validates the request, gives the passenger the first free seat of the requested class, or puts them on the waitlist if the class is full, and records the payment.

Detailed: This is the project's "booking counter". A **stored procedure** is a named PL/SQL program stored in the database and called by name. Putting the booking logic in the database means every caller (a web app, a test script, SQL Developer) follows exactly the same rules. The procedure is hardened for **concurrency**: two people booking the last seat at the same moment cannot both get it.

Simple explanation

`book_ticket` is the **booking counter**. You give it a passenger, a trip, a class, two stations and a payment mode; it gives back a PNR and a status (CONFIRMED or WAITLIST).

What it does, step by step:

1. Checks that the passenger exists.
2. **Locks the trip** so that two people booking the same trip at the same time are served one after the other, never simultaneously.
3. Checks the trip is still open for booking (SCHEDULED).
4. Checks the passenger doesn't already have a ticket on this trip.
5. Checks the train has the requested class.
6. Looks for the **first free seat** of that class on this trip. If there is one, the ticket is CONFIRMED with that seat; if not, the passenger goes on the **WAITLIST**.
7. Saves the ticket (the trigger calculates the fare) and then saves the payment.

Safety net: if, in a rare case, someone else grabbed the seat a split second earlier, the database rejects the duplicate and the procedure simply tries the next free seat (up to 50 times).

It does **not** save (commit) by itself; whoever calls it decides when to save.

Concepts used

Stored procedures, **IN/OUT parameters and defaults**, **%TYPE anchoring**, **SELECT INTO with NO_DATA_FOUND**, **nested blocks and exception handlers**, **user-defined errors**, **sequences**, **RETURNING INTO**, **correlated NOT EXISTS**, **row-limiting (FETCH FIRST)**, **pessimistic locking (SELECT … FOR UPDATE)**, **race conditions**, **unique index as the final guard**, **transaction boundaries owned by the caller**.

Things to watch out for

- `NOT EXISTS` must compare `r.schedule_id = p_schedule`; without it, a seat sold on **any** journey would look taken.
- A PNR taken by NEXTVAL in an iteration that hits DUP_VAL_ON_INDEX is lost: sequences leave gaps.
- Waitlisted passengers are charged the full fare (payment 'SUCCESS'); the refund happens when they cancel (100%) or when the journey runs without them (the data marks those REFUNDED).
- `p_from` / `p_to` are validated by the trigger, not the procedure; a wrong direction surfaces as ORA-20002 from inside book_ticket.
- Calling with a journey in the past: status may still be SCHEDULED (journeys 103–127 are past on 28-Oct), but the trigger's R3 raises -20004.

Q51 · Procedure cancel_ticket (6.2)

The query

```
-- 6.2 CANCEL_TICKET : refund by time-to-departure, release seat, promote first waitlisted ticket
--     Refund : waitlist 100% | >=48 h 90% | >=24 h 75% | >=4 h 50% | <4 h nil
--     Hardened with: PNR existence check, serialized waitlist row-level lock (FOR UPDATE)
CREATE OR REPLACE PROCEDURE cancel_ticket (p_pnr IN NUMBER, p_reason IN VARCHAR2) AS
  v_status   reservation.status%TYPE;
  v_fare     reservation.fare%TYPE;
  v_seat     reservation.seat_id%TYPE;
  v_sched    reservation.schedule_id%TYPE;
  v_class    reservation.class_code%TYPE;
  v_depart   DATE;
  v_hours    NUMBER;
  v_refund   NUMBER(8,2);
  v_promoted reservation.pnr%TYPE;
BEGIN
  BEGIN
    SELECT r.status, r.fare, r.seat_id, r.schedule_id, r.class_code,
           s.journey_date + TO_NUMBER(SUBSTR(t.dep_time, 1, 2)) / 24 + TO_NUMBER(SUBSTR(t.dep_time, 4,
2)) / 1440
    INTO   v_status, v_fare, v_seat, v_sched, v_class, v_depart
    FROM   reservation r
           JOIN train_schedule s ON s.schedule_id = r.schedule_id
           JOIN train t          ON t.train_no    = s.train_no
    WHERE  r.pnr = p_pnr
    FOR UPDATE OF r.status;
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      RAISE_APPLICATION_ERROR(-20016, 'PNR ' || p_pnr || ' does not exist');
  END;

  IF v_status NOT IN ('CONFIRMED','WAITLIST') THEN
    RAISE_APPLICATION_ERROR(-20012, 'PNR ' || p_pnr || ' is ' || v_status || ' and cannot be cancelled')
;
  END IF;
  v_hours := (v_depart - SYSDATE) * 24;
  IF v_hours < 0 THEN
    RAISE_APPLICATION_ERROR(-20013, 'Train has already departed');
  END IF;

  v_refund := ROUND(v_fare * CASE WHEN v_status = 'WAITLIST' THEN 1
                                  WHEN v_hours >= 48 THEN 0.90
                                  WHEN v_hours >= 24 THEN 0.75
                                  WHEN v_hours >= 4  THEN 0.50
                                  ELSE 0 END, 2);

  UPDATE reservation SET status = 'CANCELLED', seat_id = NULL WHERE pnr = p_pnr;
  INSERT INTO cancellation VALUES (seq_cancel.NEXTVAL, p_pnr, TRUNC(SYSDATE), v_refund, p_reason);
  UPDATE payment SET pay_status = CASE WHEN v_refund >= amount THEN 'REFUNDED'
                                       WHEN v_refund > 0 THEN 'PARTIAL_REFUND'
                                       ELSE 'SUCCESS' END
  WHERE  pnr = p_pnr;
  DBMS_OUTPUT.PUT_LINE('PNR ' || p_pnr || ' cancelled, refund Rs.' || v_refund);

  -- Promote earliest waitlisted passenger in FIFO order (loop guards against concurrent promotion)
  IF v_seat IS NOT NULL THEN
    LOOP
      BEGIN
        SELECT pnr INTO v_promoted
        FROM   reservation
        WHERE  schedule_id = v_sched
        AND    class_code  = v_class
        AND    status      = 'WAITLIST'
```

```
        ORDER  BY booking_date, pnr
        FETCH FIRST 1 ROW ONLY;

        UPDATE reservation
        SET    status = 'CONFIRMED', seat_id = v_seat
        WHERE  pnr = v_promoted
        AND    status = 'WAITLIST';

        IF SQL%ROWCOUNT > 0 THEN
          DBMS_OUTPUT.PUT_LINE('Waitlisted PNR ' || v_promoted || ' confirmed on released seat ' || v_se
at);
          EXIT;
        END IF;
      EXCEPTION
        WHEN NO_DATA_FOUND THEN
          EXIT;
      END;
    END LOOP;
  END IF;
END;
/
```

Source: parts/03_dml.sql, lines 251–329

What it is for

One line: cancels a ticket, refunds according to how long before departure it is cancelled, frees the seat, and gives that seat to the first passenger on the waitlist.

Detailed: It implements the refund policy (waitlist 100%; ≥ 48 h 90%; ≥ 24 h 75%; ≥ 4 h 50%; < 4 h nothing) and **automatic waitlist promotion** in first-come-first-served (FIFO) order.

Simple explanation

`cancel_ticket` **cancels a ticket, gives a refund and hands the freed seat to the next person on the waitlist**.

Step by step:

1. Finds the ticket and locks it (so it can't be cancelled twice at once).
2. Refuses if the ticket is already cancelled or completed, or if the train has already left.
3. Works out the exact departure time and how many hours are left, then the refund: waitlisted ticket 100%; 48+ hours 90%; 24+ hours 75%; 4+ hours 50%; less than 4 hours nothing.
4. Marks the ticket CANCELLED and removes its seat, records the cancellation, and updates the payment status.
5. If a seat was freed, finds the **earliest waitlisted passenger** for the same trip and class and gives them that seat (first come, first served).

Concepts used

String and date functions, date arithmetic, row locking in joins (`FOR UPDATE OF`), searched CASE, multi-table transactional updates, implicit cursor attributes (SQL%ROWCOUNT), optimistic re-check in UPDATE, FIFO queues in SQL, DBMS_OUTPUT.

Things to watch out for

- Cancelling a WAITLIST ticket: 100% refund → `v_refund >= amount` → REFUNDED; no seat freed → no promotion. Test suite 04 checks this.
- Cancelling less than 4 hours before departure: refund 0 → pay_status stays SUCCESS, cancellation row records 0.

- The promoted passenger's payment remains SUCCESS (they already paid).
- `TO_NUMBER(SUBSTR(...))` relies on the regex CHECK on dep_time; invalid times would raise ORA-01722.
- The live refund policy (hours) differs from the policy used to generate the sample cancellations (days) in Q29.

Q52 · Demo reset and new journey 130 (6.3)

The query

```
-- 6.3 DEMO : a new journey of 12677 seven days from today; 9 passengers book the 8-seat EC coach
-- Safe idempotency reset for demo journey 130
DELETE FROM cancellation WHERE pnr IN (SELECT pnr FROM reservation WHERE schedule_id = 130);
DELETE FROM payment WHERE pnr IN (SELECT pnr FROM reservation WHERE schedule_id = 130);
DELETE FROM reservation WHERE schedule_id = 130;
DELETE FROM train_schedule WHERE schedule_id = 130;
INSERT INTO train_schedule VALUES (130, 12677, TRUNC(SYSDATE) + 7, 'SCHEDULED');
```

Source: parts/03_dml.sql, lines 331–337

What it is for

One line: removes any leftover data for demo journey 130 and creates it again, 7 days from today.

Detailed: Makes the demo repeatable (idempotent), independent of when the script is run.

Simple explanation

This **prepares a fresh demo trip**. It first deletes anything left over from an earlier demo on trip 130 (cancellations, payments, tickets, then the trip itself, children before parents), then creates trip 130 for train 12677 **seven days from today**. So the demo works the same whichever day you run it.

Concepts used

DELETE with IN-subquery, child-before-parent deletion order, relative dates, idempotent setup.

Things to watch out for

`IN (subquery)` with an empty subquery deletes nothing; no error.

Q53 · Demo: nine bookings in a loop (6.3)

The query

```
DECLARE
  v_pnr    NUMBER;
  v_status VARCHAR2(10);
BEGIN
  FOR p IN 11 .. 19 LOOP
    book_ticket(p, 130, 'EC', 'SBC', 'ERS', 'UPI', v_pnr, v_status);
    DBMS_OUTPUT.PUT_LINE('Passenger ' || p || ' -> PNR ' || v_pnr || ' ' || v_status);
  END LOOP;
END;
/
```

Source: parts/03_dml.sql, lines 339–348

What it is for

One line: books passengers 11–19 into the 8-seat EC coach of journey 130; the ninth is waitlisted.

Detailed: Shows book_ticket confirming until the class is full and then waitlisting.

Simple explanation

This **books nine passengers** (11 to 19) into the 8-seat Executive coach of trip 130, one after the other, using `book_ticket`.

The first eight get seats 3001 to 3008. The ninth finds no free seat and goes on the **waitlist**. Each result is printed.

Concepts used

Calling procedures, OUT parameters, numeric FOR loops, all nine bookings in one transaction.

Things to watch out for

If any call raised an unhandled error, the whole block would stop and its earlier bookings would remain uncommitted until the next COMMIT or ROLLBACK.

Q54 · Demo: cancel and auto-promote (6.4)

The query

```
-- 6.4 DEMO : passenger 11 cancels; the waitlisted passenger is promoted automatically
DECLARE
  v_pnr NUMBER;
BEGIN
  SELECT pnr INTO v_pnr FROM reservation WHERE schedule_id = 130 AND passenger_id = 11;
  cancel_ticket(v_pnr, 'Change of plans');
END;
/
```

Source: parts/03_dml.sql, lines 349–356

What it is for

One line: finds passenger 11's PNR on journey 130 and cancels it, which promotes the waitlisted passenger.

Detailed: Demonstrates cancel_ticket's refund and FIFO promotion end-to-end.

Simple explanation

This finds passenger 11's ticket on trip 130 and **cancels it**.

Because the trip is a week away, the refund is 90% (₹1588.50 of ₹1765). The freed seat 3001 is automatically given to the waitlisted passenger 19, whose ticket becomes CONFIRMED.

Concepts used

SELECT INTO, procedure call, cascading business effects within one transaction.

Things to watch out for

If passenger 11 had two tickets on the journey (one cancelled, one active), SELECT INTO would raise TOO_MANY_ROWS; adding `AND status <> 'CANCELLED'` would make it robust.

Q55 · Demo result: six-table join (6.4)

The query

```
SELECT r.pnr, p.full_name, c.coach_label || '-' || st.seat_no seat, r.status, r.fare, pm.pay_status,
cn.refund_amount
FROM   reservation r
       JOIN passenger p     ON p.passenger_id = r.passenger_id
       JOIN payment pm      ON pm.pnr = r.pnr
       LEFT JOIN seat st    ON st.seat_id = r.seat_id
       LEFT JOIN coach c    ON c.coach_id = st.coach_id
       LEFT JOIN cancellation cn ON cn.pnr = r.pnr
WHERE  r.schedule_id = 130
ORDER  BY r.pnr;

COMMIT;
```

Source: parts/03_dml.sql, lines 357–367

What it is for

One line: shows the final state of every ticket on journey 130: passenger, seat, status, fare, payment status and refund.

Detailed: Proves the demo: one cancellation with a refund, eight confirmed passengers, including the promoted one on seat E1-1.

Simple explanation

This **shows all tickets of trip 130** after the demo: passenger name, coach-seat, status, fare, payment status and refund.

It uses LEFT JOINs for seat, coach and cancellation so that the cancelled ticket, which has no seat, still appears in the list. Then COMMIT saves the demo.

Concepts used

Inner vs left outer joins, chaining outer joins, NULL in concatenation (Oracle-specific behaviour), commit of the demo.

Things to watch out for

In standard SQL `'E1' || NULL` is NULL; Oracle treats NULL as '' in concatenation. Portable code would use `COALESCE`.

6. File 04_queries.sql — Views, Analysis Queries and the Capacity-Risk Module

What this file does: it turns raw tickets into management information. Three **views** compute standard measures once; ten analysis queries (Q1–Q10) and nine advanced queries (A1–A9) answer business questions; and the capacity-risk module (Part 10) classifies trains and recommends action.

MEASURE DEFINITIONS USED EVERYWHERE (FROM THE FILE'S OWN COMMENTS)

capacity = seats in all coaches of a train (per journey). **seats occupied** = tickets with status CONFIRMED or COMPLETED. **occupancy %** = seats occupied ÷ capacity × 100. **cancellation %** = cancelled tickets ÷ all tickets × 100. **demand %** = (seats occupied + waitlisted) ÷ capacity × 100, which can exceed 100%. **revenue** = fares of travelled tickets + cancellation charges retained.

LOGICAL ORDER OF A SELECT — READ THIS ONCE

SQL is written SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY, but it is **logically processed** as: **FROM/JOIN** (build the combined rows) → **WHERE** (filter rows) → **GROUP BY** (form groups) → **HAVING** (filter groups) → **SELECT** (compute output columns, including window functions) → **DISTINCT** → **ORDER BY** → **FETCH/ROWNUM limit**. This explains rules such as "you cannot use a column alias in WHERE" (aliases are created later, in SELECT) and "aggregates are allowed in HAVING but not in WHERE". The **physical** order chosen by Oracle's optimizer (which index, which join method) can be completely different; it only has to give the same result.

Q56 · VIEW v_reservation_performance

The query

```
-- VIEW 1 : RESERVATION PERFORMANCE - one row per journey (train + date)
CREATE OR REPLACE VIEW v_reservation_performance AS
SELECT s.schedule_id,
       t.train_no,
       t.train_name,
       t.route_id,
       s.journey_date,
       s.sched_status,
       cap.capacity,
       COUNT(r.pnr)                                                    AS total_bookings,
       COUNT(CASE WHEN r.status IN ('CONFIRMED','COMPLETED') THEN 1 END) AS seats_occupied,
       COUNT(CASE WHEN r.status = 'WAITLIST' THEN 1 END)               AS waitlisted,
       COUNT(CASE WHEN r.status = 'CANCELLED' THEN 1 END)              AS cancelled,
       ROUND(100 * COUNT(CASE WHEN r.status IN ('CONFIRMED','COMPLETED') THEN 1 END) / cap.capacity, 2)
                                                                       AS occupancy_pct,
       ROUND(100 * COUNT(CASE WHEN r.status = 'CANCELLED' THEN 1 END) / NULLIF(COUNT(r.pnr), 0), 2)
                                                                       AS cancellation_rate,
       NVL(SUM(CASE WHEN r.status IN ('CONFIRMED','COMPLETED') THEN r.fare
                    WHEN r.status = 'CANCELLED' THEN r.fare - cn.refund_amount END), 0)
                                                                       AS revenue
FROM   train_schedule s
       JOIN train t ON t.train_no = s.train_no
       JOIN (SELECT c.train_no, SUM(cc.seats_per_coach) AS capacity
             FROM   coach c JOIN coach_class cc ON cc.class_code = c.class_code
             GROUP  BY c.train_no) cap ON cap.train_no = t.train_no
       LEFT JOIN reservation  r  ON r.schedule_id = s.schedule_id
       LEFT JOIN cancellation cn ON cn.pnr = r.pnr
GROUP  BY s.schedule_id, t.train_no, t.train_name, t.route_id, s.journey_date, s.sched_status, cap.capac
ity;
```

Source: parts/04_queries.sql, lines 13–40

What it is for

One line: produces one row per journey with its capacity, bookings by status, occupancy %, cancellation rate and revenue.

Detailed: A **view** is a stored SELECT that behaves like a virtual table. This one is the foundation of the analysis layer: `v_train_utilization`, Q10, A6 and the risk module read it instead of repeating this 30-line calculation.

Simple explanation

A **view** is a saved query that you can use like a table. It stores no data; it runs its query every time you use it.

This view gives **one line per train trip** with: - the train's capacity (total seats), - number of bookings, seats occupied, waitlisted and cancelled, - occupancy % (occupied ÷ capacity), - cancellation rate % (cancelled ÷ bookings), - revenue (fares of people who travelled + the money kept from cancellations).

It is built once so that many other reports can simply read it instead of repeating this long calculation. Trips with no bookings still appear (LEFT JOIN), with 0.

Concepts used

Views (logical data independence and reuse), **inline views**, **inner vs left outer joins**, **GROUP BY with aggregates**, **conditional aggregation**, **COUNT(column) vs COUNT(*)**, **NULLIF/NVL**, **avoiding fan-out** (pre-aggregating capacity), **derived measures**.

Things to watch out for

- A train with **no coaches** has no row in `cap`, so the inner join removes its journeys from the view entirely (no division by zero, but the journey is invisible).
- `r.fare - cn.refund_amount`: if a CANCELLED ticket had no cancellation row, this would be NULL and SUM would skip it. Test suite 02 invariant 11 guarantees every cancelled ticket has one.
- A view stores no data. Each query re-runs the SELECT, so results are always current.

Q57 · VIEW v_train_utilization

The query

```sql
-- VIEW 2 : TRAIN UTILIZATION - one row per train across all its journeys
CREATE OR REPLACE VIEW v_train_utilization AS
SELECT v.train_no,
       v.train_name,
       v.route_id,
       so.city || '-' || sd.city                                          AS route_label,
       COUNT(*)                                                           AS journeys,
       SUM(v.capacity)                                                    AS capacity_offered,
       SUM(v.seats_occupied)                                              AS seats_occupied,
       SUM(v.waitlisted)                                                  AS waitlisted,
       SUM(v.cancelled)                                                   AS cancelled,
       SUM(v.total_bookings)                                              AS total_bookings,
       ROUND(100 * SUM(v.seats_occupied) / SUM(v.capacity), 2)            AS occupancy_pct,
       ROUND(100 * SUM(v.cancelled) / NULLIF(SUM(v.total_bookings), 0), 2) AS cancellation_rate,
       ROUND(100 * (SUM(v.seats_occupied) + SUM(v.waitlisted)) / SUM(v.capacity), 2) AS demand_pct,
       SUM(v.revenue)                                                     AS revenue
FROM   v_reservation_performance v
       JOIN route   ro ON ro.route_id = v.route_id
       JOIN station so ON so.station_code = ro.origin_code
       JOIN station sd ON sd.station_code = ro.dest_code
GROUP  BY v.train_no, v.train_name, v.route_id, so.city, sd.city;
```

Source: parts/04_queries.sql, lines 42–62

What it is for

One line: rolls the per-journey view up to one row per train, with totals, percentages and a readable route label.

Detailed: A **view built on a view**. Percentages are recalculated from summed counts, not averaged, so every journey is weighted by its size.

Simple explanation

This view takes the per-trip view (Q56) and **adds up all trips of each train**, giving **one line per train**: total trips, seats offered, seats occupied, waitlisted, cancelled, occupancy %, cancellation rate %, demand % and revenue.

Demand % = (occupied + waitlisted) ÷ capacity. It can go above 100%, which means more people wanted to travel than there were seats.

It also builds a route label like *Chennai-Bengaluru* by looking up the start and end station cities.

Concepts used

Nested views, self-referencing a parent table twice (role aliases), ratio of sums vs average of ratios.

Things to watch out for

Because this view reads `v_reservation_performance`, any defect there flows here. Dropping the base view makes this view INVALID.

Q58 · VIEW v_ticket_details

The query

```sql
-- VIEW 3 : TICKET DETAILS - de-normalised ticket for enquiry screens
CREATE OR REPLACE VIEW v_ticket_details AS
SELECT r.pnr, p.passenger_id, p.full_name, t.train_no, t.train_name, s.journey_date,
       r.from_station, r.to_station, r.class_code,
       c.coach_label, st.seat_no, st.berth_type,
       (rs2.km_from_origin - rs1.km_from_origin) AS distance_km,
       r.fare, r.status
FROM   reservation r
       JOIN passenger p       ON p.passenger_id = r.passenger_id
       JOIN train_schedule s  ON s.schedule_id = r.schedule_id
       JOIN train t           ON t.train_no = s.train_no
       JOIN route_stop rs1    ON rs1.route_id = t.route_id AND rs1.station_code = r.from_station
       JOIN route_stop rs2    ON rs2.route_id = t.route_id AND rs2.station_code = r.to_station
       LEFT JOIN seat st      ON st.seat_id = r.seat_id
       LEFT JOIN coach c      ON c.coach_id = st.coach_id;
```

Source: parts/04_queries.sql, lines 64–78

What it is for

One line: a de-normalised, readable ticket with passenger, train, date, stations, coach/seat and the distance travelled.

Detailed: "Enquiry screen" data: everything a ticket-checker would want, in one row. Q8 uses it to sum kilometres per passenger.

Simple explanation

This view shows **each ticket in a readable form**: passenger name, train, date, stations, class, coach, seat, berth type, distance travelled, fare and status, all in one line.

The distance is calculated from the stop list: destination km − boarding km. Tickets without a seat still appear.

Concepts used

De-normalised views for convenience, composite join conditions, chaining outer joins, derived columns.

Things to watch out for

The route_id condition in the ROUTE_STOP joins is essential; without it the join fans out.

Q59 · Querying the two summary views

The query

```
SELECT * FROM v_reservation_performance ORDER BY train_no, journey_date;
SELECT * FROM v_train_utilization ORDER BY occupancy_pct DESC;
```

Source: parts/04_queries.sql, lines 85–86

What it is for

One line: displays the journey view sorted by train and date, and the train view sorted from highest to lowest occupancy.

Detailed: Shows that a view is queried exactly like a table.

Simple explanation

These two lines simply **display the two summary views**: the trip view sorted by train and date, and the train view sorted from fullest to emptiest train.

Concepts used

Querying views, ORDER BY with multiple keys, ASC (default) / DESC.

Things to watch out for

With ties in occupancy (12639 and 12027 both 96.88%) their relative order is not defined unless a tie-breaker column is added.

Q60 · Q1 Train-wise reservation statistics

The query

```sql
-- Q1 TRAIN-WISE RESERVATION STATISTICS  (conditional aggregation / pivot)
SELECT t.train_no, t.train_name,
       COUNT(*)                                                  AS bookings,
       COUNT(CASE WHEN r.status = 'COMPLETED' THEN 1 END)        AS completed,
       COUNT(CASE WHEN r.status = 'CONFIRMED' THEN 1 END)        AS confirmed,
       COUNT(CASE WHEN r.status = 'WAITLIST' THEN 1 END)         AS waitlist,
       COUNT(CASE WHEN r.status = 'CANCELLED' THEN 1 END)        AS cancelled,
       ROUND(AVG(r.fare), 2)                                     AS avg_fare,
       MIN(r.booking_date)                                       AS first_booking,
       MAX(r.booking_date)                                       AS last_booking
FROM   reservation r
       JOIN train_schedule s ON s.schedule_id = r.schedule_id
       JOIN train t          ON t.train_no = s.train_no
GROUP  BY t.train_no, t.train_name
ORDER  BY bookings DESC;
```

Source: parts/04_queries.sql, lines 92–106

What it is for

One line: for each train: total bookings, a count per status, average fare and the first and last booking dates.

Detailed: A **pivot** of status counts per train using conditional aggregation.

Simple explanation

Report Q1 – per train: how many bookings it has, how many are completed, confirmed, waitlisted and cancelled, the average fare, and the first and last booking dates.

The status counts are done in one go: for each status, count only the tickets with that status. The busiest train is shown first.

Concepts used

GROUP BY, COUNT/AVG/MIN/MAX, conditional aggregation (pivot), alias in ORDER BY.

Things to watch out for

AVG ignores NULL fares (none exist here). 12677's last booking 28-OCT-2026 is the demo booking date.

Q61 · Q2 Station-wise passenger volume

The query

```
-- Q2 STATION-WISE PASSENGER VOLUME  (boarding + alighting, stations with no traffic included)
SELECT st.station_code, st.station_name, st.city,
       NVL(b.boarding, 0)                        AS boarded,
       NVL(a.alighting, 0)                       AS alighted,
       NVL(b.boarding, 0) + NVL(a.alighting, 0)  AS total_footfall
FROM   station st
       LEFT JOIN (SELECT from_station AS code, COUNT(*) AS boarding FROM reservation
                  WHERE status IN ('CONFIRMED','COMPLETED') GROUP BY from_station) b ON b.code = st.stat
ion_code
       LEFT JOIN (SELECT to_station AS code, COUNT(*) AS alighting FROM reservation
                  WHERE status IN ('CONFIRMED','COMPLETED') GROUP BY to_station) a   ON a.code = st.stat
ion_code
ORDER  BY total_footfall DESC;
```

Source: parts/04_queries.sql, lines 108–118

What it is for

One line: for every station, the number of passengers boarding, alighting and the total.

Detailed: Uses two pre-aggregated subqueries and LEFT JOINs so that stations with no traffic still appear with zeros.

Simple explanation

Report Q2 – per station: how many passengers got on (boarded), got off (alighted), and the total.

It first counts boardings per station and alightings per station separately, then attaches both counts to the full station list. Stations with no traffic still appear, with 0 (that is what NVL does: turns "nothing" into 0).

Concepts used

Inline views, outer joins, NVL for NULL handling, pre-aggregation before joining.

Things to watch out for

NULL + number = NULL in SQL; always NVL before adding outer-joined values.

Q62 · Q3 Route utilization

The query

```sql
-- Q3 ROUTE UTILIZATION
SELECT ro.route_name, ro.distance_km,
       COUNT(*)                                          AS trains,
       SUM(u.journeys)                                   AS journeys,
       SUM(u.capacity_offered)                           AS seats_offered,
       SUM(u.seats_occupied)                             AS seats_sold,
       ROUND(100 * SUM(u.seats_occupied) / SUM(u.capacity_offered), 2) AS route_occupancy_pct,
       SUM(u.revenue)                                    AS revenue
FROM   v_train_utilization u JOIN route ro ON ro.route_id = u.route_id
GROUP  BY ro.route_name, ro.distance_km
ORDER  BY route_occupancy_pct DESC;
```

Source: parts/04_queries.sql, lines 120–130

What it is for

One line: per route: number of trains, journeys, seats offered and sold, occupancy and revenue.

Detailed: Aggregates the train view one level further, up to routes.

Simple explanation

Report Q3 – per route: how many trains, trips, seats offered and seats sold, the occupancy % and the revenue. It adds up the train view (Q57) by route.

Concepts used

Multi-level aggregation through views, GROUP BY on functionally dependent columns.

Things to watch out for

This **revenue** follows the view's definition (travelled fares + cancellation charges). Q9 computes revenue from payments and gives slightly larger numbers for routes 1 and 2 (₹1,015 and ₹400 more), because waitlisted tickets on upcoming journeys have been paid but not yet refunded. Both are correct for their definitions; be ready to explain the difference.

Q63 · Q4 Coach occupancy

The query

```sql
-- Q4 COACH OCCUPANCY  (per coach across all journeys of its train)
SELECT t.train_no, t.train_name, c.coach_label, c.class_code,
       cc.seats_per_coach * j.journeys                      AS seat_journeys,
       COUNT(r.pnr)                                         AS occupied,
       ROUND(100 * COUNT(r.pnr) / (cc.seats_per_coach * j.journeys), 1) AS occupancy_pct
FROM   coach c
       JOIN coach_class cc ON cc.class_code = c.class_code
       JOIN train t        ON t.train_no = c.train_no
       JOIN (SELECT train_no, COUNT(*) AS journeys FROM train_schedule GROUP BY train_no) j
                           ON j.train_no = c.train_no
       LEFT JOIN seat st   ON st.coach_id = c.coach_id
       LEFT JOIN reservation r ON r.seat_id = st.seat_id AND r.status IN ('CONFIRMED','COMPLETED')
GROUP  BY t.train_no, t.train_name, c.coach_label, c.class_code, cc.seats_per_coach, j.journeys
ORDER  BY t.train_no, c.coach_label;
```

Source: parts/04_queries.sql, lines 132–145

What it is for

One line: occupancy of each coach across all journeys of its train.

Detailed: Shows which coaches (classes) fill and which stay empty, useful for deciding coach composition.

Simple explanation

Report Q4 – per coach: how full each coach was over all trips of its train.

Occupancy = seats sold ÷ (seats in the coach × number of trips). Coaches that sold nothing still appear, because the "sold tickets only" condition is placed inside the join, not in WHERE.

Concepts used

Filter placement in outer joins (ON vs WHERE), seat-journey capacity, grouping by coach.

Things to watch out for

Occupancy here counts tickets per seat across **all** journeys of the train, including demo journey 130 for 12677 (4 journeys).

Q64 · Q5a Cancellation statistics by train

The query

```sql
-- Q5a CANCELLATION STATISTICS BY TRAIN
SELECT t.train_no, t.train_name,
       COUNT(r.pnr)                                          AS bookings,
       COUNT(cn.cancel_id)                                   AS cancellations,
       ROUND(100 * COUNT(cn.cancel_id) / COUNT(r.pnr), 2)    AS cancel_rate_pct,
       NVL(SUM(cn.refund_amount), 0)                         AS refunded,
       NVL(SUM(CASE WHEN cn.cancel_id IS NOT NULL THEN r.fare - cn.refund_amount END), 0) AS charges_ret
ained,
       ROUND(AVG(s.journey_date - cn.cancelled_on), 1)       AS avg_days_before_journey
FROM   reservation r
       JOIN train_schedule s     ON s.schedule_id = r.schedule_id
       JOIN train t              ON t.train_no = s.train_no
       LEFT JOIN cancellation cn ON cn.pnr = r.pnr
GROUP  BY t.train_no, t.train_name
ORDER  BY cancel_rate_pct DESC;
```

Source: parts/04_queries.sql, lines 147–160

What it is for

One line: per train: bookings, cancellations, cancellation rate, money refunded, charges retained and how many days before the journey people cancel on average.

Detailed: Shows which trains lose bookings and how much revenue cancellation charges recover.

Simple explanation

Report Q5a – cancellations per train: bookings, cancellations, cancellation rate %, total refunded, money kept (fare − refund), and on average how many days before travel people cancelled.

Concepts used

COUNT(column) vs COUNT(*), aggregate NULL handling, date arithmetic inside aggregates.

Things to watch out for

Division by COUNT(r.pnr) is safe because every group has at least one ticket (groups come from reservations).

Q65 · Q5b Cancellation reasons

The query

```sql
-- Q5b CANCELLATION REASONS  (share of total using RATIO_TO_REPORT)
SELECT reason, COUNT(*) AS cancellations,
       ROUND(100 * RATIO_TO_REPORT(COUNT(*)) OVER (), 1) AS share_pct,
       SUM(refund_amount) AS refunded
FROM   cancellation
GROUP  BY reason
ORDER  BY cancellations DESC;
```

Source: parts/04_queries.sql, lines 162–168

What it is for

One line: count of cancellations per reason with each reason's share of the total.

Detailed: Introduces the analytic function **RATIO_TO_REPORT**.

Simple explanation

Report Q5b – cancellations per reason: how many cancellations had each reason and what share of all cancellations that is (for example, "Change of plans" = 27.8%).

Concepts used

Analytic (window) functions applied on top of aggregates. Window functions are evaluated after GROUP BY, so they can take an aggregate as input.

Things to watch out for

NULL reasons would form their own group (reason is optional).

Q66 · Q6a Peak travel routes (DENSE_RANK)

The query

```sql
-- Q6a PEAK TRAVEL ROUTES  (ranked by passengers carried)
SELECT ro.route_name,
       COUNT(*)                                        AS passengers,
       DENSE_RANK() OVER (ORDER BY COUNT(*) DESC)   AS demand_rank
FROM   reservation r
       JOIN train_schedule s ON s.schedule_id = r.schedule_id
       JOIN train t          ON t.train_no = s.train_no
       JOIN route ro         ON ro.route_id = t.route_id
WHERE  r.status IN ('CONFIRMED','COMPLETED','WAITLIST')
GROUP  BY ro.route_name;
```

Source: parts/04_queries.sql, lines 170–179

What it is for

One line: ranks routes by the number of passengers who booked (confirmed, completed or waitlisted).

Detailed: Waitlisted passengers are included because they represent **demand**, even if they did not get a seat.

Simple explanation

Report Q6a – busiest routes: counts passengers per route (including waitlisted ones, because they also wanted to travel) and gives each route a rank: 1 = busiest.

Concepts used

Ranking window functions, WHERE before GROUP BY, demand vs supply.

Things to watch out for

Routes with zero non-cancelled tickets would not appear (driving table is reservation).

Q67 · Q6b Top 5 origin–destination pairs

The query

```sql
-- Q6b TOP 5 ORIGIN-DESTINATION PAIRS (peak travel segments)
SELECT fs.city || ' -> ' || ts.city AS segment, COUNT(*) AS passengers
FROM   reservation r
       JOIN station fs ON fs.station_code = r.from_station
       JOIN station ts ON ts.station_code = r.to_station
WHERE  r.status IN ('CONFIRMED','COMPLETED')
GROUP  BY fs.city, ts.city
ORDER  BY passengers DESC
FETCH FIRST 5 ROWS ONLY;
```

Source: parts/04_queries.sql, lines 181–189

What it is for

One line: the five busiest city-to-city segments.

Detailed: Groups tickets by boarding city and destination city, sorts, and keeps the top 5 rows.

Simple explanation

Report Q6b – top 5 city pairs: counts passengers for each "from city → to city" pair (like *Chennai → Madurai*), sorts from most to least, and keeps only the top 5.

Concepts used

Row limiting (FETCH FIRST vs ROWNUM), grouping by expressions' components, self-join of a lookup table.

Things to watch out for

Grouping by city (not station) merges different stations of the same city.

Q68 · Q6c Peak travel days of the week

The query

```sql
-- Q6c PEAK TRAVEL DAYS OF THE WEEK
SELECT TO_CHAR(s.journey_date, 'DY') AS travel_day,
       COUNT(*)                      AS passengers,
       ROUND(100 * RATIO_TO_REPORT(COUNT(*)) OVER (), 1) AS share_pct
FROM   reservation r JOIN train_schedule s ON s.schedule_id = r.schedule_id
WHERE  r.status IN ('CONFIRMED','COMPLETED')
GROUP  BY TO_CHAR(s.journey_date, 'DY')
ORDER  BY passengers DESC;
```

Source: parts/04_queries.sql, lines 191–198

What it is for

One line: passengers per day of the week with each day's share.

Detailed: Groups by an **expression**, the day name of the journey date.

Simple explanation

Report Q6c – busiest days of the week: turns each trip date into a day name (SUN, MON…), counts passengers per day, and shows each day's share. Sunday is the busiest.

Concepts used

Grouping by expressions, date formatting, NLS language dependence.

Things to watch out for

Results depend on session language settings; for language-independent output use `TO_CHAR(d, 'DY', 'NLS_DATE_LANGUAGE=ENGLISH')`.

Q69 · Q7 Highest and lowest utilized trains

The query

```sql
-- Q7 HIGHEST AND LOWEST UTILIZED TRAINS
SELECT CASE WHEN rk_high = 1 THEN 'HIGHEST' ELSE 'LOWEST' END AS position,
       train_no, train_name, route_label, occupancy_pct
FROM  (SELECT u.*, RANK() OVER (ORDER BY occupancy_pct DESC) AS rk_high,
                   RANK() OVER (ORDER BY occupancy_pct ASC)  AS rk_low
       FROM   v_train_utilization u)
WHERE  rk_high = 1 OR rk_low = 1;
```

Source: parts/04_queries.sql, lines 200–206

What it is for

One line: finds the train(s) with the highest and the lowest occupancy.

Detailed: Two rankings, one ascending and one descending, computed in an inline view; the outer query keeps rank 1 of either.

Simple explanation

Report Q7 – fullest and emptiest trains: ranks trains by occupancy from top and from bottom, then keeps the train(s) ranked first in each. If two trains tie for top (as 12639 and 12027 do at 96.88%), both are shown.

Concepts used

Window functions in inline views, why WHERE cannot use them directly, tie handling with RANK.

Things to watch out for

If only one train existed, it would be both highest and lowest; the CASE would label it HIGHEST.

Q70 · Q8 Passenger travel frequency

The query

```sql
-- Q8 PASSENGER TRAVEL FREQUENCY  (trips, distance, spend, banding with CASE)
SELECT p.passenger_id, p.full_name, p.home_city,
       COUNT(*)                    AS trips,
       SUM(v.distance_km)          AS km_travelled,
       SUM(v.fare)                 AS total_spent,
       CASE WHEN COUNT(*) >= 6 THEN 'FREQUENT'
            WHEN COUNT(*) >= 3 THEN 'REGULAR'
            ELSE 'OCCASIONAL' END  AS traveller_type
FROM   v_ticket_details v JOIN passenger p ON p.passenger_id = v.passenger_id
WHERE  v.status IN ('CONFIRMED','COMPLETED')
GROUP  BY p.passenger_id, p.full_name, p.home_city
ORDER  BY trips DESC, total_spent DESC;
```

Source: parts/04_queries.sql, lines 208–219

What it is for

One line: for each passenger: trips, kilometres, money spent and a traveller category (FREQUENT / REGULAR / OCCASIONAL).

Detailed: Uses the ticket view for distance and a CASE on an aggregate to band customers, like a loyalty programme.

Simple explanation

Report Q8 – how often each passenger travels: number of trips, kilometres travelled and money spent, plus a label: - 6 or more trips → FREQUENT - 3 to 5 trips → REGULAR - fewer → OCCASIONAL

Concepts used

Aggregates inside CASE, multi-key sorting, using views in joins.

Things to watch out for

`p.home_city` exists only after the RENAME in 03_dml.sql (Q34).

Q71 · Q9 Revenue by train with ROLLUP

The query

```sql
-- Q9 REVENUE BY TRAIN  (route subtotals and grand total with ROLLUP)
--    The refund per ticket is worked out once in the inline view, then summed.
SELECT NVL(route_name, '** ALL ROUTES **')                                    AS route_name,
       NVL(train_name, DECODE(GROUPING(route_name), 0, '-- route total --', ' ')) AS train_name,
       SUM(amount)                                                            AS collected,
       SUM(refunded)                                                          AS refunded,
       SUM(amount) - SUM(refunded)                                            AS net_revenue
FROM  (SELECT ro.route_name, t.train_name, pm.amount,
              NVL(cn.refund_amount, 0)
              + CASE WHEN pm.pay_status = 'REFUNDED' AND cn.cancel_id IS NULL THEN pm.amount ELSE 0 END
AS refunded
       FROM   payment pm
              JOIN reservation r        ON r.pnr = pm.pnr
              JOIN train_schedule s     ON s.schedule_id = r.schedule_id
              JOIN train t              ON t.train_no = s.train_no
              JOIN route ro             ON ro.route_id = t.route_id
              LEFT JOIN cancellation cn ON cn.pnr = r.pnr) x
GROUP  BY ROLLUP (x.route_name, x.train_name)
ORDER  BY GROUPING(x.route_name), x.route_name, GROUPING(x.train_name), net_revenue DESC;
```

Source: parts/04_queries.sql, lines 221–238

What it is for

One line: money collected, refunded and net revenue per train, with a subtotal per route and a grand total.

Detailed: `GROUP BY ROLLUP` produces several levels of totals in one query. The refund per ticket is computed once in the inline view `x` and then summed (simplified on 28 Oct 2026; same result as the original).

Simple explanation

Report Q9 – money per train: money collected, money refunded and net revenue (collected − refunded) for each train, plus a **subtotal for each route** and a **grand total** at the end.

`ROLLUP` is what automatically adds the subtotal and total lines. The refund of each ticket is worked out once first, then everything is added up.

Concepts used

ROLLUP (super-aggregates), **GROUPING()**, inline views, NULL-safe sums, ORDER BY with a mix of expressions and aliases, **alias name resolution**.

Things to watch out for

On subtotal rows the grouped columns are NULL; a real NULL route_name would be indistinguishable from a subtotal without GROUPING(), which is why GROUPING is used for labels.

Q72 · Q10 Routes requiring additional capacity (HAVING)

The query

```sql
-- Q10 ROUTES REQUIRING ADDITIONAL CAPACITY  (HAVING on aggregated journey data)
SELECT ro.route_name,
       COUNT(*)                                                    AS journeys,
       SUM(CASE WHEN v.occupancy_pct >= 95 THEN 1 ELSE 0 END)      AS sold_out_journeys,
       SUM(v.waitlisted)                                           AS unserved_passengers,
       ROUND(100 * SUM(v.seats_occupied) / SUM(v.capacity), 2)     AS occupancy_pct
FROM   v_reservation_performance v JOIN route ro ON ro.route_id = v.route_id
GROUP  BY ro.route_name
HAVING SUM(v.waitlisted) > 0
   AND 100 * SUM(v.seats_occupied) / SUM(v.capacity) >= 75
ORDER  BY unserved_passengers DESC;
```

Source: parts/04_queries.sql, lines 240–250

What it is for

One line: routes that turned passengers away **and** are at least 75% full overall.

Detailed: Demonstrates HAVING with two aggregate conditions.

Simple explanation

Report Q10 – routes that need more capacity: routes where passengers were turned away (waitlisted) **and** the trains were at least 75% full overall.

Because these conditions are about totals of a group (not single rows), they go in `HAVING`, not `WHERE`.

Concepts used

WHERE vs HAVING (row filter vs group filter), aggregate conditions.

Things to watch out for

`SUM(CASE … ELSE 0 END)` here sums 1s and 0s; `COUNT(CASE … THEN 1 END)` would be equivalent.

SUBQUERY VOCABULARY — READ THIS ONCE

A **subquery** is a SELECT inside another statement. A **scalar subquery** returns one value (one row, one column). A **multiple-row subquery** returns a list, used with `IN`, `ANY`, `ALL` or `EXISTS`. An **inline view** is a subquery in FROM. A **correlated subquery** refers to a column of the outer query, so it is (logically) re-evaluated for every outer row; a **non-correlated** subquery is independent and runs once. A **WITH clause** (common table expression, CTE) names a subquery so it can be referenced, even several times, in the main query.

Q73 · A1 Ten-table join: a passenger's travel history

The query

```sql
-- A1 MULTI-TABLE JOIN (10 relations) : complete travel history of one passenger.
--    Station is joined twice (boarding and destination) using table aliases.
SELECT r.pnr, t.train_no, t.train_name, s.journey_date,
       fs.station_name AS boarding, ts.station_name AS destination,
       cc.class_name, c.coach_label || '-' || st.seat_no || ' ' || st.berth_type AS seat,
       r.fare, pm.pay_mode, r.status
FROM   passenger p
       JOIN reservation r     ON r.passenger_id = p.passenger_id
       JOIN train_schedule s  ON s.schedule_id = r.schedule_id
       JOIN train t           ON t.train_no = s.train_no
       JOIN station fs        ON fs.station_code = r.from_station
       JOIN station ts        ON ts.station_code = r.to_station
       JOIN coach_class cc    ON cc.class_code = r.class_code
       JOIN payment pm        ON pm.pnr = r.pnr
       LEFT JOIN seat st      ON st.seat_id = r.seat_id
       LEFT JOIN coach c      ON c.coach_id = st.coach_id
WHERE  p.passenger_id = 1
ORDER  BY s.journey_date;
```

Source: parts/04_queries.sql, lines 256–273

What it is for

One line: lists every ticket of passenger 1 with train, date, both station names, class, coach-seat-berth, fare, payment mode and status.

Detailed: Demonstrates a join across ten table references, a table joined twice under different aliases, and outer joins for optional data.

Simple explanation

A1 – a passenger's full travel history (passenger 1): every ticket with train, date, boarding and destination station names, class, coach-seat, fare, payment mode and status.

It joins ten tables together. The station table is used twice, once for the boarding station and once for the destination, with different nicknames (aliases).

Concepts used

Multi-way joins following foreign keys, table aliases, role-playing joins, inner vs outer joins, implicit type conversion.

Things to watch out for

With many joins, one wrong ON condition can silently multiply rows. Count rows before and after adding each join when debugging.

Q74 · A2 Correlated subquery: trains above their route's average

The query

```sql
-- A2 CORRELATED SUBQUERY : trains whose occupancy is above the average occupancy of THEIR route.
--    The inner query is re-evaluated for each outer row using u.route_id.
SELECT u.train_no, u.train_name, u.route_label, u.occupancy_pct,
       (SELECT ROUND(AVG(x.occupancy_pct), 2) FROM v_train_utilization x WHERE x.route_id = u.route_id)
AS route_avg_pct
FROM   v_train_utilization u
WHERE  u.occupancy_pct > (SELECT AVG(x.occupancy_pct)
                          FROM   v_train_utilization x
                          WHERE  x.route_id = u.route_id)
ORDER  BY u.route_id, u.occupancy_pct DESC;
```

Source: parts/04_queries.sql, lines 275–283

What it is for

One line: trains whose occupancy is higher than the average occupancy of trains on the **same** route.

Detailed: The comparison value differs for each train (its own route's average), which is exactly what a correlated subquery provides.

Simple explanation

A2 – trains fuller than their route's average. For each train, it calculates the average occupancy of all trains on the **same route** and keeps the train if it is above that average.

The inner calculation depends on the train being checked (its route), which makes it a **correlated subquery**.

Concepts used

Correlated scalar subqueries in WHERE and SELECT, aliasing the same view twice (u and x).

Things to watch out for

Route 4 has one train, so its average equals its own value and `>` excludes it.

Q75 · A3 Passengers with repeated cancellations

The query

```sql
-- A3 PASSENGERS WITH REPEATED CANCELLATIONS (2 or more) with cancellation ratio (scalar subquery)
SELECT p.passenger_id, p.full_name,
       COUNT(*)                                                              AS cancellations,
       (SELECT COUNT(*) FROM reservation x WHERE x.passenger_id = p.passenger_id) AS total_bookings,
       ROUND(100 * COUNT(*) / (SELECT COUNT(*) FROM reservation x WHERE x.passenger_id = p.passenger_id)
, 1)
                                                                             AS cancel_ratio_pct,
       LISTAGG(t.train_no, ', ') WITHIN GROUP (ORDER BY s.journey_date)      AS trains_cancelled
FROM   reservation r
       JOIN passenger p      ON p.passenger_id = r.passenger_id
       JOIN train_schedule s ON s.schedule_id = r.schedule_id
       JOIN train t          ON t.train_no = s.train_no
WHERE  r.status = 'CANCELLED'
GROUP  BY p.passenger_id, p.full_name
HAVING COUNT(*) >= 2
ORDER  BY cancellations DESC;
```

Source: parts/04_queries.sql, lines 285–299

What it is for

One line: passengers with two or more cancellations, their total bookings, cancellation ratio and the trains they cancelled.

Detailed: Combines GROUP BY/HAVING with correlated scalar subqueries and string aggregation (LISTAGG).

Simple explanation

A3 – passengers who cancelled 2 or more times: how many times they cancelled, how many tickets they booked in total, the cancellation % and the list of trains they cancelled (joined into one text with `LISTAGG`).

Concepts used

Correlated subqueries inside a grouped query (they may reference GROUP BY columns), LISTAGG, HAVING.

Things to watch out for

LISTAGG raises ORA-01489 if the result exceeds 4000 bytes; `ON OVERFLOW TRUNCATE` avoids that.

Q76 · A4 Nested subquery: routes above average demand

The query

```sql
-- A4 NESTED SUBQUERY : routes whose passenger demand is above the average route demand
SELECT ro.route_name, COUNT(*) AS passengers
FROM   reservation r
       JOIN train_schedule s ON s.schedule_id = r.schedule_id
       JOIN train t          ON t.train_no = s.train_no
       JOIN route ro         ON ro.route_id = t.route_id
WHERE  r.status <> 'CANCELLED'
GROUP  BY ro.route_name
HAVING COUNT(*) > (SELECT AVG(route_pax)
                   FROM  (SELECT COUNT(*) AS route_pax
                          FROM   reservation r2
                                 JOIN train_schedule s2 ON s2.schedule_id = r2.schedule_id
                                 JOIN train t2          ON t2.train_no = s2.train_no
                          WHERE  r2.status <> 'CANCELLED'
                          GROUP  BY t2.route_id))
ORDER  BY passengers DESC;
```

Source: parts/04_queries.sql, lines 301–316

What it is for

One line: routes whose passenger count is above the average passenger count of all routes.

Detailed: An aggregate compared against an **average of aggregates**, computed in a two-level nested subquery.

Simple explanation

A4 – routes with above-average demand. It counts passengers on each route, works out the average count across all routes (185.25), and keeps the routes above it (Chennai–Bengaluru and Chennai–Madurai).

Concepts used

Nested (non-correlated) subqueries, subquery in HAVING, average of grouped counts.

Things to watch out for

Outer and inner filters must match, or you compare different measures.

Q77 · A5 WITH clause: stations above average traffic

The query

```sql
-- A5 STATIONS HANDLING HIGHER-THAN-AVERAGE PASSENGER TRAFFIC (WITH clause + subquery)
WITH footfall AS (
  SELECT station_code, COUNT(*) AS movements
  FROM  (SELECT from_station AS station_code FROM reservation WHERE status IN ('CONFIRMED','COMPLETED')
         UNION ALL
         SELECT to_station FROM reservation WHERE status IN ('CONFIRMED','COMPLETED'))
  GROUP  BY station_code)
SELECT st.station_code, st.station_name, f.movements,
       ROUND((SELECT AVG(movements) FROM footfall), 1) AS network_avg
FROM   footfall f JOIN station st ON st.station_code = f.station_code
WHERE  f.movements > (SELECT AVG(movements) FROM footfall)
ORDER  BY f.movements DESC;
```

Source: parts/04_queries.sql, lines 318–329

What it is for

One line: stations whose boardings + alightings exceed the network average.

Detailed: Defines the footfall calculation once with WITH, then uses it three times.

Simple explanation

A5 – stations busier than average. It counts how many passengers got on or off at each station, works out the average, and lists the stations above it.

The `WITH` part gives the station-count calculation a name (`footfall`) so it can be reused without writing it again.

Concepts used

Common table expressions, UNION ALL vs UNION, scalar subqueries on a CTE.

Things to watch out for

The average excludes stations with zero traffic, unlike Q2, which lists TVC with 0. Be ready to say which definition you intend.

Q78 · A6 NOT EXISTS + EXISTS: trains needing capacity

The query

```sql
-- A6 TRAINS REQUIRING CAPACITY ENHANCEMENT (correlated NOT EXISTS + EXISTS):
--    every journey ran at >= 90% occupancy AND at least one passenger was left on the waitlist
SELECT u.train_no, u.train_name, u.occupancy_pct, u.waitlisted
FROM   v_train_utilization u
WHERE  NOT EXISTS (SELECT 1 FROM v_reservation_performance v
                   WHERE  v.train_no = u.train_no AND v.occupancy_pct < 90)
AND    EXISTS     (SELECT 1 FROM reservation r JOIN train_schedule s ON s.schedule_id = r.schedule_id
                   WHERE  s.train_no = u.train_no AND r.status = 'WAITLIST');
```

Source: parts/04_queries.sql, lines 331–338

What it is for

One line: trains where **every** journey ran at ≥ 90% occupancy **and** at least one passenger was waitlisted.

Detailed: "Every journey" is expressed as "there is no journey below 90%", the classic **NOT EXISTS** pattern for universal conditions (relational division).

Simple explanation

A6 – trains that need more capacity: trains where **every** trip was at least 90% full **and** at least one passenger was waitlisted.

"Every trip was 90%+ full" is checked as "there is **no** trip below 90%" (`NOT EXISTS`), and "someone was waitlisted" as "there **is** a waitlisted ticket" (`EXISTS`).

Concepts used

EXISTS/NOT EXISTS, universal quantification ("for all") via double negation, correlated subqueries, short-circuit evaluation (EXISTS stops at the first row).

Things to watch out for

A train with no journeys would satisfy NOT EXISTS vacuously; the EXISTS (waitlist) condition prevents it from being listed.

Q79 · A7 Correlated HAVING: above-average spenders per city

The query

```sql
-- A7 CORRELATED : passengers whose total spend exceeds the average spend of passengers from the same city
SELECT p.passenger_id, p.full_name, p.home_city, SUM(r.fare) AS spent
FROM   passenger p JOIN reservation r ON r.passenger_id = p.passenger_id
WHERE  r.status IN ('CONFIRMED','COMPLETED')
GROUP  BY p.passenger_id, p.full_name, p.home_city
HAVING SUM(r.fare) > (SELECT SUM(r2.fare) / COUNT(DISTINCT r2.passenger_id)
                      FROM   reservation r2 JOIN passenger p2 ON p2.passenger_id = r2.passenger_id
                      WHERE  r2.status IN ('CONFIRMED','COMPLETED')
                      AND    p2.home_city = p.home_city)
ORDER  BY p.home_city, spent DESC;
```

Source: parts/04_queries.sql, lines 340–349

What it is for

One line: passengers who spent more than the average spend of travellers from their own home city.

Detailed: The correlated subquery sits in **HAVING** and references a GROUP BY column (`p.home_city`).

Simple explanation

A7 – passengers who spend more than average for their city. For each passenger, it compares their total spending with the average spending of travellers from the same home city, and keeps those above it.

Concepts used

Correlated subqueries in HAVING, COUNT(DISTINCT), per-group comparison.

Things to watch out for

Passengers with NULL home_city: `p2.home_city = p.home_city` is never true for NULL, so the subquery returns NULL and the comparison is UNKNOWN → excluded.

Q80 · A8 IN and NOT IN: loyal passengers

The query

```sql
-- A8 IN / NOT IN : loyal passengers - 3 or more trips on near-full trains (>= 90% occupancy), never
cancelled
SELECT p.passenger_id, p.full_name, p.home_city
FROM   passenger p
WHERE  p.passenger_id IN (SELECT r.passenger_id
                          FROM   reservation r JOIN train_schedule s ON s.schedule_id = r.schedule_id
                          WHERE  r.status IN ('CONFIRMED','COMPLETED')
                          AND    s.train_no IN (SELECT train_no FROM v_train_utilization WHERE occupancy
_pct >= 90)
                          GROUP  BY r.passenger_id
                          HAVING COUNT(*) >= 3)
AND    p.passenger_id NOT IN (SELECT passenger_id FROM reservation WHERE status = 'CANCELLED')
ORDER  BY p.passenger_id;
```

Source: parts/04_queries.sql, lines 351–361

What it is for

One line: passengers with 3 or more trips on near-full trains (≥ 90% occupancy) who have never cancelled.

Detailed: A multiple-row subquery with IN (which itself contains another IN), plus NOT IN.

Simple explanation

A8 – loyal passengers: passengers who made 3 or more trips on very full trains (90%+ occupancy) **and** never cancelled a ticket.

`IN` keeps passengers who are in the first list; `NOT IN` removes passengers who appear in the list of people who cancelled.

Concepts used

IN/NOT IN with subqueries, nested subqueries, GROUP BY/HAVING inside a subquery, **NOT IN and NULL**.

Things to watch out for

IN with a subquery returning duplicates is fine; duplicates do not multiply outer rows (unlike a join).

Q81 · A9 Correlated subquery on a CTE: under-used coaches

The query

```sql
-- A9 CORRELATED (on a WITH clause) : coaches whose occupancy % is below the average of their own
train's coaches
WITH coach_occ AS (
  SELECT c.train_no, c.coach_id, c.coach_label, c.class_code,
         ROUND(100 * COUNT(r.pnr) / (cc.seats_per_coach * j.journeys), 1) AS occupancy_pct
  FROM   coach c
         JOIN coach_class cc ON cc.class_code = c.class_code
         JOIN (SELECT train_no, COUNT(*) AS journeys FROM train_schedule GROUP BY train_no) j ON j.train
_no = c.train_no
         JOIN seat st ON st.coach_id = c.coach_id
         LEFT JOIN reservation r ON r.seat_id = st.seat_id AND r.status IN ('CONFIRMED','COMPLETED')
  GROUP  BY c.train_no, c.coach_id, c.coach_label, c.class_code, cc.seats_per_coach, j.journeys)
SELECT o.train_no, o.coach_label, o.class_code, o.occupancy_pct,
       (SELECT ROUND(AVG(x.occupancy_pct), 1) FROM coach_occ x WHERE x.train_no = o.train_no) AS train_a
vg_pct
FROM   coach_occ o
WHERE  o.occupancy_pct < (SELECT AVG(x.occupancy_pct) FROM coach_occ x WHERE x.train_no = o.train_no) -
10
ORDER  BY o.train_no, o.coach_label;
```

Source: parts/04_queries.sql, lines 363–377

What it is for

One line: coaches whose occupancy is more than 10 points below the average of their own train's coaches.

Detailed: Finds coaches that a train could do without or reassign.

Simple explanation

A9 – under-used coaches: coaches whose occupancy is more than 10 points below the average of the other coaches on the same train. Useful for deciding which coaches a train could do without.

Concepts used

CTE referenced several times, correlated subquery on a CTE, threshold comparisons.

Things to watch out for

The displayed average is rounded, while the WHERE uses the unrounded average; borderline cases may look inconsistent by 0.05.

The Capacity-Risk Module (Part 10, BTL-6 challenge)

The module answers a management question: **which trains need more capacity?** It classifies each train with three rules, stored as data:

Status	Rule (default thresholds)	Meaning and action
CAPACITY REQUIRED	occupancy ≥ 90% **and** demand ≥ 100% **and** cancellation rate ≤ 10%	Full, people are turned away, and cancellations will not free enough seats → attach coaches or run a special train

Status	Rule (default thresholds)	Meaning and action
HIGH DEMAND	not the above, but occupancy ≥ 75% **or** demand ≥ 90%	Strong demand that is still serviceable → monitor weekly, extra quota on peak days
NORMAL	everything else	No action; below 50% occupancy suggests reviewing coach composition

Q82 · CREATE TABLE risk_threshold and its rows

The query

```
-- Classification rules (thresholds are stored in a table so management can tune them):
--   CAPACITY REQUIRED : occupancy >= 90%  AND  demand >= 100%  AND  cancellation rate <= 10%
--       the train is full, passengers are being turned away (waitlist), and the waitlist is
--       unlikely to clear through cancellations -> attach coaches / run a special train.
--   HIGH DEMAND       : not the above, but occupancy >= 75%  OR  demand >= 90%
--       strong but currently serviceable demand (or high demand offset by many cancellations)
--       -> monitor every week, consider extra quota on peak days.
--   NORMAL            : everything else.
CREATE TABLE risk_threshold (
  param_code   VARCHAR2(20)  CONSTRAINT pk_risk_threshold PRIMARY KEY,
  param_value  NUMBER(6,2)   NOT NULL CONSTRAINT ck_rt_value CHECK (param_value >= 0),
  description  VARCHAR2(100) NOT NULL
);
INSERT INTO risk_threshold VALUES ('CR_MIN_OCCUPANCY',  90, 'Capacity Required: minimum occupancy %');
INSERT INTO risk_threshold VALUES ('CR_MIN_DEMAND',    100, 'Capacity Required: minimum demand %
(waitlist exists)');
INSERT INTO risk_threshold VALUES ('CR_MAX_CANCEL',      10, 'Capacity Required: maximum cancellation
rate %');
INSERT INTO risk_threshold VALUES ('HD_MIN_OCCUPANCY',  75, 'High Demand: minimum occupancy %');
INSERT INTO risk_threshold VALUES ('HD_MIN_DEMAND',     90, 'High Demand: minimum demand %');
```

Source: parts/04_queries.sql, lines 382–399

What it is for

One line: stores the five tunable thresholds of the classification rules as rows.

Detailed: A **parameter table**: the rules read their limits from data, so management can change them without editing code.

Simple explanation

This creates a small table of **settings for the risk rules**, with the five limits used to classify trains:

- CAPACITY REQUIRED needs occupancy ≥ 90%, demand ≥ 100%, cancellation rate ≤ 10%.
- HIGH DEMAND needs occupancy ≥ 75% or demand ≥ 90%.

Keeping the limits in a table means management can change a rule with one UPDATE, without touching any code.

Concepts used

Key–value (entity–attribute–value) parameter tables, separating policy from code.

Things to watch out for

These INSERTs run **after** the DDL in the same script; the COMMIT after CREATE TABLE capacity_risk_log (and the CREATE itself, as DDL) makes them permanent.

Q83 · CREATE TABLE capacity_risk_log

The query

```
CREATE TABLE capacity_risk_log (
  run_date           DATE         NOT NULL,
  train_no           NUMBER(5)    NOT NULL CONSTRAINT fk_crl_train REFERENCES train(train_no),
  occupancy_pct      NUMBER(6,2),
  cancellation_rate  NUMBER(6,2),
  demand_pct         NUMBER(6,2),
  risk_status        VARCHAR2(20) NOT NULL,
  CONSTRAINT pk_capacity_risk_log PRIMARY KEY (run_date, train_no)
);
COMMIT;
```

Source: parts/04_queries.sql, lines 401–410

What it is for

One line: a history table storing each day's classification of each train.

Detailed: Keeping a log allows trends over time ("12639 has been CAPACITY REQUIRED for 3 weeks").

Simple explanation

This creates the **daily log table** that remembers how each train was classified on each day. A train can have only one entry per day.

Concepts used

Snapshot/history tables, composite PK with a date.

Things to watch out for

The table has no index on train_no alone (it is the second PK column), so "history of one train" queries scan more; fine at this size.

Q84 · FUNCTION fn_capacity_risk

The query

```
CREATE OR REPLACE FUNCTION fn_capacity_risk (
  p_occupancy NUMBER, p_cancel_rate NUMBER, p_demand NUMBER
) RETURN VARCHAR2 IS
  v_cr_occ NUMBER; v_cr_dem NUMBER; v_cr_can NUMBER; v_hd_occ NUMBER; v_hd_dem NUMBER;
BEGIN
  SELECT MAX(DECODE(param_code, 'CR_MIN_OCCUPANCY', param_value)),
         MAX(DECODE(param_code, 'CR_MIN_DEMAND',    param_value)),
         MAX(DECODE(param_code, 'CR_MAX_CANCEL',    param_value)),
         MAX(DECODE(param_code, 'HD_MIN_OCCUPANCY', param_value)),
         MAX(DECODE(param_code, 'HD_MIN_DEMAND',    param_value))
  INTO   v_cr_occ, v_cr_dem, v_cr_can, v_hd_occ, v_hd_dem
  FROM   risk_threshold;

  IF p_occupancy >= v_cr_occ AND p_demand >= v_cr_dem AND NVL(p_cancel_rate, 0) <= v_cr_can THEN
    RETURN 'CAPACITY REQUIRED';
  ELSIF p_occupancy >= v_hd_occ OR p_demand >= v_hd_dem THEN
    RETURN 'HIGH DEMAND';
  END IF;
  RETURN 'NORMAL';
END;
/
```

Source: parts/04_queries.sql, lines 412–432

What it is for

One line: takes a train's occupancy, cancellation rate and demand and returns its risk category.

Detailed: A **stored function** returns a value and can be called inside SQL, like a built-in function. Centralising the rules means the report, the procedure and the what-if analysis all classify identically.

Simple explanation

This **function** takes a train's occupancy, cancellation rate and demand, and **returns its label**: CAPACITY REQUIRED, HIGH DEMAND or NORMAL.

It first reads the five limits from the settings table, then: - if the train is very full, people are being turned away and few cancel → **CAPACITY REQUIRED**; - otherwise, if it is quite full or demand is high → **HIGH DEMAND**; - otherwise → **NORMAL**.

Because it is a function, it can be used inside any SELECT, so every report labels trains the same way.

Concepts used

Functions vs procedures, calling PL/SQL from SQL, IF/ELSIF/ELSE, pivoting rows to columns, NULL handling with NVL, AND/OR precedence.

Things to watch out for

If RISK_THRESHOLD is empty, the aggregate query still returns one row of NULLs (no NO_DATA_FOUND), every comparison is UNKNOWN, and the function returns NORMAL for every train: silently wrong.

Q85 · 10.1 Decision-support report

The query

```
-- 10.1 DECISION-SUPPORT REPORT
SELECT u.train_no                                                          AS "Train",
       u.route_label                                                       AS "Route",
       u.occupancy_pct                                                     AS "Occupancy %",
       u.cancellation_rate                                                 AS "Cancellation Rate",
       u.demand_pct                                                        AS "Average Demand",
       ROUND(AVG(u.occupancy_pct) OVER (PARTITION BY u.route_id), 2)       AS "Route Avg Occ %",
       fn_capacity_risk(u.occupancy_pct, u.cancellation_rate, u.demand_pct) AS "Risk Status",
       CASE fn_capacity_risk(u.occupancy_pct, u.cancellation_rate, u.demand_pct)
            WHEN 'CAPACITY REQUIRED' THEN 'Add ' || CEIL(u.waitlisted / u.journeys) || ' seat(s)/
journey: attach extra coach'
            WHEN 'HIGH DEMAND'       THEN 'Monitor weekly; extra quota on peak days'
            ELSE CASE WHEN u.occupancy_pct < 50 THEN 'Under-used: review coach composition'
                      ELSE 'No action' END
       END                                                                 AS "Recommended Action"
FROM   v_train_utilization u
ORDER  BY DECODE(fn_capacity_risk(u.occupancy_pct, u.cancellation_rate, u.demand_pct),
                 'CAPACITY REQUIRED', 1, 'HIGH DEMAND', 2, 3),
          u.occupancy_pct DESC;
```

Source: parts/04_queries.sql, lines 439–456

What it is for

One line: the management report: each train's measures, its route average, risk status and a recommended action, most urgent first.

Detailed: Combines a view, a window function, a stored function and nested CASE expressions.

Simple explanation

The management report: for every train it shows occupancy, cancellation rate, demand, the average occupancy of its route, its risk label (from the function), and a **recommended action**:

- CAPACITY REQUIRED → "Add N seats per trip: attach extra coach" (N = waitlisted ÷ trips, rounded up).
- HIGH DEMAND → "Monitor weekly".
- NORMAL → "No action", or "review coach composition" if less than 50% full.

The most urgent trains are listed first.

Concepts used

Window functions vs GROUP BY, user-defined functions in SELECT/ORDER BY, simple vs searched CASE, custom ordering with DECODE, quoted identifiers.

Things to watch out for

`u.waitlisted / u.journeys` : journeys is never 0 for a train in the view (every row comes from at least one journey).

Q86 · PROCEDURE prc_capacity_risk_report (10.2)

The query

```
-- 10.2 PROCEDURE : stores today's classification in CAPACITY_RISK_LOG and prints a summary
CREATE OR REPLACE PROCEDURE prc_capacity_risk_report AS
BEGIN
  DELETE FROM capacity_risk_log WHERE run_date = TRUNC(SYSDATE);
  INSERT INTO capacity_risk_log
  SELECT TRUNC(SYSDATE), train_no, occupancy_pct, cancellation_rate, demand_pct,
         fn_capacity_risk(occupancy_pct, cancellation_rate, demand_pct)
  FROM   v_train_utilization;

  DBMS_OUTPUT.PUT_LINE('TRAIN CAPACITY RISK SUMMARY - ' || TO_CHAR(SYSDATE, 'DD-MON-YYYY'));
  FOR rec IN (SELECT risk_status, COUNT(*) AS trains,
                     LISTAGG(train_no, ', ') WITHIN GROUP (ORDER BY train_no) AS train_list
              FROM   capacity_risk_log WHERE run_date = TRUNC(SYSDATE)
              GROUP  BY risk_status ORDER BY risk_status) LOOP
    DBMS_OUTPUT.PUT_LINE(RPAD(rec.risk_status, 20) || rec.trains || ' train(s): ' || rec.train_list);
  END LOOP;
  COMMIT;
END;
/
EXEC prc_capacity_risk_report
```

Source: parts/04_queries.sql, lines 458–477

What it is for

One line: saves today's classification of every train into CAPACITY_RISK_LOG and prints a summary.

Detailed: Turns the report into a repeatable daily job with history.

Simple explanation

This **procedure** is the daily job: it deletes today's old log entries (so running it twice a day is safe), writes today's label for every train into the log, prints a short summary (how many trains in each category and which ones) and saves.

`EXEC` runs it.

Concepts used

Procedures with DML and COMMIT, idempotent batch jobs (delete-then-insert), cursor FOR loops, LISTAGG, RPAD.

Things to watch out for

The COMMIT inside the procedure also commits any uncommitted work of the **caller**: test suite 06 calls it, which commits anything pending in that session. Many teams prefer to leave COMMIT to the caller (as book_ticket does).

Q87 · 10.3 What-if analysis with ROLLBACK

The query

```
-- 10.3 WHAT-IF ANALYSIS : tolerate up to 15% cancellations -> which trains change category?
UPDATE risk_threshold SET param_value = 15 WHERE param_code = 'CR_MAX_CANCEL';
SELECT l.train_no, l.risk_status AS current_status,
       fn_capacity_risk(l.occupancy_pct, l.cancellation_rate, l.demand_pct) AS what_if_status
FROM   capacity_risk_log l
WHERE  l.run_date = TRUNC(SYSDATE)
AND    l.risk_status <> fn_capacity_risk(l.occupancy_pct, l.cancellation_rate, l.demand_pct);
ROLLBACK;
```

Source: parts/04_queries.sql, lines 479–486

What it is for

One line: temporarily raises the tolerated cancellation rate to 15%, shows which trains would change category, then undoes the change.

Detailed: Demonstrates using a transaction as a sandbox: change, observe, roll back.

Simple explanation

What-if analysis: "What if we allow up to 15% cancellations instead of 10%?"

1. It temporarily changes the limit to 15.
2. It re-labels the trains using the new limit and shows only those whose label would change (train 12027 would become CAPACITY REQUIRED).
3. `ROLLBACK` undoes the change, so the real setting stays 10.

It is a safe experiment: nothing is permanently changed and other users never see the temporary value.

Concepts used

Transactions as sandboxes, read-your-own-writes, isolation (other sessions keep seeing 10 throughout), ROLLBACK.

Things to watch out for

The query depends on today's log rows; if the procedure did not run today, it returns nothing.

7. Runner Script and SQL*Plus Commands

R1 · parts/run_all.sql — the master runner

The script

```
--  MASTER RUNNER SCRIPT : MODULAR EXECUTION
--  Railway Reservation and Passenger Service Management System
--  Executes modular parts in dependency order using SQL*Plus relative prefix (@@):
--    1. 01_ddl.sql     (Clean-up, Tables, Sequences, Indexes, Triggers)
--    2. 02_data.sql    (Static Reference & Operational Sample Data)
--    3. 02b_seat.sql   (Standalone / Idempotent Seat Population Check)
--    4. 03_dml.sql     (Payments, Cancellations, ALTER, DML, Tests, Procedures)
--    5. 04_queries.sql (Views, Queries Q1-Q10, Advanced Queries A1-A9, Risk Module)
-- ================================================================================
SET DEFINE OFF
SET SERVEROUTPUT ON SIZE UNLIMITED
SET LINESIZE 220
SET PAGESIZE 200
SET FEEDBACK ON
WHENEVER SQLERROR CONTINUE

PROMPT ================================================================================
PROMPT [STEP 1/5] Running 01_ddl.sql (Schema Creation and Constraints)...
PROMPT ================================================================================
@@01_ddl.sql

PROMPT ================================================================================
PROMPT [STEP 2/5] Running 02_data.sql (Loading Baseline Sample Data)...
PROMPT ================================================================================
@@02_data.sql

PROMPT ================================================================================
PROMPT [STEP 3/5] Running 02b_seat.sql (Idempotent Seat Validation)...
PROMPT ================================================================================
@@02b_seat.sql

PROMPT ================================================================================
PROMPT [STEP 4/5] Running 03_dml.sql (DML, Tests, Procedures, Demo)...
PROMPT ================================================================================
@@03_dml.sql

PROMPT ================================================================================
PROMPT [STEP 5/5] Running 04_queries.sql (Views, Analytics and Risk Module)...
PROMPT ================================================================================
@@04_queries.sql

PROMPT ================================================================================
PROMPT Master modular execution completed successfully.
PROMPT ================================================================================
```

Source: parts/run_all.sql, lines 2–45

What it is for

One line: runs the five part files in dependency order with sensible session settings.

Simple explanation

This file is the **start button** for the whole project. It sets up the screen output, then runs the five project files one after another in the right order: first build the tables, then load the data, then the changes and procedures, then the reports.

- `SET SERVEROUTPUT ON` lets you see messages printed by the procedures.
- `WHENEVER SQLERROR CONTINUE` tells it not to stop on an error (some tests are meant to fail).

- `@@file.sql` means "run this other file from the same folder".
- `PROMPT` just prints a heading so you can follow the progress.

RUNNING IN VS CODE (SQL DEVELOPER EXTENSION)

`run_all.sql` uses `@@`, so it must be run as a **script** (F5, *Run Script*), and its output appears in the *Script Output* tab, with DBMS_OUTPUT messages in the *DBMS Output* tab. To see one query as a grid in *Query Result*, open 04_queries.sql, place the cursor in the query and press Ctrl+Enter.